

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



HỒ NGUYỄN AN KHANG – 52200020

HỆ THỐNG CHẤM BÀI TRỰC TUYẾN -
TDTUOJ

KHÓA LUẬN TỐT NGHIỆP
KỸ THUẬT PHẦN MỀM

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



HỒ NGUYỄN AN KHANG – 52200020

HỆ THỐNG CHẤM BÀI TRỰC TUYẾN - TDTUOJ

KHÓA LUẬN TỐT NGHIỆP KỸ THUẬT PHẦN MỀM

Người hướng dẫn

ThS. Doãn Xuân Thanh

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

Công trình được hoàn thành tại **Trường Đại học Tôn Đức Thắng**.

Cán bộ hướng dẫn khoa học: **ThS. Doãn Xuân Thanh**

(Ghi rõ học hàm, học vị, họ tên và chữ ký)

Khóa luận được bảo vệ tại **Hội đồng đánh giá Khóa luận của Trường Đại học Tôn Đức Thắng** vào ngày tháng năm

Xác nhận của Chủ tịch Hội đồng đánh giá Khóa luận và Trưởng khoa quản lý chuyên ngành sau khi nhận Khóa luận đã được sửa chữa (nếu có).

CHỦ TỊCH HỘI ĐỒNG

TRƯỞNG KHOA

LỜI CẢM ƠN

Em xin chân thành cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng đã tận tình giảng dạy và trang bị cho em những kiến thức nền tảng vững chắc trong suốt quá trình học tập. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn **ThS. Doãn Xuân Thanh** đã dành thời gian định hướng, góp ý và hỗ trợ em trong từng giai đoạn thực hiện đề tài này. Bên cạnh đó, em xin cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện để em có thể hoàn thành tốt dự án.

TP. Hồ Chí Minh, ngày ... tháng ... năm 20...

Tác giả

(Ký tên và ghi rõ họ tên)

LỜI CAM ĐOAN

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của **ThS. Doãn Xuân Thanh**. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày ... tháng ... năm 20...

Tác giả

(Ký tên và ghi rõ họ tên)

HỆ THỐNG CHẤM BÀI TRỰC TUYẾN - TDTUOJ

TÓM TẮT

Dự án này trình bày quá trình nghiên cứu, thiết kế và triển khai ban đầu của Hệ thống Chấm bài Trực tuyến TDTUOJ – một nền tảng được xây dựng nhằm tự động hóa quy trình chấm bài lập trình và hỗ trợ tổ chức thi cử cho sinh viên Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng.

Hệ thống được thiết kế để phục vụ ba nhóm người dùng chính gồm sinh viên, giảng viên và quản trị viên, với các chức năng cốt lõi bao gồm nộp và chấm bài tự động cho nhiều ngôn ngữ lập trình, quản lý cuộc thi và bài toán, và phân tích hỗ trợ học tập thông minh bằng Large Language Model – Mô hình ngôn ngữ lớn (LLM). Kiến trúc hệ thống được xây dựng theo mô hình hai lớp tách biệt rõ ràng giữa phía máy chủ (backend) và phía giao diện người dùng (frontend), tích hợp với ba dịch vụ bên ngoài gồm Judging Service (Judge0), LLM và AI Parser Service.

Đến thời điểm báo cáo giữa kỳ, hệ thống đã hoàn thành phần lớn các chức năng cốt lõi. Phía backend được xây dựng bằng Java với Spring Boot, đã hiện thực hóa đầy đủ các Application Programming Interface (API) quản lý người dùng, quản lý bài toán và luồng xử lý bài nộp tích hợp Judge0. Phía frontend được xây dựng bằng React, đã hoàn thiện giao diện xem chi tiết bài toán theo slug, quản lý người dùng, tính năng nộp bài và hỗ trợ học tập bằng LLM.

ONLINE JUDGE SYSTEM - TDTUOJ

ABSTRACT

This project presents the research, design, and initial implementation of the TDTUOJ Online Judge System – a platform built to automate programming submission grading and support examination organization for students of the Faculty of Information Technology at Ton Duc Thang University.

The system is designed to serve three main user groups: students, lecturers, and administrators, with core features including automatic submission judging for multiple programming languages via Judge0, contest and problem management, and AI-powered learning support via LLM integration. The system architecture follows a clear two-layer separation between a Java Spring Boot backend and a React frontend, integrated with three external services: Judging Service (Judge0), LLM, and AI Parser Service.

As of the midterm report, the majority of core features have been implemented. The backend provides complete API coverage for user management, problem management, and Judge0-integrated submission processing. The frontend has completed the problem detail view by slug, user management interface, submission functionality, and LLM-powered hint and code explanation features.

MỤC LỤC

LỜI CẢM ƠN

LỜI CAM ĐOAN

TÓM TẮT

ABSTRACT

DANH MỤC HÌNH VẼ **vi**

DANH MỤC BẢNG BIỂU **viii**

DANH MỤC CÁC CHỮ VIẾT TẮT **ix**

1	Mở đầu và tổng quan đề tài	1
1.1	Lý do chọn đề tài	1
1.2	Khảo sát	1
1.3	Mục đích nghiên cứu	2
1.4	Phạm vi nghiên cứu	2
1.5	Ý nghĩa nghiên cứu	3
2	Kiến trúc nền tảng	4
2.1	Vòng đời phát triển phần mềm	4
2.2	Các công nghệ sử dụng	5
2.2.1	Phía máy chủ – Java và Spring Boot	5
2.2.2	Cơ sở dữ liệu – PostgreSQL	6
2.2.3	Công nghệ đóng gói – Docker	6
2.2.4	Hàng đợi và Bộ nhớ đệm – Redis	7
2.2.5	Dịch vụ chấm bài – Judge0	7

2.2.6	Phía giao diện – React	8
2.2.7	Lưu trữ file – AWS S3	8
2.2.8	Tích hợp LLM	9
2.2.9	Hạ tầng đám mây – Microsoft Azure	10
2.2.10	Tích hợp và triển khai liên tục – GitHub Actions	10
2.2.11	Kiểm thử tải và giám sát – k6, Prometheus và Grafana	11
3	Phân tích hệ thống	12
3.1	Thu thập yêu cầu	12
3.2	Xử lý nghiệp vụ	13
3.2.1	Các bên liên quan và thực thể nghiệp vụ	13
3.2.2	Các tác nhân hệ thống	15
3.3	Yêu cầu chức năng (Functional Requirements)	18
3.3.1	Tổng quan Use Case	18
3.3.2	Các Use Case chính	28
3.4	Yêu cầu phi chức năng (Non-functional Requirements)	51
3.4.1	Hiệu năng và thời gian phản hồi	52
3.4.2	Khả năng mở rộng	52
3.4.3	Bảo mật	52
3.4.4	Độ tin cậy và tính sẵn sàng	53
3.4.5	Khả năng bảo trì và khả năng sử dụng	53
4	Thiết kế hệ thống	54
4.1	Thiết kế kiến trúc	54
4.2	Thiết kế cơ sở dữ liệu / ERD	55
4.2.1	Tổng quan	55
4.2.2	Sơ đồ thực thể – quan hệ (ERD)	56
4.3	Thiết kế giao diện	69
4.3.1	Trang chi tiết bài toán và nộp bài	69

4.3.2	Trang cuộc thi và bảng xếp hạng	70
4.3.3	Trang quản trị	71
4.3.4	Trang trực quan hóa cấu trúc dữ liệu	72
4.3.5	Trang theo dõi tiến độ bài thực hành	73
4.4	Thiết kế hiệu năng và bảo mật	74
4.4.1	Xác thực và phân quyền bằng JWT	74
4.4.2	Giới hạn tần suất truy cập với Redis	76
5	Cài đặt và kiểm thử	78
5.1	Môi trường và công cụ cài đặt	78
5.2	Cài đặt hệ thống	79
5.2.1	Tổ chức mã nguồn theo tầng	79
5.2.2	Luồng nộp bài và chấm bài bất đồng bộ	80
5.2.3	Tính điểm đánh giá năng lực sau cuộc thi	80
5.3	Thiết kế API	82
5.3.1	Tổng quan các nhóm API	82
5.3.2	Công cụ tài liệu hóa và kiểm thử API	83
5.4	Mô tả các chức năng tiêu biểu	84
5.4.1	Chức năng nộp bài	84
5.4.2	Chức năng trực quan hóa cấu trúc dữ liệu	85
5.4.3	Chức năng hỗ trợ bằng mô hình ngôn ngữ lớn	86
5.5	Kiểm thử đơn vị	87
5.6	Kiểm thử tải và giám sát	88
6	Triển khai hệ thống	93
6.1	Hạ tầng và công cụ triển khai	93
6.2	Quy trình triển khai và tích hợp liên tục	94
6.3	Kết quả triển khai và chi phí vận hành	95

7 Kết luận	97
7.1 Kết quả đạt được	97
7.2 Hạn chế	98
7.3 Hướng phát triển	98
TÀI LIỆU THAM KHẢO	99

DANH MỤC HÌNH VẼ

3.1	Sơ đồ Use Case tổng quan của hệ thống TDTUOJ	27
3.2	Sơ đồ hoạt động cho Use Case Nộp bài giải	30
3.3	Sơ đồ tuần tự cho Use Case Nộp bài giải	31
3.4	Sơ đồ hoạt động cho Use Case Tạo bài toán	34
3.5	Sơ đồ tuần tự cho Use Case Tạo bài toán	35
3.6	Sơ đồ hoạt động cho Use Case Trực quan hóa và gỡ lỗi mã nguồn . . .	38
3.7	Sơ đồ tuần tự cho Use Case Trực quan hóa và gỡ lỗi mã nguồn	39
3.8	Sơ đồ hoạt động cho Use Case Tham gia cuộc thi	42
3.9	Sơ đồ tuần tự cho Use Case Tham gia cuộc thi	43
3.10	Sơ đồ hoạt động cho Use Case Gợi ý học tập bằng LLM	46
3.11	Sơ đồ tuần tự cho Use Case Gợi ý học tập bằng LLM	47
3.12	Sơ đồ hoạt động cho Use Case Quản lý bài thực hành	50
3.13	Sơ đồ tuần tự cho Use Case Quản lý bài thực hành	51
4.1	Kiến trúc tổng quan của hệ thống TDTUOJ	55
4.2	Sơ đồ thực thể – quan hệ của hệ thống TDTUOJ	57
4.3	Phân mảnh ERD – Người dùng	58
4.4	Phân mảnh ERD – Bài toán	60
4.5	Phân mảnh ERD – Bài nộp	62
4.6	Phân mảnh ERD – Cuộc thi	64
4.7	Phân mảnh ERD – Tổ chức	66
4.8	Phân mảnh ERD – Bài thực hành	68
4.9	Giao diện trang chi tiết bài toán và kết quả nộp bài	70
4.10	Giao diện trang cuộc thi và bảng xếp hạng	71
4.11	Giao diện trang quản trị bài toán	72
4.12	Giao diện trình minh họa cấu trúc dữ liệu	73

4.13	Giao diện trang theo dõi tiến độ bài thực hành	74
4.14	Sơ đồ tuần tự luồng xác thực và phân quyền bằng JSON Web Token (JWT)	75
5.1	Bảng điều khiển giám sát tổng quan trên Grafana	89
5.2	Số lượt chấp nhận và từ chối theo tầng khi bị dồn tần suất	91
5.3	Giám sát hệ thống trong kịch bản nộp bài	92

DANH MỤC BẢNG BIỂU

3.1	Các tác nhân của hệ thống TDTUOJ	15
3.2	Danh sách các Use Case của hệ thống TDTUOJ	18
3.3	Bảng đặc tả Use case Nộp bài giải	28
3.4	Bảng đặc tả Use case Tạo bài toán	31
3.5	Bảng đặc tả Use case Trực quan hóa và gỡ lỗi mã nguồn	36
3.6	Bảng đặc tả Use case Tham gia cuộc thi	40
3.7	Bảng đặc tả Use case Gợi ý học tập bằng LLM	44
3.8	Bảng đặc tả Use case Quản lý bài thực hành	48
4.1	Mô tả chi tiết thực thể users	58
4.2	Mô tả chi tiết thực thể problems	60
4.3	Mô tả chi tiết thực thể submissions	62
4.4	Mô tả chi tiết thực thể contests	64
4.5	Mô tả chi tiết thực thể organizations	67
4.6	Mô tả chi tiết thực thể labs	68
4.7	Cấu hình các bậc giới hạn tần suất truy cập	77
5.1	Các thành phần trong môi trường cài đặt	78
5.2	Các tầng trong mã nguồn phía máy chủ	79
5.3	Tổng quan các nhóm API của hệ thống TDTUOJ	82
5.4	Các endpoint API của chức năng nộp bài	84
5.5	Các endpoint API của chức năng hỗ trợ bằng LLM	86
5.6	Độ phủ dòng lệnh của các phân hệ dịch vụ cốt lõi	88
5.7	Một số trường hợp kiểm thử đơn vị tiêu biểu	88
5.8	Các chỉ số giám sát chính	90
6.1	Các thành phần hạ tầng triển khai chính	94

6.2	Chi phí vận hành các thành phần triển khai	96
-----	--	----

DANH MỤC CÁC CHỮ VIẾT TẮT

AC	Accepted
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface
CE	Compilation Error
CI/CD	Continuous Integration/Continuous Deployment – Tích hợp và triển khai liên tục
CNTT	Công nghệ Thông tin
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DNS	Domain Name System – Hệ thống phân giải tên miền
ERD	Entity-Relationship Diagram – Sơ đồ thực thể quan hệ
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ICPC	International Collegiate Programming Contest
JDK	Java Development Kit
JPA	Java Persistence API
JSON	JavaScript Object Notation
JVM	Java Virtual Machine – Máy ảo Java
JWT	JSON Web Token

LLM	Large Language Model – Mô hình ngôn ngữ lớn
MLE	Memory Limit Exceeded
CSV	Comma-Separated Values
MVC	Model-View-Controller
OJ	Online Judge
PDF	Portable Document Format
PK	Primary Key – Khóa chính
FK	Foreign Key – Khóa ngoại
RE	Runtime Error
REST	Representational State Transfer
SSH	Secure Shell
TDTU	Trường Đại học Tôn Đức Thắng
TLE	Time Limit Exceeded
TLS	Transport Layer Security
UI	User Interface
UX	User Experience
VM	Virtual Machine – Máy ảo
WA	Wrong Answer

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Hiện nay, việc tổ chức các kỳ thi lập trình và rèn luyện kỹ năng giải thuật cho sinh viên tại Trường Đại học Tôn Đức Thắng vẫn còn nhiều hạn chế về mặt công cụ hỗ trợ. Chấm bài thủ công tiêu tốn nhiều thời gian và công sức của giảng viên, đồng thời khó đảm bảo tính nhất quán khi số lượng sinh viên ngày càng tăng. Bên cạnh đó, sinh viên thiếu một môi trường luyện tập chủ động với phản hồi tức thì để tự nâng cao kỹ năng lập trình ngoài giờ học.

Để giải quyết những vấn đề này, đề tài đề xuất xây dựng Hệ thống Chấm bài Trực tuyến TDTUOJ – một nền tảng tự động hóa toàn bộ quy trình chấm bài, hỗ trợ tổ chức thi cử quy mô lớn và tạo ra môi trường luyện tập lập trình chuyên nghiệp trong khuôn khổ học thuật của nhà trường.

1.2 Khảo sát

Trước khi xây dựng TDTUOJ, nhóm đã tiến hành khảo sát một số hệ thống Online Judge (OJ) phổ biến hiện có nhằm đánh giá ưu nhược điểm và xác định khoảng trống mà hệ thống cần lấp đầy.

Codeforces là một trong những nền tảng thi lập trình lớn nhất thế giới, nổi bật với hệ thống rating Elo uy tín, kho bài toán phong phú và hạ tầng chịu tải cao. Tuy nhiên, Codeforces hoàn toàn là nền tảng công cộng và không cung cấp cơ chế quản lý lớp học, phân quyền theo tổ chức hay tích hợp với quy trình đào tạo nội bộ của một trường đại học cụ thể.

LeetCode tập trung vào luyện tập kỹ năng phỏng vấn với giao diện hiện đại và tính năng giải thích lời giải chi tiết. Mặc dù vậy, nền tảng này không hỗ trợ tổ chức thi cử theo lịch học, không có cơ chế phân quyền giảng viên – sinh viên và không thể tùy biến theo nhu cầu học thuật nội bộ.

VNOJ (Vietnam Online Judge) là hệ thống OJ mã nguồn mở phổ biến tại Việt

Nam, hỗ trợ tiếng Việt và có cộng đồng người dùng trong nước lớn. Dù VNOJ phù hợp cho luyện tập cá nhân và một số kỳ thi học sinh sinh viên, hệ thống vẫn thiếu các tính năng nâng cao như hỗ trợ học tập bằng LLM, nhập đề bài tự động từ Portable Document Format (PDF) và cơ chế quản lý tổ chức linh hoạt theo mô hình đại học.

Qua khảo sát, nhóm nhận thấy các hệ thống hiện có đều không được thiết kế để phục vụ nhu cầu đặc thù của một đơn vị đào tạo cụ thể: quản lý phân quyền theo vai trò giảng viên và sinh viên, tích hợp Artificial Intelligence – Trí tuệ nhân tạo (AI) hỗ trợ học tập trực tiếp trong luồng nộp bài, và khả năng vận hành độc lập trong môi trường nội bộ của nhà trường. Đây chính là khoảng trống mà TDTUOJ được xây dựng để lấp đầy.

1.3 Mục đích nghiên cứu

TDTUOJ được xây dựng nhằm đạt một tập hợp các mục tiêu cụ thể và có tính ứng dụng cao. Về mặt kỹ thuật, hệ thống hỗ trợ đa ngôn ngữ lập trình, cho phép người dùng nộp bài viết bằng C, C++, Java và Python, đáp ứng nhu cầu đa dạng của sinh viên ở nhiều trình độ và môn học khác nhau. Song song với đó, hệ thống cung cấp đầy đủ công cụ để quản lý cuộc thi và bài tập, bao gồm tổ chức cuộc thi, quản lý danh sách bài toán, chấm điểm tự động và xếp hạng người tham gia theo thời gian thực.

Ngoài các chức năng cốt lõi, hệ thống tích hợp một số tính năng nâng cao phục vụ học tập. Tính năng phân tích mã nguồn bằng LLM cho phép giải thích mã nguồn, gợi ý hướng giải và phân tích độ phức tạp thuật toán ngay trên giao diện – giúp sinh viên hiểu rõ hơn về hiệu suất và logic của giải pháp mình đề xuất. Ngoài ra, hệ thống tích hợp AI Parser Service để tự động đọc đề bài từ file PDF và chuyển đổi thành dữ liệu có cấu trúc, giúp giảng viên tiết kiệm đáng kể thời gian soạn thảo.

1.4 Phạm vi nghiên cứu

Đề tài tập trung xây dựng một hệ thống hoàn chỉnh phục vụ sinh viên và giảng viên Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng. Hệ thống quản lý ba vai trò với phân quyền rõ ràng: User (người dùng thông thường), Contest Creator

(người tổ chức và quản lý cuộc thi), và Admin (quản trị viên với toàn quyền điều hành). Về mặt kiến trúc, phạm vi đề tài bao gồm phát triển một backend Representational State Transfer (REST)ful API bằng Java Spring Boot, một giao diện người dùng bằng React, và tích hợp ba dịch vụ bên ngoài: Judging Service (Judge0), LLM và AI Parser Service.

Những nội dung nằm ngoài phạm vi của đề tài bao gồm: phát triển ứng dụng di động, tích hợp hệ thống quản lý học tập (LMS) hiện có của nhà trường, xây dựng hạ tầng chấm bài tùy chỉnh thay thế Judge0, và hỗ trợ các ngôn ngữ lập trình ngoài bốn ngôn ngữ đã nêu ở giai đoạn đầu.

1.5 Ý nghĩa nghiên cứu

Về ý nghĩa thực tiễn, với sinh viên, hệ thống tạo ra môi trường luyện tập lập trình với phản hồi tức thì, phân tích thuật toán và hỗ trợ học tập từ LLM, giúp người học phát triển tư duy giải quyết vấn đề một cách chủ động. Với giảng viên, hệ thống giải phóng nguồn lực khỏi các công việc lặp lại như chấm bài thủ công và soạn thảo đề bài, từ đó tập trung nhiều hơn vào thiết kế nội dung giảng dạy.

Về ý nghĩa khoa học, đề tài đặt ra và giải quyết một số bài toán kỹ thuật có giá trị nghiên cứu: xây dựng luồng xử lý bài nộp bất đồng bộ có khả năng mở rộng theo chiều ngang, tích hợp LLM vào quy trình hỗ trợ học tập, và áp dụng bộ nhớ đệm để phục vụ bảng xếp hạng thời gian thực. Ngoài ra, TDTUOJ góp phần xây dựng phong trào lập trình thi đấu tại Trường Đại học Tôn Đức Thắng (TDTU).

CHƯƠNG 2. KIẾN TRÚC NỀN TẢNG

2.1 Vòng đời phát triển phần mềm

Để phù hợp với bối cảnh phát triển cá nhân, yêu cầu hệ thống có thể thay đổi trong quá trình triển khai và thời gian thực hiện có hạn của một đề tài tốt nghiệp, dự án TDTUOJ áp dụng phương pháp phát triển theo hướng Agile kết hợp với mô hình quản lý công việc bằng Kanban board trên nền tảng Trello.

Thay vì chia dự án thành các sprint có độ dài cố định như Scrum, phương pháp tiếp cận được chọn là luồng công việc liên tục (continuous flow): các hạng mục công việc được đưa vào thực hiện tuần tự dựa trên mức độ ưu tiên, và tiến độ được điều chỉnh linh hoạt theo từng giai đoạn phát triển thực tế. Đây là lựa chọn thực tế và hiệu quả cho một dự án cá nhân, nơi việc tổ chức hợp sprint hay điều phối nhóm không phát sinh và mọi quyết định kỹ thuật được đưa ra trực tiếp dựa trên kết quả của từng tác vụ đã hoàn thành.

Kanban board trong Trello được tổ chức thành bốn cột phản ánh vòng đời của từng hạng mục công việc: **Backlog** (đã xác định nhưng chưa lên lịch), **To Do** (đã phân tích và sẵn sàng thực hiện), **Doing** (đang triển khai, có giới hạn số lượng công việc đồng thời) và **Done** (đã hoàn thành, kiểm thử và tích hợp vào nhánh chính). Mỗi hạng mục trong Backlog được mô tả đủ chi tiết để ước lượng thời gian và xác định phụ thuộc kỹ thuật trước khi chuyển sang To Do, nhờ đó luôn rõ ràng về công việc tiếp theo và có cơ sở hữu hình để đánh giá tiến độ so với kế hoạch tổng thể.

Quá trình phát triển được chia thành ba giai đoạn lớn, phản ánh thứ tự ưu tiên kỹ thuật từ nền tảng đến tính năng nâng cao. Giai đoạn đầu tập trung vào việc xây dựng nền tảng kiến trúc và các tính năng cốt lõi: thiết lập cấu trúc dự án Spring Boot, cấu hình PostgreSQL và Docker Compose, hiện thực hóa hệ thống xác thực và phân quyền bằng JWT, và hoàn thiện luồng nộp bài tích hợp Judge0. Giai đoạn thứ hai mở rộng sang quản lý cuộc thi và bảng xếp hạng: xây dựng các API quản lý contest, tích hợp Redis cho submission queue và leaderboard cache, và phát triển giao diện

frontend tương ứng bằng React. Giai đoạn thứ ba bổ sung các tính năng nâng cao: tích hợp LLM cho hỗ trợ học tập, xây dựng AI Parser Service để nhập đề bài từ PDF, và hoàn thiện giao diện quản trị.

2.2 Các công nghệ sử dụng

Hệ thống TDTUOJ được xây dựng trên một tập hợp các công nghệ được lựa chọn có chủ đích, đảm bảo sự cân bằng giữa hiệu năng, khả năng bảo trì và tính phù hợp với quy mô của một dự án học thuật. Phần này trình bày lần lượt từng công nghệ, lý do lựa chọn và vai trò cụ thể của nó trong kiến trúc tổng thể.

2.2.1 Phía máy chủ – Java và Spring Boot

Phía máy chủ (backend) của TDTUOJ được xây dựng bằng ngôn ngữ Java với nền tảng phát triển ứng dụng (framework) Spring Boot. Spring Boot là một framework phát triển ứng dụng doanh nghiệp trưởng thành, cung cấp cơ chế cấu hình tự động (auto-configuration) giúp rút ngắn đáng kể thời gian thiết lập dự án so với Spring thuần túy, đồng thời tích hợp sẵn một hệ sinh thái phong phú gồm Spring Security, Spring Data Java Persistence API (JPA) và Spring Web.

Kiến trúc backend tuân theo mô hình Model-View-Controller (MVC) phân lớp rõ ràng. Lớp Controller tiếp nhận các yêu cầu Hypertext Transfer Protocol (HTTP) và định tuyến đến lớp Service tương ứng. Lớp Service chứa toàn bộ logic nghiệp vụ, thực hiện xác thực dữ liệu và điều phối các thao tác cần thiết. Lớp Repository, được hiện thực hóa thông qua Spring Data JPA, chịu trách nhiệm tương tác với cơ sở dữ liệu thông qua các entity và query có kiểu dữ liệu tường minh.

Bảo mật được xử lý thông qua Spring Security kết hợp với JWT. Khi người dùng đăng nhập thành công, hệ thống tạo ra một access token có thời hạn ngắn và một refresh token có thời hạn dài hơn. Mọi yêu cầu API sau đó cần đính kèm access token trong header Authorization; Spring Security sẽ xác thực token, giải mã thông tin người dùng và kiểm tra quyền hạn trước khi cho phép yêu cầu tiếp tục xử lý. Cơ chế này đảm bảo hệ thống hoàn toàn không lưu trạng thái phiên (stateless) ở phía

server, phù hợp với kiến trúc RESTful và dễ dàng mở rộng theo chiều ngang khi cần.

2.2.2 Cơ sở dữ liệu – PostgreSQL

PostgreSQL được chọn làm hệ quản trị cơ sở dữ liệu chính của hệ thống. So với MySQL, PostgreSQL cung cấp hỗ trợ tốt hơn cho các kiểu dữ liệu phức tạp như JSONB, kiểu mảng và các biểu thức chỉ mục nâng cao – những tính năng phát huy giá trị trong ngữ cảnh lưu trữ cấu hình bài toán và kết quả chấm bài đa dạng. Ngoài ra, PostgreSQL có cơ chế xử lý giao dịch đồng thời (MVCC) mạnh mẽ, đảm bảo tính toàn vẹn dữ liệu trong các tình huống nhiều bài nộp được xử lý đồng thời.

Kết nối giữa Spring Boot và PostgreSQL được quản lý thông qua HikariCP – bể kết nối (connection pool) mặc định của Spring Boot, nổi bật với hiệu năng cao và chi phí phụ trội (overhead) thấp. Các tham số pool như kích thước tối đa, thời gian chờ kết nối và thời gian nhàn rỗi được cấu hình phù hợp với tải dự kiến của hệ thống, đảm bảo việc tái sử dụng kết nối hiệu quả và tránh tình trạng cạn kiệt kết nối trong giờ cao điểm.

2.2.3 Công nghệ đóng gói – Docker

Toàn bộ hệ thống TDTUOJ được đóng gói và triển khai bằng Docker, với Docker Compose đóng vai trò điều phối các service chạy cùng nhau trong một môi trường thống nhất. Cách tiếp cận này mang lại lợi thế quan trọng là tính nhất quán giữa môi trường phát triển và môi trường triển khai: mọi thành phần từ backend, database đến judging service đều chạy trong container có cấu hình xác định, loại bỏ hoàn toàn các vấn đề phụ thuộc môi trường.

Hạng mục đặc biệt cần lưu ý là Judge0 – dịch vụ chấm bài mã nguồn mở – được tự triển khai (self-hosted) thông qua Docker Compose thay vì sử dụng API công cộng có phí. Judge0 yêu cầu quyền truy cập đặc quyền (privileged container) để có thể tạo môi trường sandbox cô lập khi thực thi mã nguồn người dùng, và Docker cung cấp đúng cấp độ kiểm soát cần thiết cho yêu cầu này. Việc tự triển khai Judge0 cũng đảm bảo hệ thống hoạt động hoàn toàn nội bộ mà không phụ thuộc vào dịch vụ bên

ngoài, phù hợp với yêu cầu vận hành độc lập của một nền tảng học thuật.

2.2.4 Hàng đợi và Bộ nhớ đệm – Redis

Redis được tích hợp vào hệ thống để phục vụ hai mục đích có tính chất khác nhau nhưng đều đòi hỏi tốc độ truy cập cao.

Mục đích thứ nhất là hàng đợi bài nộp (submission queue). Khi người dùng nộp bài, yêu cầu không được gửi đồng bộ trực tiếp đến Judge0 mà được đặt vào hàng đợi Redis. Một tiến trình xử lý riêng biệt (worker) lấy yêu cầu từ hàng đợi và gửi đến Judge0 theo thứ tự, sau đó cập nhật kết quả vào cơ sở dữ liệu khi có phản hồi. Cơ chế này tách biệt luồng tiếp nhận yêu cầu khỏi luồng xử lý chấm bài, giúp hệ thống phản hồi người dùng ngay lập tức và xử lý bài nộp bất đồng bộ một cách có kiểm soát, đặc biệt quan trọng trong các thời điểm nhiều người nộp bài đồng thời.

Mục đích thứ hai là cache bảng xếp hạng (leaderboard cache). Dữ liệu bảng xếp hạng của một cuộc thi cần được truy vấn thường xuyên và có tính chất thay đổi theo từng lần nộp bài thành công – việc tính toán lại từ cơ sở dữ liệu quan hệ mỗi lần có yêu cầu sẽ gây tải không cần thiết. Redis lưu trữ bảng xếp hạng đã tính sẵn và chỉ làm mới khi có bài nộp mới ảnh hưởng đến thứ hạng, giảm đáng kể số lượng truy vấn nặng đến PostgreSQL.

2.2.5 Dịch vụ chấm bài – Judge0

Judge0 là dịch vụ thực thi mã nguồn mã nguồn mở, cung cấp khả năng biên dịch và chạy mã trong môi trường sandbox cô lập với giới hạn thời gian và bộ nhớ được kiểm soát chặt chẽ. Judge0 hỗ trợ hơn 60 ngôn ngữ lập trình, trong đó TDTUOJ hiện khai thác bốn ngôn ngữ chính gồm C, C++, Java và Python.

Luồng tích hợp hoạt động như sau: sau khi một yêu cầu chấm bài được lấy ra từ hàng đợi Redis, hệ thống gửi mã nguồn cùng với các test case tương ứng đến API của Judge0 thông qua giao thức HTTP. Judge0 thực thi mã trong container sandbox, so sánh đầu ra với đáp án mẫu và trả về kết quả bao gồm trạng thái (Accepted (AC), Wrong Answer (WA), Time Limit Exceeded (TLE), Memory Limit Exceeded (MLE),

Runtime Error (RE), Compilation Error (CE)) cùng với thời gian thực thi và lượng bộ nhớ sử dụng. Hệ thống sau đó cập nhật bản ghi submission trong PostgreSQL và thông báo kết quả đến frontend thông qua cơ chế polling hoặc WebSocket.

2.2.6 Phía giao diện – React

Giao diện người dùng (frontend) của TDTUOJ được xây dựng bằng React – thư viện JavaScript phổ biến cho phép xây dựng giao diện theo mô hình thành phần (component) và quản lý trạng thái ứng dụng một cách có cấu trúc. React cho phép xây dựng ứng dụng trang đơn (Single-Page Application) trong đó việc điều hướng giữa các trang diễn ra hoàn toàn ở phía client mà không cần tải lại trang, mang lại trải nghiệm người dùng mượt mà và phản hồi nhanh.

Giao tiếp giữa frontend và backend được thực hiện thông qua các RESTful API HTTP. Toàn bộ dữ liệu trao đổi sử dụng định dạng JavaScript Object Notation (JSON), và access token JWT được đính kèm trong header của mỗi yêu cầu cần xác thực. Cấu trúc component của ứng dụng được tổ chức theo từng domain chức năng – người dùng, bài toán, cuộc thi, bài nộp – giúp việc bảo trì và mở rộng tính năng trở nên thuận tiện.

2.2.7 Lưu trữ file – AWS S3

Hệ thống sử dụng AWS S3 (Simple Storage Service) để lưu trữ các tệp nội dung bài toán thay vì nhúng trực tiếp vào cơ sở dữ liệu quan hệ. Cụ thể, S3 được dùng để lưu trữ hai loại nội dung chính: tệp Markdown (.md) chứa đề bài của từng bài toán, và các tệp văn bản (.txt) chứa dữ liệu đầu vào và đầu ra kỳ vọng của từng test case.

Việc tách nội dung đề bài và test case ra khỏi PostgreSQL mang lại một số lợi thế thực tế. Đề bài dạng Markdown có thể chứa các đoạn mô tả dài, công thức toán học và ví dụ minh họa – những nội dung có kích thước biến động lớn và không cần lập chỉ mục tìm kiếm ở cấp cơ sở dữ liệu. Tương tự, dữ liệu test case có thể bao gồm đầu vào nhiều dòng với kích thước đáng kể tùy theo bài toán; lưu trữ chúng trên S3 giúp tránh làm phình to bảng dữ liệu và đơn giản hóa quy trình thêm hoặc cập nhật

test case mà không cần thay đổi schema.

Chiến lược lưu trữ trên S3 được tổ chức theo cấu trúc key có phân cấp rõ ràng, phân tách theo loại nội dung và định danh bài toán, giúp việc truy xuất và quản lý vòng đời của tệp được thực hiện một cách hệ thống. Backend chỉ lưu trữ đường dẫn S3 tham chiếu trong PostgreSQL; khi cần phục vụ nội dung cho người dùng hoặc cho Judge0, hệ thống đọc trực tiếp nội dung tệp từ S3 thông qua API nội bộ. Về định hướng phát triển, hệ thống có thể mở rộng để hỗ trợ lưu trữ tệp PDF đề bài gốc khi tính năng nhập đề từ PDF được tích hợp đầy đủ trong tương lai.

2.2.8 Tích hợp LLM

TDTUOJ tích hợp LLM như một lớp hỗ trợ học tập thông minh, được thiết kế để bổ sung vào luồng giải bài của sinh viên mà không thay thế quá trình tư duy độc lập. Tích hợp này hoạt động theo mô hình proxy: backend đóng vai trò trung gian giữa frontend và LLM API bên ngoài, cho phép kiểm soát nội dung yêu cầu, giới hạn tần suất gọi và ghi nhật ký sử dụng.

Hệ thống cung cấp hai nhóm tính năng hỗ trợ học tập chính. Nhóm tính năng thứ nhất phục vụ sinh viên trực tiếp trong quá trình giải bài: tính năng **gợi ý hướng giải** (hint generation) cung cấp một gợi ý có định hướng mà không tiết lộ lời giải hoàn chỉnh, giúp sinh viên tự tìm ra bước tiếp theo; tính năng **phân tích độ phức tạp** (complexity analysis) nhận vào đoạn mã nguồn và bài toán tương ứng, sau đó trả về đánh giá độ phức tạp thời gian và không gian kèm theo giải thích chi tiết. Nhóm tính năng thứ hai phục vụ giảng viên trong quá trình soạn thảo nội dung: tính năng **AI Parser** cho phép tải lên tệp PDF đề bài, LLM sẽ phân tích cấu trúc tài liệu và tự động điền các trường thông tin bài toán bao gồm tiêu đề, mô tả, định dạng đầu vào – đầu ra và ví dụ mẫu, giúp giảng viên tiết kiệm đáng kể thời gian nhập liệu thủ công.

Tất cả các yêu cầu đến LLM đều được xây dựng với prompt có cấu trúc rõ ràng ở phía backend, trong đó ngữ cảnh bài toán (đề bài, giới hạn, ví dụ) được đưa vào system prompt để đảm bảo phản hồi của LLM luôn liên quan và chính xác theo từng bài toán cụ thể.

2.2.9 Hạ tầng đám mây – Microsoft Azure

Hệ thống được triển khai trên hạ tầng đám mây (cloud) của Microsoft Azure, sử dụng một máy chủ ảo (virtual machine) chạy hệ điều hành Ubuntu. Phương án máy chủ ảo được lựa chọn thay cho các dịch vụ nền tảng (Platform as a Service) bởi đặc thù của Judge0: dịch vụ chấm bài này cần chạy ở chế độ đặc quyền (privileged container) và yêu cầu trình quản lý tài nguyên ở mức nhân hệ điều hành (cgroup) để giới hạn bộ nhớ cho từng lần thực thi mã – những quyền kiểm soát mà các nền tảng đóng gói sẵn thường không cho phép. Máy chủ ảo trao toàn quyền quản trị hệ điều hành, nhờ đó nhóm tác giả có thể tinh chỉnh nhân Linux để Judge0 vận hành ổn định.

Trên máy chủ, toàn bộ các thành phần phía máy chủ (backend, PostgreSQL, Redis, Judge0) được dựng bằng Docker Compose, đặt sau một máy chủ proxy ngược (reverse proxy) Caddy đảm nhiệm việc cấp và gia hạn chứng chỉ Hypertext Transfer Protocol Secure (HTTPS) tự động cho tên miền `api.tdtuoj.me`. Riêng phía giao diện người dùng (frontend) được triển khai tách biệt trên nền tảng Vercel. Chi tiết quy trình và kết quả triển khai được trình bày ở Chương 6.

2.2.10 Tích hợp và triển khai liên tục – GitHub Actions

Để rút ngắn chu kỳ đưa thay đổi từ mã nguồn lên môi trường vận hành, hệ thống áp dụng quy trình tích hợp liên tục và triển khai liên tục (Continuous Integration / Continuous Deployment – Continuous Integration/Continuous Deployment – Tích hợp và triển khai liên tục (CI/CD)) bằng GitHub Actions. Quy trình được kích hoạt tự động mỗi khi có thay đổi được đẩy lên nhánh chính, và gồm hai giai đoạn nối tiếp nhau.

Giai đoạn thứ nhất là cổng kiểm tra biên dịch: mã nguồn phía máy chủ được biên dịch bằng Maven trên môi trường Java Development Kit (JDK) 21 và chạy bộ kiểm thử đơn vị, nhằm bảo đảm chỉ những thay đổi vượt qua kiểm tra mới được phép triển khai. Giai đoạn thứ hai thực hiện triển khai: hệ thống kết nối an toàn qua Secure Shell (SSH) vào máy chủ ảo, kéo về phiên bản mã nguồn mới nhất và dựng lại các

container bằng Docker Compose. Cách tổ chức này loại bỏ thao tác triển khai thủ công, giảm thiểu sai sót và bảo đảm môi trường vận hành luôn đồng bộ với nhánh chính.

2.2.11 Kiểm thử tải và giám sát – k6, Prometheus và Grafana

Nhằm đánh giá hiệu năng của hệ thống dưới điều kiện nhiều người dùng truy cập đồng thời, nhóm tác giả sử dụng công cụ k6 để mô phỏng tải. Các kịch bản kiểm thử tải tái hiện những tình huống thực tế như duyệt nội dung, nộp bài đồng loạt và kiểm tra cơ chế giới hạn tần suất truy cập, qua đó xác định ngưỡng chịu tải và các điểm nghẽn của hệ thống.

Song song với kiểm thử tải, hệ thống được giám sát bằng bộ đôi Prometheus và Grafana. Prometheus định kỳ thu thập các số liệu (metrics) do phía máy chủ phát ra, bao gồm cả các số liệu nghiệp vụ tùy biến như độ sâu hàng đợi bài nộp và thời gian chấm bài; Grafana truy vấn và trực quan hóa các số liệu này thành các bảng điều khiển (dashboard) theo thời gian thực. Sự kết hợp giữa k6 và bộ giám sát cho phép quan sát trực tiếp phản ứng của hệ thống khi tải tăng cao. Kết quả kiểm thử tải và giám sát được phân tích chi tiết ở Chương 5.

CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG

3.1 Thu thập yêu cầu

Thu thập yêu cầu là giai đoạn nền tảng của quy trình phát triển phần mềm: chất lượng sản phẩm cuối cùng phụ thuộc trực tiếp vào mức độ đầy đủ và chính xác của các yêu cầu được xác định từ đầu. Trong đề tài này, hai kỹ thuật chính được áp dụng để thu thập yêu cầu là phỏng vấn và khảo sát. Đối với kỹ thuật phỏng vấn, nhằm có được cái nhìn đầy đủ từ cả hai phía – người học và người giảng dạy – tác giả đã tiến hành phỏng vấn trực tiếp với một số sinh viên đang theo học tại Khoa Công nghệ Thông tin và một giảng viên phụ trách các học phần lập trình. Để đảm bảo tính hệ thống và đầy đủ của thông tin, các câu hỏi phỏng vấn được chuẩn bị trước theo cấu trúc có định hướng, tập trung vào những khó khăn thực tế trong quá trình học tập, rèn luyện kỹ năng lập trình và tham gia các kỳ thi lập trình nội bộ. Các buổi phỏng vấn được thực hiện theo hình thức trao đổi trực tiếp, cho phép tác giả đặt thêm câu hỏi làm rõ khi cần thiết, từ đó nắm bắt được những nhu cầu tiềm ẩn mà người dùng có thể không diễn đạt trực tiếp trong phiếu khảo sát.

Bên cạnh phỏng vấn, việc nói chuyện trực tiếp với một số cá nhân tuy cung cấp thông tin có chiều sâu nhưng chưa đủ để phản ánh ý kiến của toàn bộ cộng đồng sinh viên. Vì vậy, kỹ thuật khảo sát trực tuyến cũng được áp dụng nhằm nâng cao độ tin cậy và tính đại diện của dữ liệu thu thập được. Bảng khảo sát được thiết kế với các câu hỏi rõ ràng, dễ hiểu, hướng đến các vấn đề phổ biến mà sinh viên gặp phải khi luyện tập lập trình và thi cử, đồng thời khảo sát mức độ quan tâm đối với các tính năng hỗ trợ học tập thông minh như gợi ý thuật toán hay phân tích mã nguồn bằng AI. Khảo sát được phân phối đến sinh viên từ nhiều khóa học và năm học khác nhau nhằm đảm bảo sự đa dạng trong phản hồi.

Kết luận: Sau khi tổng hợp kết quả từ phỏng vấn và khảo sát, tác giả nhận thấy rằng phần lớn sinh viên gặp khó khăn trong việc tìm kiếm môi trường luyện tập lập trình có phản hồi tức thì, đặc biệt ngoài giờ học chính khóa. Các giảng viên cũng bày tỏ

nhu cầu cần một công cụ chấm bài tự động, giảm thiểu thời gian xử lý thủ công và hỗ trợ tổ chức thi cử quy mô lớn một cách hiệu quả. Đây là cơ sở thực tiễn trực tiếp cho việc xác định các yêu cầu chức năng và phi chức năng của hệ thống TDTUOJ được trình bày trong các phần tiếp theo.

3.2 Xử lý nghiệp vụ

Dựa trên kết quả thu thập yêu cầu từ các buổi phỏng vấn và khảo sát, phần này trình bày phân tích chi tiết về các quy trình nghiệp vụ trọng tâm và các bên liên quan chính trong hệ thống.

3.2.1 Các bên liên quan và thực thể nghiệp vụ

3.2.1.1 Bài toán (Problem)

Bài toán là đối tượng trung tâm của hệ thống. Mỗi bài toán được sở hữu và quản lý bởi người dùng có vai trò Contest Creator hoặc Admin. Quy trình quản lý bài toán có sự phân cấp rõ ràng: bài toán có thể được tạo mới, chỉnh sửa và xóa bởi người có thẩm quyền, đồng thời phải được cấu hình đầy đủ về giới hạn thời gian, giới hạn bộ nhớ, danh sách test case và đề bài trước khi đưa vào sử dụng trong cuộc thi hoặc công khai cho sinh viên luyện tập. Khi bài toán được công khai, sinh viên có thể tìm kiếm, xem chi tiết và nộp bài giải trực tiếp trên hệ thống.

Để bảo đảm tính công bằng, vòng đời hiển thị của bài toán được kiểm soát chặt chẽ: chỉ những bài toán riêng tư, do chính người tạo sở hữu và chưa có lượt nộp từ người dùng khác mới được phép đưa vào cuộc thi; bài toán riêng tư đang thuộc một cuộc thi chưa kết thúc không thể được công khai thủ công. Khi cuộc thi kết thúc, hệ thống tự động chuyển các bài toán của cuộc thi sang trạng thái công khai để bổ sung vào kho luyện tập chung. Bài toán dùng trong bài thực hành (lab) cũng phải là bài riêng tư thuộc sở hữu của giảng viên, nhưng được giữ nguyên trạng thái riêng tư sau khi hết thời hạn.

3.2.1.2 Bài nộp (Submission)

Bài nộp được tạo ra khi sinh viên gửi lời giải cho một bài toán. Mỗi bài nộp ghi lại đầy đủ thông tin gồm mã nguồn, ngôn ngữ lập trình được chọn, thời điểm nộp và kết quả chấm bài. Kết quả của bài nộp phản ánh mức độ chính xác và hiệu quả của lời giải thông qua một trong các trạng thái: AC, WA, TLE, MLE, RE hoặc CE. Dữ liệu bài nộp được lưu trữ lâu dài và là cơ sở để tính toán bảng xếp hạng trong các cuộc thi, cũng như thống kê tiến độ học tập của từng sinh viên.

3.2.1.3 Cuộc thi (Contest)

Cuộc thi là đơn vị tổ chức thi cử trong hệ thống, được tạo và quản lý bởi người dùng có vai trò Contest Creator hoặc Admin. Một cuộc thi bao gồm tập hợp các bài toán riêng tư được chọn lọc từ kho đề của người tạo, thời gian bắt đầu và kết thúc xác định, cũng như danh sách người tham gia đã đăng ký. Trong suốt thời gian diễn ra cuộc thi, bảng xếp hạng được cập nhật theo thời gian thực dựa trên kết quả bài nộp của từng thí sinh; bài nộp của mỗi thí sinh được ẩn khỏi những người dùng khác nhằm ngăn việc tham khảo lời giải giữa chừng. Người tạo cuộc thi có thể cấu hình khoảng đóng băng bảng xếp hạng (scoreboard freeze) theo thể thức International Collegiate Programming Contest (ICPC): trong những phút cuối của cuộc thi, người xem thông thường chỉ thấy ảnh chụp bảng xếp hạng tại thời điểm bắt đầu đóng băng, trong khi người tạo cuộc thi và Admin vẫn theo dõi được diễn biến trực tiếp. Sau khi cuộc thi kết thúc, hệ thống tự động tính toán điểm xếp hạng, mở lại quyền xem bài nộp, công khai các bài toán của cuộc thi và lưu kết quả để phục vụ mục đích thống kê và báo cáo.

3.2.1.4 Kết quả chấm bài (Judge Result)

Kết quả chấm bài được tạo ra bởi Judge0 sau khi xử lý một bài nộp. Đây là đầu ra của quá trình thực thi mã nguồn trong môi trường sandbox có kiểm soát, bao gồm trạng thái kết quả, thời gian thực thi thực tế và lượng bộ nhớ tiêu thụ trên từng test case. Kết quả này được hệ thống tiếp nhận qua cơ chế bất đồng bộ, cập nhật vào

cơ sở dữ liệu và phản hồi đến người dùng thông qua giao diện frontend.

3.2.2 Các tác nhân hệ thống

Hệ thống TDTUOJ được thiết kế để phục vụ ba nhóm tác nhân người dùng với phân quyền rõ ràng, cùng với ba hệ thống bên ngoài đóng vai trò dịch vụ hỗ trợ. Bảng 3.1 mô tả chi tiết từng tác nhân và vai trò tương ứng trong hệ thống.

Bảng 3.1: Các tác nhân của hệ thống TDTUOJ

STT	Tác nhân	Mô tả
1	User (Người dùng)	Đây là tác nhân cơ bản nhất trong hệ thống, đại diện cho sinh viên hoặc người học lập trình nói chung. Người dùng có thể đăng ký tài khoản, đăng nhập, tìm kiếm và xem chi tiết bài toán, nộp bài giải và xem lịch sử bài nộp của bản thân. Người dùng cũng có thể đăng ký tham gia các cuộc thi đang diễn ra, xem bảng xếp hạng và sử dụng các tính năng hỗ trợ học tập do LLM cung cấp như gợi ý hướng giải và phân tích độ phức tạp thuật toán.

(tiếp theo Bảng 3.1)

STT	Tác nhân	Mô tả
2	Contest Creator (Người tổ chức)	Tác nhân này chịu trách nhiệm tổ chức và quản lý các hoạt động thi cử và nội dung bài toán trong hệ thống. Contest Creator có thể tạo mới, chỉnh sửa và xóa bài toán, quản lý danh sách test case, tổ chức và điều hành các cuộc thi bao gồm cấu hình thời gian, danh sách bài toán và người tham gia. Ngoài ra, Contest Creator có thể sử dụng tính năng AI Parser để nhập đề bài tự động từ tệp PDF, giúp tiết kiệm thời gian soạn thảo nội dung một cách đáng kể.
3	Admin (Quản trị viên)	Đây là tác nhân có quyền hạn cao nhất trong hệ thống, chịu trách nhiệm vận hành và quản trị toàn diện nền tảng. Admin có toàn quyền đối với tài khoản người dùng bao gồm tạo mới, chỉnh sửa, vô hiệu hóa hoặc thay đổi vai trò của bất kỳ tài khoản nào trong hệ thống. Ngoài ra, Admin kế thừa toàn bộ quyền hạn của Contest Creator và có thể can thiệp vào bất kỳ bài toán hay cuộc thi nào khi cần thiết.

(tiếp theo Bảng 3.1)

STT	Tác nhân	Mô tả
4	Judge0	Đây là dịch vụ chấm bài bên ngoài được tự triển khai (self-hosted) thông qua Docker. Judge0 đóng vai trò thực thi mã nguồn trong môi trường sandbox cô lập, so sánh đầu ra với đáp án mẫu và trả về kết quả chấm bài cho hệ thống. Judge0 không tương tác trực tiếp với người dùng mà hoạt động hoàn toàn như một dịch vụ nền được gọi thông qua API nội bộ.
5	LLM Service	Đây là dịch vụ LLM bên ngoài được tích hợp thông qua API. Backend đóng vai trò proxy trung gian, tiếp nhận yêu cầu từ frontend, xây dựng prompt có cấu trúc với ngữ cảnh bài toán phù hợp, gửi đến LLM và chuyển tiếp phản hồi về cho người dùng. Dịch vụ này hỗ trợ các tính năng gợi ý hướng giải, phân tích độ phức tạp thuật toán và giải thích mã nguồn.

(tiếp theo Bảng 3.1)

STT	Tác nhân	Mô tả
6	AI Parser Service	Đây là dịch vụ phân tích tài liệu PDF sử dụng LLM làm nền tảng. Khi Contest Creator tải lên một tệp đề bài dưới định dạng PDF, AI Parser Service phân tích cấu trúc tài liệu và trích xuất tự động các trường thông tin cần thiết của bài toán bao gồm tiêu đề, mô tả, định dạng đầu vào – đầu ra và các ví dụ mẫu, trả về dữ liệu có cấu trúc để hệ thống tự động điền vào biểu mẫu tạo bài toán.

3.3 Yêu cầu chức năng (Functional Requirements)

3.3.1 Tổng quan Use Case

Hệ thống TDTUOJ được phân tích theo mô hình Use Case với ba nhóm tác nhân người dùng chính (User, Contest Creator, Admin) và ba hệ thống bên ngoài (Judge0, LLM Service, AI Parser Service). Sơ đồ Use Case tổng quan được thể hiện trong Hình 3.1.

Bảng 3.2: Danh sách các Use Case của hệ thống TDTUOJ

Mã UC	Tên Use Case	Mô tả
Nhóm 1: Xác thực và tài khoản		
UC01	Đăng ký tài khoản	Người dùng tạo tài khoản mới bằng cách cung cấp thông tin cần thiết. Hệ thống xác thực dữ liệu đầu vào và tạo tài khoản với vai trò User mặc định.

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC02	Đăng nhập	Người dùng đăng nhập vào hệ thống bằng tên đăng nhập và mật khẩu. Hệ thống xác thực thông tin đăng nhập và cấp phát cặp access token – refresh token theo cơ chế JWT.
UC03	Đăng xuất	Người dùng đăng xuất khỏi hệ thống. Hệ thống thu hồi token phiên làm việc hiện tại.
UC04	Đổi mật khẩu	Người dùng thay đổi mật khẩu tài khoản của mình sau khi xác thực mật khẩu cũ.
UC05	Cập nhật thông tin cá nhân	Người dùng cập nhật thông tin hồ sơ cá nhân bao gồm tên hiển thị và ảnh đại diện.
Nhóm 2: Quản lý bài toán		
UC06	Xem danh sách bài toán	Người dùng xem danh sách các bài toán đang công khai trong hệ thống. Hệ thống hỗ trợ tìm kiếm theo tên, lọc theo thẻ tag và mức độ khó.
UC07	Xem chi tiết bài toán	Người dùng xem toàn bộ thông tin của một bài toán bao gồm đề bài, định dạng đầu vào – đầu ra, giới hạn thời gian, giới hạn bộ nhớ và các ví dụ mẫu.
UC08	Tạo bài toán	Contest Creator hoặc Admin tạo một bài toán mới, nhập đầy đủ thông tin mô tả, cấu hình giới hạn và thêm các test case tương ứng.

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC09	Chỉnh sửa bài toán	Contest Creator hoặc Admin cập nhật thông tin của một bài toán đã tồn tại bao gồm nội dung đề bài, cấu hình và danh sách test case. Việc công khai thủ công một bài toán đang thuộc cuộc thi chưa kết thúc bị từ chối; bài toán sẽ được công khai tự động khi cuộc thi kết thúc.
UC10	Xóa bài toán	Contest Creator hoặc Admin xóa một bài toán khỏi hệ thống. Thao tác này chỉ được phép nếu bài toán không đang được sử dụng trong bất kỳ cuộc thi nào đang diễn ra.
Nhóm 3: Nộp bài và xem kết quả		
UC11	Nộp bài giải	Người dùng nộp mã nguồn lời giải cho một bài toán với ngôn ngữ lập trình được chọn. Hệ thống đưa bài nộp vào hàng đợi Redis và xử lý bất đồng bộ thông qua Judge0.
UC12	Xem kết quả bài nộp	Người dùng xem kết quả chấm bài của một lần nộp cụ thể bao gồm trạng thái, thời gian thực thi và lượng bộ nhớ sử dụng trên từng test case.
UC13	Xem lịch sử bài nộp	Người dùng xem toàn bộ lịch sử các lần nộp bài của bản thân theo từng bài toán hoặc toàn hệ thống, có thể lọc theo trạng thái kết quả và ngôn ngữ lập trình.
Nhóm 4: Quản lý và tham gia cuộc thi		

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC14	Tạo cuộc thi	Contest Creator hoặc Admin tạo một cuộc thi mới, cấu hình thông tin tổng quan, thời gian bắt đầu – kết thúc, khoảng đóng băng, bảng xếp hạng và danh sách bài toán. Chỉ các bài toán riêng tư thuộc sở hữu của người tạo và chưa có lượt nộp từ người dùng khác mới được phép đưa vào cuộc thi.
UC15	Chỉnh sửa cuộc thi	Contest Creator hoặc Admin cập nhật thông tin của một cuộc thi, bao gồm điều chỉnh cấu hình hoặc thay thế bài toán trước khi cuộc thi bắt đầu.
UC16	Xóa cuộc thi	Contest Creator hoặc Admin xóa một cuộc thi chưa diễn ra khỏi hệ thống.
UC17	Đăng ký tham gia cuộc thi	Người dùng đăng ký tham gia một cuộc thi đang mở đăng ký. Hệ thống ghi nhận thông tin đăng ký và cấp quyền truy cập các bài toán trong cuộc thi khi đến thời gian quy định.
UC18	Tham gia cuộc thi	Người dùng đã đăng ký truy cập vào không gian thi cử, xem danh sách bài toán của cuộc thi và nộp bài trong khoảng thời gian cho phép.

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC19	Xem bảng xếp hạng	Người dùng xem bảng xếp hạng của một cuộc thi được cập nhật theo thời gian thực. Dữ liệu bảng xếp hạng được phục vụ từ cache Redis nhằm đảm bảo hiệu năng. Trong khoảng đóng băng cuối cuộc thi, người xem thông thường nhận ảnh chụp bảng xếp hạng tại thời điểm đóng băng; người tạo cuộc thi và Admin vẫn xem được bảng trực tiếp.
Nhóm 5: Hỗ trợ học tập bằng LLM		
UC20	Gợi ý hướng giải	Người dùng yêu cầu gợi ý có định hướng cho một bài toán. LLM cung cấp gợi ý về hướng tiếp cận mà không tiết lộ lời giải hoàn chỉnh, nhằm kích thích tư duy giải quyết vấn đề của người học.
UC21	Phân tích độ phức tạp thuật toán	Người dùng gửi đoạn mã nguồn của lời giải để nhận phân tích độ phức tạp thời gian và không gian kèm theo giải thích chi tiết từ LLM.
UC22	Giải thích mã nguồn	Người dùng yêu cầu giải thích một đoạn mã nguồn. LLM phân tích logic, cấu trúc và ý nghĩa của từng phần trong đoạn mã, giúp người học hiểu rõ hơn về cách tiếp cận của lời giải.

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC23	Nhập đề bài từ PDF (AI Parser)	Contest Creator tải lên tệp đề bài dạng PDF. AI Parser Service phân tích tài liệu và tự động điền các trường thông tin bài toán, giúp rút ngắn thời gian soạn thảo nội dung.
Nhóm 6: Quản trị hệ thống (Admin)		
UC24	Xem danh sách người dùng	Admin xem toàn bộ danh sách tài khoản trong hệ thống, có thể tìm kiếm và lọc theo vai trò hoặc trạng thái tài khoản.
UC25	Xem chi tiết tài khoản	Admin xem thông tin chi tiết của một tài khoản người dùng bất kỳ.
UC26	Tạo tài khoản	Admin tạo tài khoản mới cho người dùng và gán vai trò phù hợp ngay tại thời điểm tạo.
UC27	Cập nhật thông tin tài khoản	Admin chỉnh sửa thông tin hoặc thay đổi vai trò của một tài khoản người dùng bất kỳ.
UC28	Vô hiệu hóa tài khoản	Admin vô hiệu hóa tài khoản của một người dùng. Tài khoản bị vô hiệu hóa sẽ không thể đăng nhập vào hệ thống cho đến khi được khôi phục.
Nhóm 7: Trực quan hóa và tương tác bài toán		
UC29	Trực quan hóa và gỡ lỗi mã nguồn	Người dùng chạy mã nguồn ở chế độ trực quan hóa (visualize) để theo dõi trạng thái biến và cấu trúc dữ liệu qua từng bước thực thi, hỗ trợ gỡ lỗi và hiểu rõ thuật toán.

(tiếp theo Bảng 3.2)

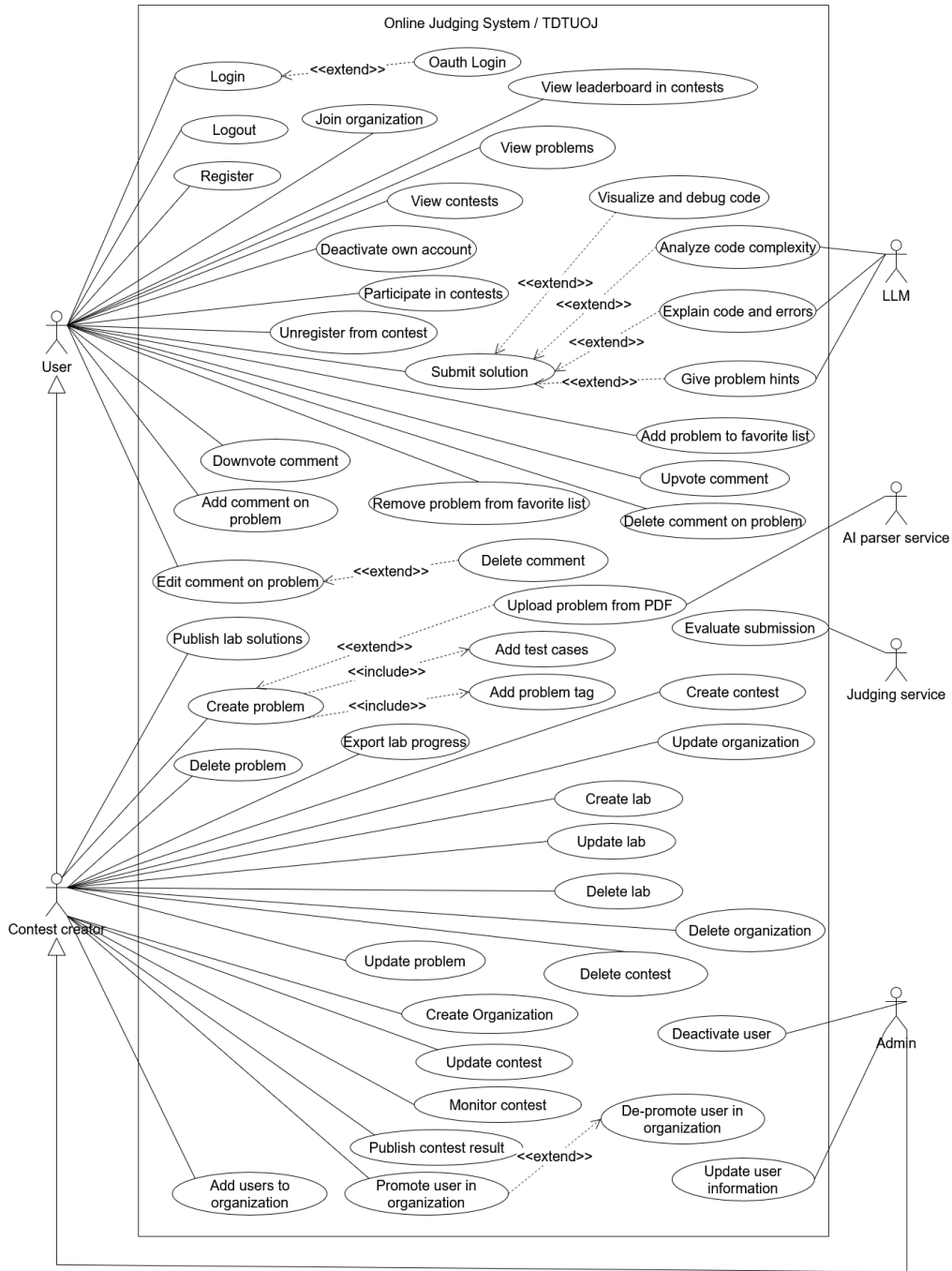
Mã UC	Tên Use Case	Mô tả
UC30	Thêm bình luận trên bài toán	Người dùng đăng một bình luận trao đổi về bài toán.
UC31	Trả lời bình luận	Người dùng trả lời một bình luận đã có, tạo thành luồng thảo luận nhiều cấp.
UC32	Chỉnh sửa bình luận	Người dùng chỉnh sửa nội dung bình luận do chính mình đăng.
UC33	Xóa bình luận	Người dùng xóa bình luận do chính mình đăng.
UC34	Bình chọn bình luận	Người dùng bình chọn tán thành (upvote) hoặc phản đối (downvote) một bình luận.
UC35	Thêm bài toán vào danh sách yêu thích	Người dùng đánh dấu một bài toán vào danh sách yêu thích để truy cập nhanh về sau.
UC36	Xóa bài toán khỏi danh sách yêu thích	Người dùng gỡ một bài toán khỏi danh sách yêu thích của mình.
Nhóm 8: Quản lý bài thực hành (Lab)		
UC37	Tạo lab	Contest Creator tạo một bài thực hành (lab) trong tổ chức, chọn các bài toán riêng tư thuộc sở hữu của mình và thiết lập thời hạn nộp bài.
UC38	Chỉnh sửa lab	Contest Creator cập nhật thông tin của một lab, bao gồm danh sách bài toán và thời hạn.
UC39	Xóa lab	Contest Creator xóa một lab khỏi tổ chức.

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC40	Xem tiến độ lab	Contest Creator xem tiến độ làm bài của từng thành viên theo từng bài toán trong lab.
UC41	Xuất tiến độ lab	Contest Creator xuất bảng tiến độ của lab ra tệp Excel để lưu trữ hoặc báo cáo.
UC42	Công bố lời giải lab	Contest Creator công bố lời giải của lab cho thành viên sau khi đã hết thời hạn nộp bài.
Nhóm 9: Quản lý tổ chức		
UC43	Tham gia tổ chức	Người dùng tham gia một tổ chức để truy cập các lab và cuộc thi nội bộ của tổ chức đó.
UC44	Tạo tổ chức	Contest Creator tạo một tổ chức mới và trở thành người quản lý của tổ chức.
UC45	Chỉnh sửa tổ chức	Contest Creator cập nhật thông tin của tổ chức do mình quản lý.
UC46	Xóa tổ chức	Contest Creator xóa một tổ chức do mình quản lý khỏi hệ thống.
UC47	Thêm thành viên vào tổ chức	Contest Creator thêm người dùng vào tổ chức.
UC48	Thăng quyền thành viên	Contest Creator nâng vai trò của một thành viên trong tổ chức.
UC49	Giáng quyền thành viên	Contest Creator hạ vai trò của một thành viên trong tổ chức.
Nhóm 10: Cuộc thi và tài khoản (bổ sung)		

(tiếp theo Bảng 3.2)

Mã UC	Tên Use Case	Mô tả
UC50	Xem danh sách cuộc thi	Người dùng xem danh sách các cuộc thi công khai cùng trạng thái sắp diễn ra, đang diễn ra hoặc đã kết thúc.
UC51	Hủy đăng ký cuộc thi	Người dùng hủy đăng ký một cuộc thi trước khi cuộc thi bắt đầu.
UC52	Theo dõi cuộc thi	Contest Creator theo dõi diễn biến cuộc thi theo thời gian thực, bao gồm bài nộp và thứ hạng của thí sinh.
UC53	Công bố kết quả cuộc thi	Contest Creator công bố kết quả chính thức của cuộc thi sau khi kết thúc và cập nhật điểm xếp hạng (rating) của thí sinh.
UC54	Vô hiệu hóa tài khoản cá nhân	Người dùng tự vô hiệu hóa tài khoản của chính mình.
UC55	Đăng nhập bằng Google	Người dùng đăng nhập vào hệ thống thông qua tài khoản Google theo cơ chế OAuth.
UC56	Quản lý test case	Contest Creator thêm, chỉnh sửa hoặc xóa các test case của một bài toán, phân loại test case mẫu và test case ẩn.
UC57	Đánh giá bài nộp	Hệ thống chấm bài Judge0 thực thi và đánh giá bài nộp, trả về kết quả chấm cho hệ thống.



Hình 3.1: Sơ đồ Use Case tổng quan của hệ thống TDTUOJ

3.3.2 Các Use Case chính

3.3.2.1 Use case Nộp bài giải

Nộp bài giải là Use Case trung tâm của hệ thống, phản ánh hoạt động cốt lõi của một nền tảng chấm bài trực tuyến. Người dùng sau khi đọc đề bài sẽ viết lời giải trong trình soạn thảo mã nguồn tích hợp, chọn ngôn ngữ lập trình phù hợp và gửi bài lên hệ thống để được chấm tự động.

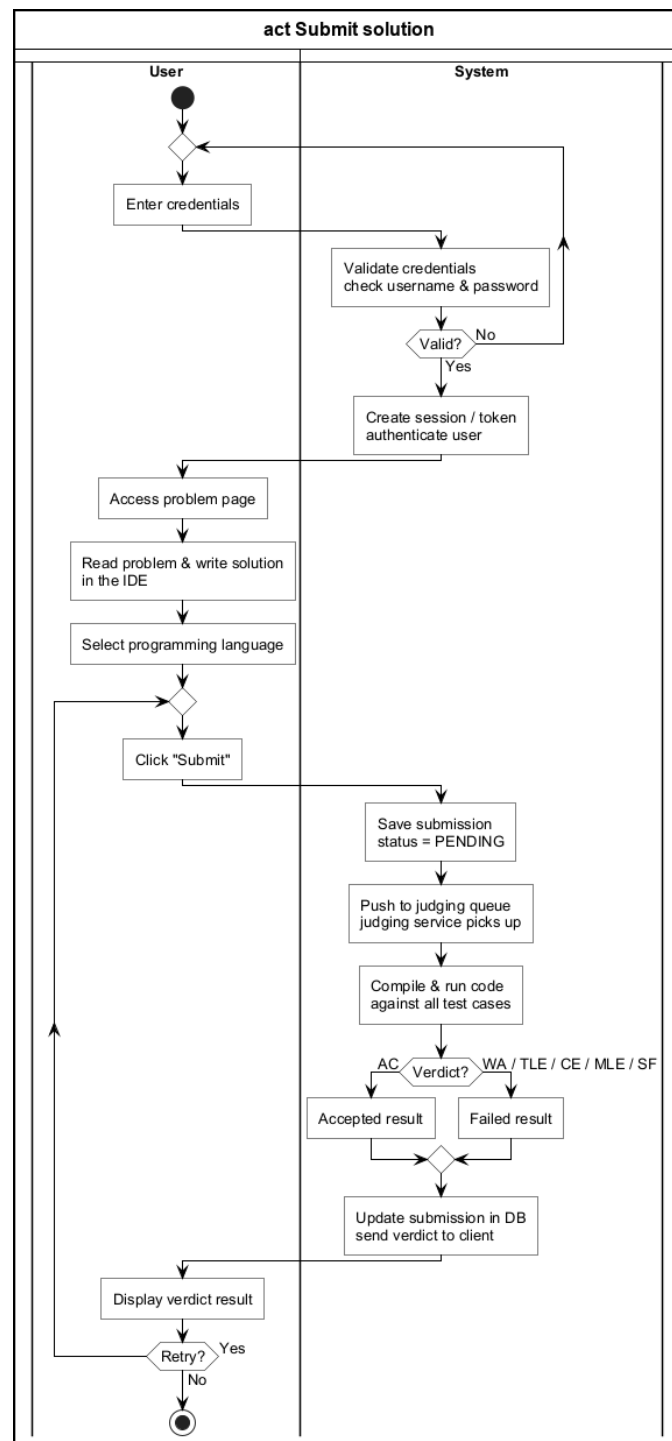
Sau khi tiếp nhận bài nộp, hệ thống lưu bản ghi với trạng thái chờ xử lý (PENDING), đưa bài vào hàng đợi và chuyển đến dịch vụ chấm bài Judge0 để biên dịch và thực thi trên toàn bộ test case. Kết quả chấm (verdict) cùng thời gian và bộ nhớ sử dụng được cập nhật và hiển thị lại cho người dùng. Bảng 3.3 trình bày đặc tả chi tiết của Use Case này.

Bảng 3.3: Bảng đặc tả Use case Nộp bài giải

Use Case	Nộp bài giải
Bối cảnh	Người dùng muốn kiểm tra tính đúng đắn và hiệu quả của lời giải mình viết cho một bài toán.
Mô tả	Người dùng gửi mã nguồn lời giải cho một bài toán bằng ngôn ngữ lập trình được chọn; hệ thống chấm bài tự động qua Judge0 và trả về kết quả.
Tác nhân	Người dùng (sinh viên).
Sự kiện kích hoạt	Người dùng nhấn nút “Nộp bài” trên trang chi tiết bài toán.
Điều kiện tiên quyết	+ Người dùng đã đăng nhập vào hệ thống. + Bài toán đang công khai hoặc thuộc cuộc thi mà người dùng được phép truy cập.
Kết quả	Bài nộp được chấm tự động; kết quả chấm (AC, WA, TLE, MLE, RE hoặc CE) được hiển thị và lưu lại để phục vụ thống kê và xếp hạng.

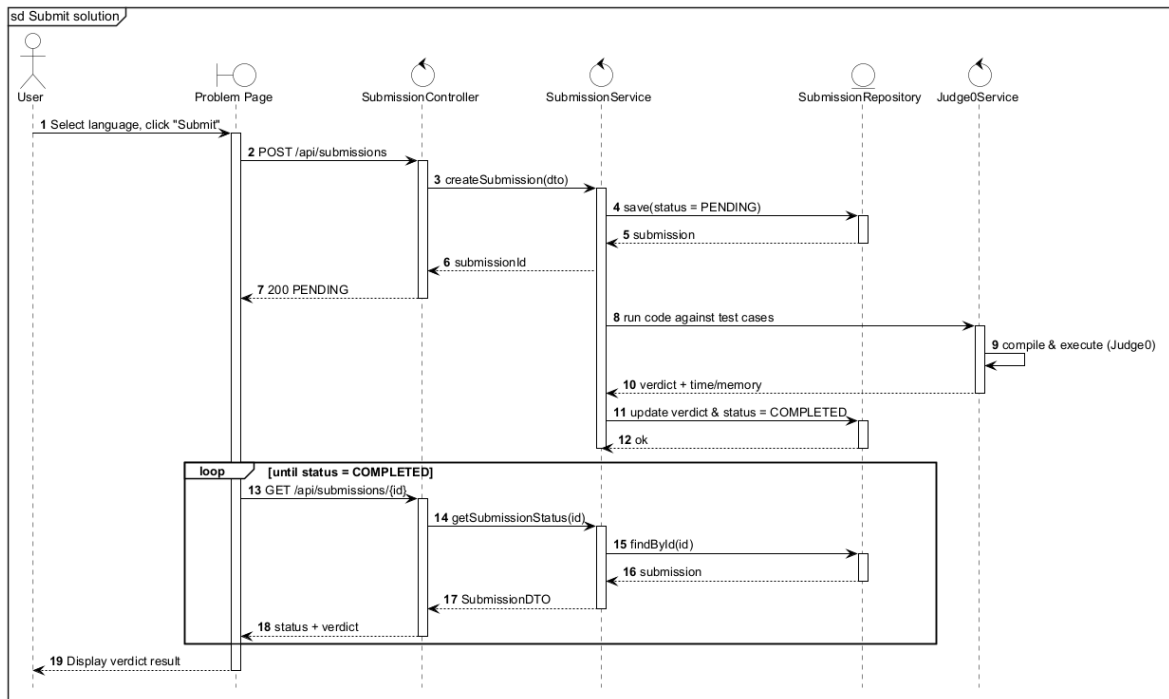
Luồng sự kiện	Tác nhân	Hệ thống
	1. Người dùng viết lời giải trong trình soạn thảo và chọn ngôn ngữ lập trình. 2. Người dùng nhấn nút “Nộp bài”. 3. Người dùng xem kết quả bài nộp.	2.1 Hệ thống lưu bài nộp với trạng thái chờ xử lý (PENDING) và đưa vào hàng đợi Redis. 2.2 Hệ thống gửi mã nguồn cùng test case đến Judge0 để biên dịch và thực thi. 2.3 Hệ thống nhận kết quả chấm, cập nhật verdict, thời gian và bộ nhớ vào cơ sở dữ liệu. 3.1 Hệ thống hiển thị kết quả chấm (verdict) cùng thời gian và bộ nhớ cho người dùng.
Ngoại lệ	Mã nguồn rỗng hoặc chưa chọn ngôn ngữ $\Rightarrow$ hệ thống hiển thị thông báo lỗi. Bài nộp biên dịch lỗi $\Rightarrow$ hệ thống trả về kết quả CE (Compilation Error).	

Luồng xử lý của Use Case được mô tả trong sơ đồ hoạt động ở Hình 3.2.



Hình 3.2: Sơ đồ hoạt động cho Use Case Nộp bài giải

Sự tương tác giữa người dùng và các thành phần của hệ thống được thể hiện qua sơ đồ tuần tự trong Hình 3.3.



Hình 3.3: Sơ đồ tuần tự cho Use Case Nộp bài giải

3.3.2.2 Use case Tạo bài toán

Tạo bài toán là Use Case cốt lõi của tác nhân Contest Creator, cho phép giảng viên xây dựng kho đề phục vụ giảng dạy và tổ chức thi. Hệ thống hỗ trợ hai cách soạn đề: nhập thủ công toàn bộ nội dung, hoặc tải lên tập tin PDF để dịch vụ trích xuất bằng AI (AI Parser) tự động bóc tách đề bài và các ví dụ mẫu, sau đó người tạo đề rà soát và chỉnh sửa lại.

Sau khi hoàn tất phần mô tả, người tạo đề gắn thẻ phân loại, thiết lập mức độ khó cùng phạm vi hiển thị, rồi bổ sung các trường hợp kiểm thử (test case) bằng cách nhập tay hoặc nhờ AI sinh tự động. Khi lưu, hệ thống kiểm tra tính hợp lệ, sinh đường dẫn rút gọn (slug) và lưu bài toán cùng toàn bộ test case vào cơ sở dữ liệu. Bảng 3.4 trình bày đặc tả chi tiết của Use Case này.

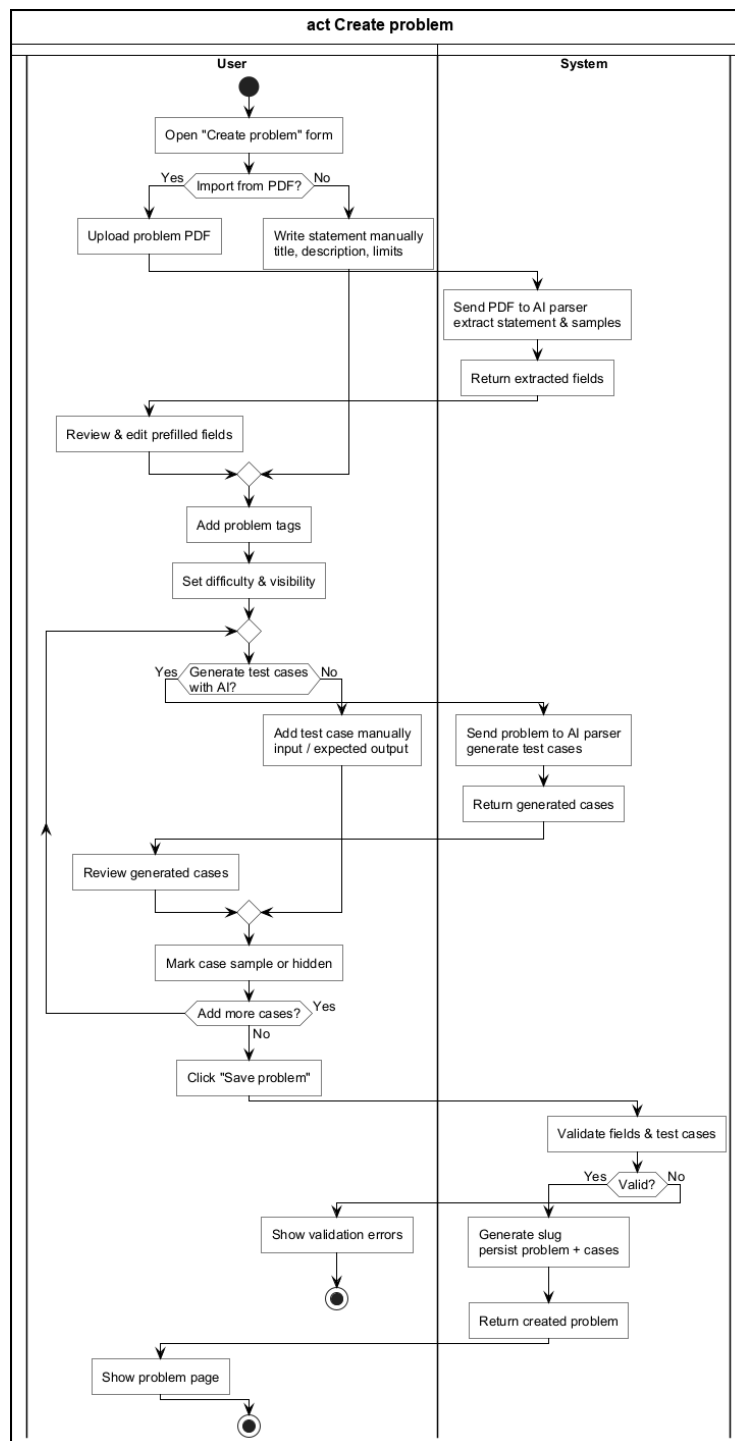
Bảng 3.4: Bảng đặc tả Use case Tạo bài toán

Use Case	Tạo bài toán
----------	--------------

Bối cảnh	Giảng viên muốn bổ sung một bài toán mới vào kho đề để giao bài thực hành hoặc tổ chức cuộc thi.	
Mô tả	Người tạo đề soạn nội dung bài toán (thủ công hoặc trích xuất từ PDF), gắn thẻ, thiết lập cấu hình và thêm test case; hệ thống kiểm tra và lưu bài toán.	
Tác nhân	Contest Creator (giảng viên), Admin.	
Sự kiện kích hoạt	Người tạo đề mở biểu mẫu “Tạo bài toán”.	
Điều kiện tiên quyết	+ Người dùng đã đăng nhập với vai trò Contest Creator hoặc Admin.	
Kết quả	Bài toán mới được lưu cùng các test case; hệ thống sinh slug và hiển thị trang chi tiết bài toán vừa tạo.	
Luồng sự kiện	Tác nhân	Hệ thống
	1. Người tạo đề mở biểu mẫu “Tạo bài toán” và chọn cách soạn (thủ công hoặc nhập từ PDF). 2. (Tùy chọn) Người tạo đề tải lên tập tin PDF. 3. Người tạo đề chỉnh sửa nội dung, gắn thẻ, thiết lập mức độ khó và phạm vi hiển thị.	2.1 Hệ thống gửi tập tin đến dịch vụ AI để trích xuất đề bài và ví dụ mẫu. 2.2 Hệ thống điền sẵn các trường thông tin và trả về cho người tạo đề rà soát.

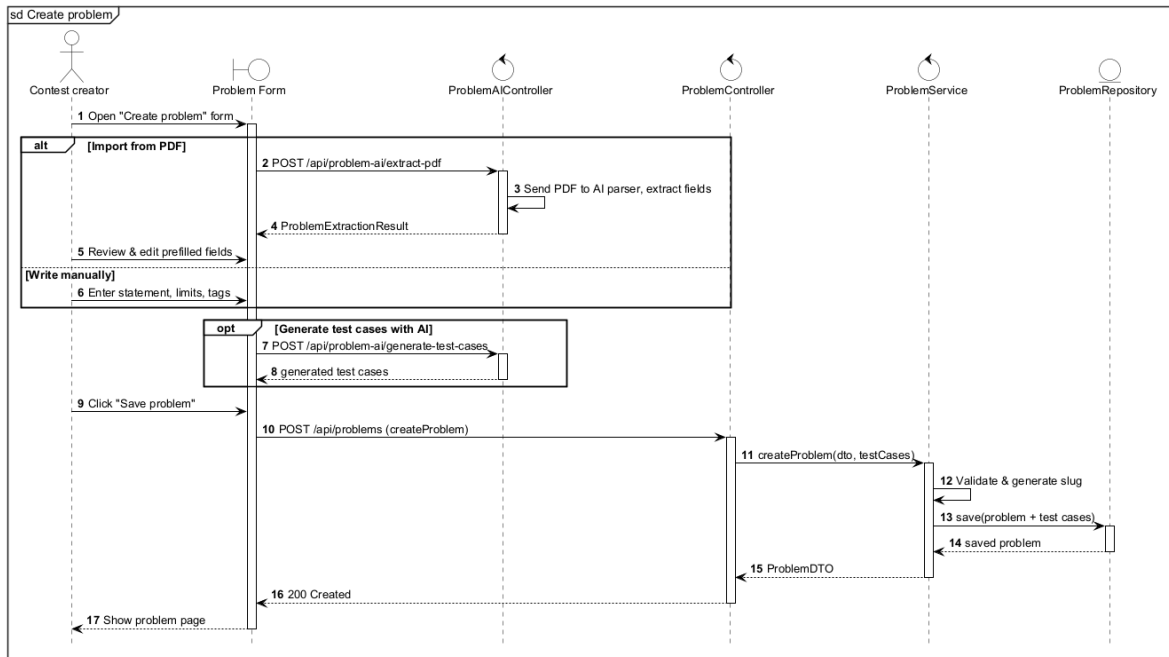
	4. Người tạo đề thêm test case bằng cách nhập tay hoặc yêu cầu AI sinh tự động, và đánh dấu test case là ví dụ mẫu hay ẩn. 5. Người tạo đề nhấn nút “Lưu bài toán”. 6. Người tạo đề xem trang chi tiết bài toán.	4.1 Khi được yêu cầu, hệ thống gọi dịch vụ AI sinh các test case và trả về cho người tạo đề. 5.1 Hệ thống kiểm tra tính hợp lệ của các trường thông tin và test case. 5.2 Hệ thống sinh slug, lưu bài toán cùng test case vào cơ sở dữ liệu và trả về bài toán đã tạo. 6.1 Hệ thống hiển thị trang bài toán vừa được tạo.
Ngoại lệ	Thiếu trường bắt buộc hoặc chưa có test case hợp lệ $\Rightarrow$ hệ thống hiển thị thông báo lỗi và không lưu bài toán.	

Luồng xử lý của Use Case được mô tả trong sơ đồ hoạt động ở Hình 3.4.



Hình 3.4: Sơ đồ hoạt động cho Use Case Tạo bài toán

Sự tương tác giữa người tạo đề và các thành phần của hệ thống được thể hiện qua sơ đồ tuần tự trong Hình 3.5.



Hình 3.5: Sơ đồ tuần tự cho Use Case Tạo bài toán

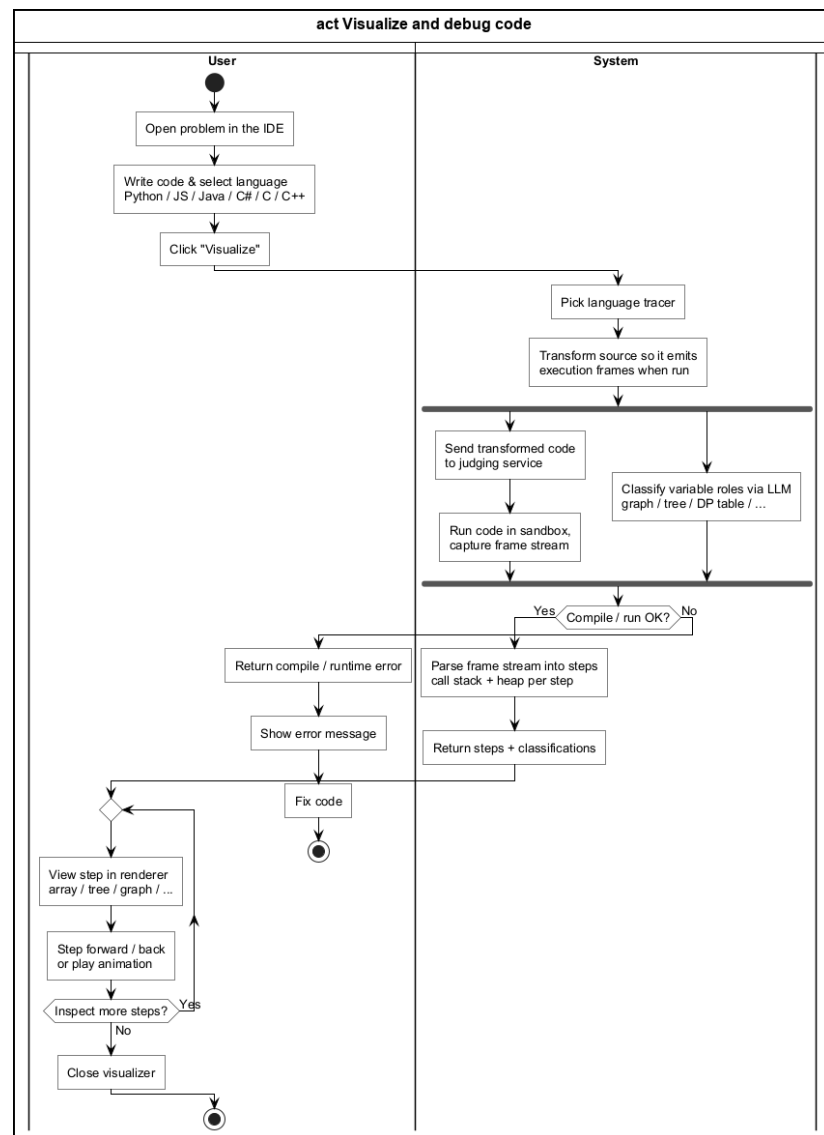
3.3.2.3 Use case Trực quan hóa và gỡ lỗi mã nguồn

Trực quan hóa và gỡ lỗi mã nguồn là Use Case đặc trưng của hệ thống, hỗ trợ người học hiểu cách lời giải vận hành thay vì chỉ nhận về kết quả đúng hay sai. Người dùng viết mã nguồn, chọn ngôn ngữ lập trình, sau đó yêu cầu hệ thống minh họa quá trình thực thi.

Hệ thống chọn bộ ghi vết (tracer) tương ứng với ngôn ngữ để biến đổi mã nguồn sao cho khi chạy, chương trình tự phát ra chuỗi khung trạng thái (frame) mô tả ngăn xếp lời gọi và các biến tại từng dòng lệnh, rồi gửi mã đã biến đổi đến dịch vụ chấm bài để thực thi trong môi trường cô lập. Song song, một mô hình ngôn ngữ lớn phân loại vai trò ngữ nghĩa của từng biến (đồ thị, cây, bảng quy hoạch động...) để trình minh họa chọn cách hiển thị phù hợp. Vết thực thi được phân tích thành chuỗi các bước; trình minh họa hiển thị từng bước dưới dạng mảng, cây, đồ thị hoặc cấu trúc tương ứng để người dùng theo dõi. Bảng 3.5 trình bày đặc tả chi tiết của Use Case này.

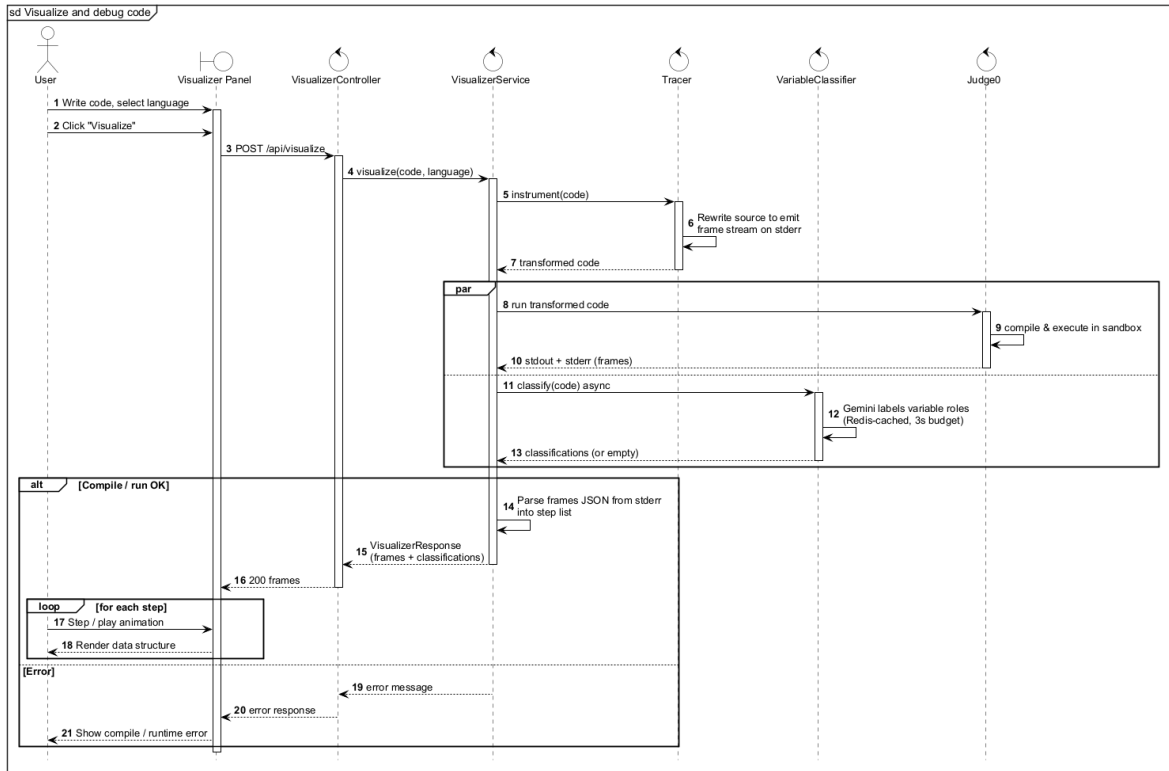
Bảng 3.5: Bảng đặc tả Use case Trực quan hóa và gỡ lỗi mã nguồn

Use Case	Trực quan hóa và gỡ lỗi mã nguồn	
Bối cảnh	Người dùng muốn quan sát trạng thái các biến và cấu trúc dữ liệu qua từng bước để hiểu hoặc gỡ lỗi lời giải.	
Mô tả	Hệ thống biến đổi mã nguồn để tự ghi vết khi chạy, thực thi qua dịch vụ chấm bài và phân tích vết thành các bước minh họa cho người dùng xem lại.	
Tác nhân	Người dùng (sinh viên).	
Sự kiện kích hoạt	Người dùng nhấn nút “Trực quan hóa” (Visualize) trong trình soạn thảo.	
Điều kiện tiên quyết	+ Người dùng đã viết mã nguồn và chọn ngôn ngữ lập trình được hỗ trợ (Python, JavaScript, Java, C#, C, C++).	
Kết quả	Quá trình thực thi được minh họa theo từng bước; người dùng có thể duyệt tới lui hoặc phát hoạt ảnh để quan sát trạng thái dữ liệu.	
Luồng sự kiện	Tác nhân	Hệ thống
	1. Người dùng viết mã nguồn và chọn ngôn ngữ lập trình.	



Hình 3.6: Sơ đồ hoạt động cho Use Case Trực quan hóa và gỡ lỗi mã nguồn

Sự tương tác giữa người dùng và các thành phần của hệ thống được thể hiện qua sơ đồ tuần tự trong Hình 3.7.



Hình 3.7: Sơ đồ tuần tự cho Use Case Trực quan hóa và gỡ lỗi mã nguồn

3.3.2.4 Use case Tham gia cuộc thi

Tham gia cuộc thi cho phép người dùng thi đấu trực tuyến trên hệ thống theo thể thức ICPC. Người dùng đăng ký tham gia một cuộc thi và được xác nhận tham gia ngay, miễn là cuộc thi còn trong thời gian cho phép đăng ký và chưa đạt giới hạn số lượng người tham gia.

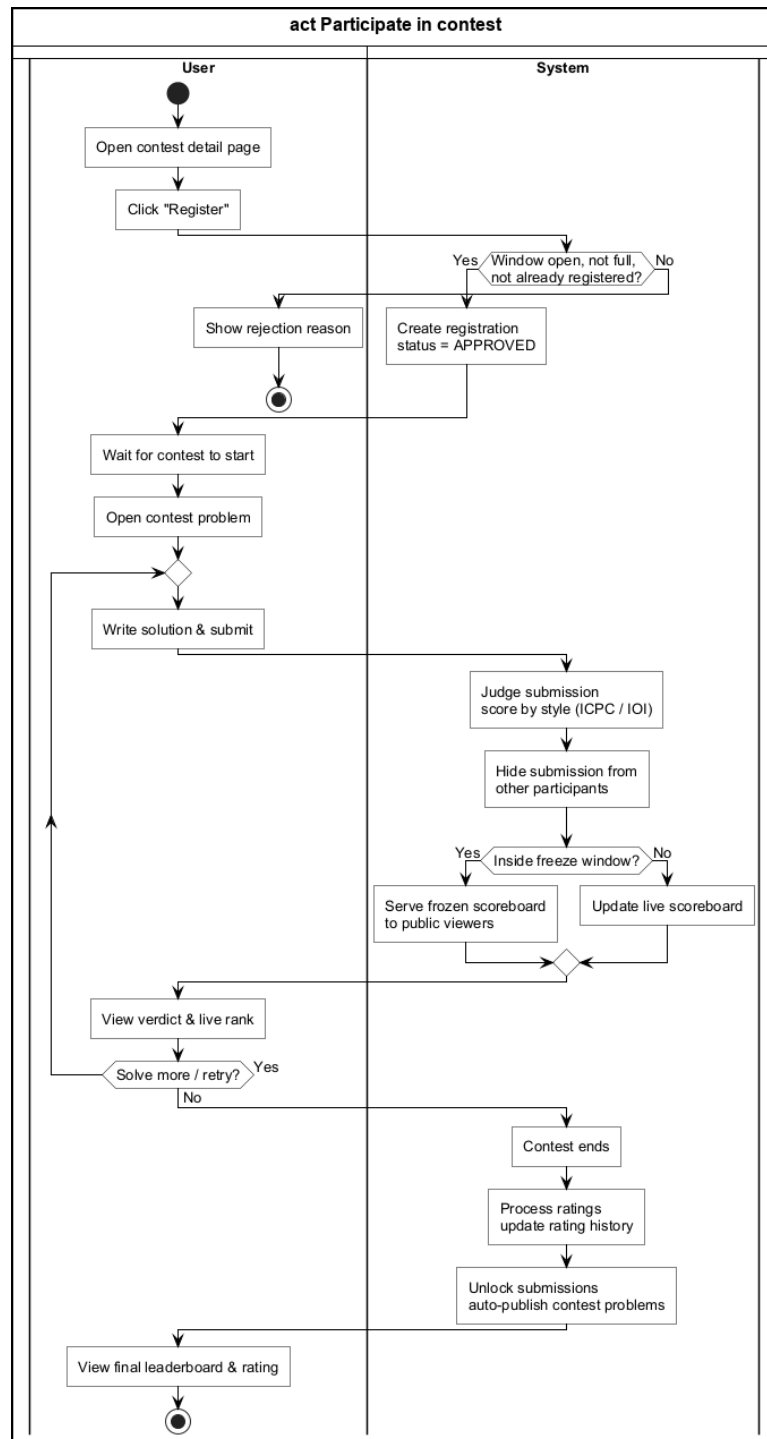
Trong thời gian diễn ra, người dùng mở từng bài toán, viết lời giải và nộp bài; hệ thống chấm bài, tính điểm theo thể thức và cập nhật bảng xếp hạng theo thời gian thực để người dùng theo dõi thứ hạng. Nhằm bảo đảm tính công bằng, bài nộp trong cuộc thi được ẩn khỏi những người dùng khác cho đến khi cuộc thi kết thúc; nếu người tạo cuộc thi cấu hình khoảng đóng băng, bảng xếp hạng công khai ngừng cập nhật trong những phút cuối theo thể thức ICPC. Khi cuộc thi kết thúc, hệ thống tự động tính toán và cập nhật lịch sử điểm xếp hạng (rating), mở lại quyền xem bài nộp và công khai các bài toán của cuộc thi vào kho luyện tập. Bảng 3.6 trình bày đặc tả chi tiết của Use Case này.

Bảng 3.6: Bảng đặc tả Use case Tham gia cuộc thi

Use Case	Tham gia cuộc thi	
Bối cảnh	Người dùng muốn thi đấu trong một cuộc thi lập trình được tổ chức trên hệ thống.	
Mô tả	Người dùng đăng ký, nộp bài trong thời gian thi và theo dõi bảng xếp hạng; hệ thống chấm bài, tính điểm theo thể thức và cập nhật rating khi kết thúc.	
Tác nhân	Người dùng (sinh viên); Contest Creator (tổ chức và công bố kết quả).	
Sự kiện kích hoạt	Người dùng nhấn nút “Đăng ký” trên trang chi tiết cuộc thi.	
Điều kiện tiên quyết	+ Người dùng đã đăng nhập. + Cuộc thi đang ở trạng thái cho phép đăng ký.	
Kết quả	Người dùng được ghi nhận tham gia; điểm và thứ hạng được tính, bảng xếp hạng cuối cùng cùng rating được cập nhật.	
Luồng sự kiện	Tác nhân	Hệ thống
	1. Người dùng mở trang cuộc thi và nhấn nút “Đăng ký”.	1.1 Hệ thống kiểm tra điều kiện đăng ký (còn thời hạn, chưa đầy chỗ, chưa đăng ký) rồi tạo bản ghi đăng ký với trạng thái APPROVED và xác nhận tham gia ngay.

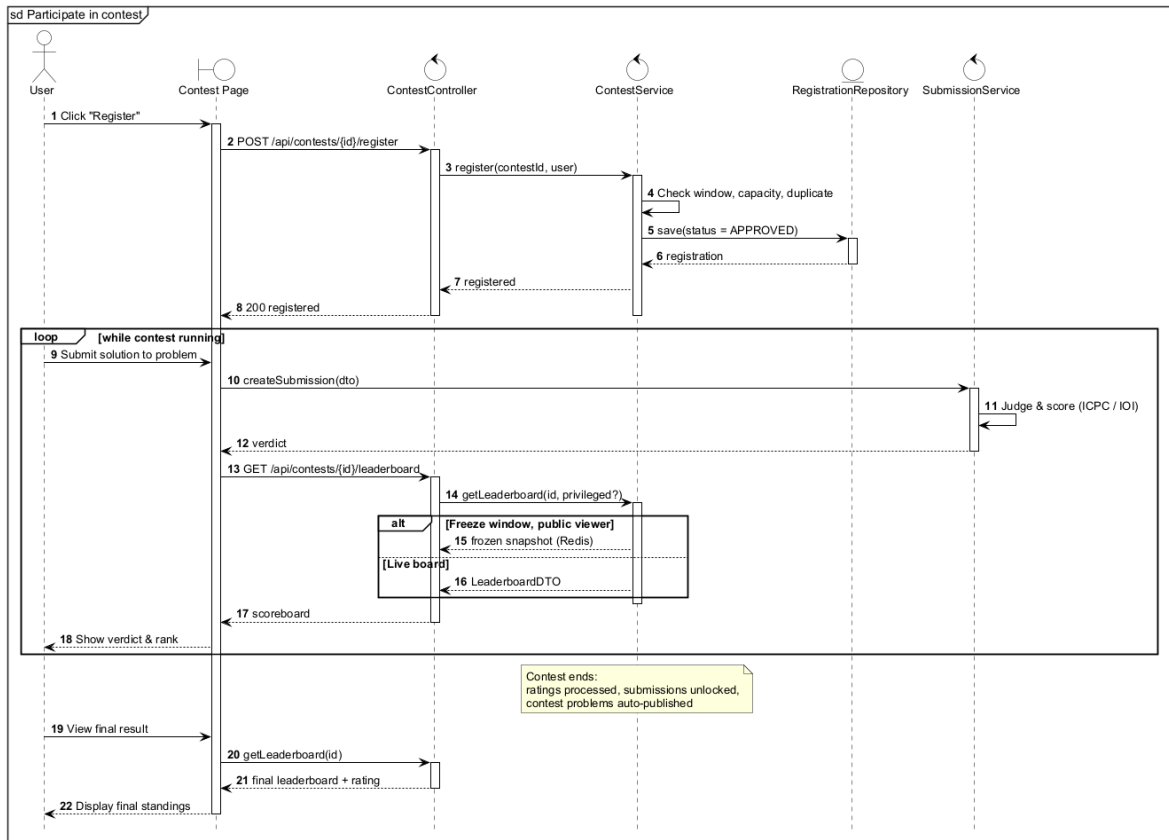
	2. Người dùng chờ đến giờ thi, mở bài toán và nộp lời giải. 3. Người dùng xem kết quả chấm và thứ hạng trực tiếp. 4. Người dùng xem kết quả chung cuộc.	2.1 Hệ thống chấm bài nộp và tính điểm theo thể thức ICPC. 2.2 Hệ thống ẩn bài nộp khỏi những người dùng khác cho đến khi cuộc thi kết thúc. 2.3 Hệ thống cập nhật bảng xếp hạng theo thời gian thực; trong khoảng đóng băng, người xem thông thường nhận ảnh chụp bảng xếp hạng tại thời điểm đóng băng. 3.1 Hệ thống hiển thị kết quả chấm cùng thứ hạng hiện thời. 4.1 Khi cuộc thi kết thúc, hệ thống tự động tính toán điểm xếp hạng và cập nhật lịch sử rating. 4.2 Hệ thống mở lại quyền xem bài nộp, công khai các bài toán của cuộc thi và hiển thị bảng xếp hạng cuối cùng.
Ngoại lệ	Cuộc thi đã kết thúc, đã đạt giới hạn số người tham gia, hoặc người dùng đã đăng ký trước đó $\Rightarrow$ hệ thống từ chối và thông báo lý do.	

Luồng xử lý của Use Case được mô tả trong sơ đồ hoạt động ở Hình 3.8.



Hình 3.8: Sơ đồ hoạt động cho Use Case Tham gia cuộc thi

Sự tương tác giữa người dùng và các thành phần của hệ thống được thể hiện qua sơ đồ tuần tự trong Hình 3.9.



Hình 3.9: Sơ đồ tuần tự cho Use Case Tham gia cuộc thi

3.3.2.5 Use case Gợi ý học tập bằng LLM

Gợi ý học tập bằng LLM hỗ trợ người dùng vượt qua điểm vướng mắc mà không tiết lộ lời giải hoàn chỉnh. Người dùng mở khung gợi ý trong trang bài toán, chọn mô hình ngôn ngữ lớn (Gemini, Claude hoặc GPT-4o), nhập câu hỏi hoặc chọn một mẫu yêu cầu sẵn có như gợi ý hướng tiếp cận, giải thích đề bài hay phân tích độ phức tạp.

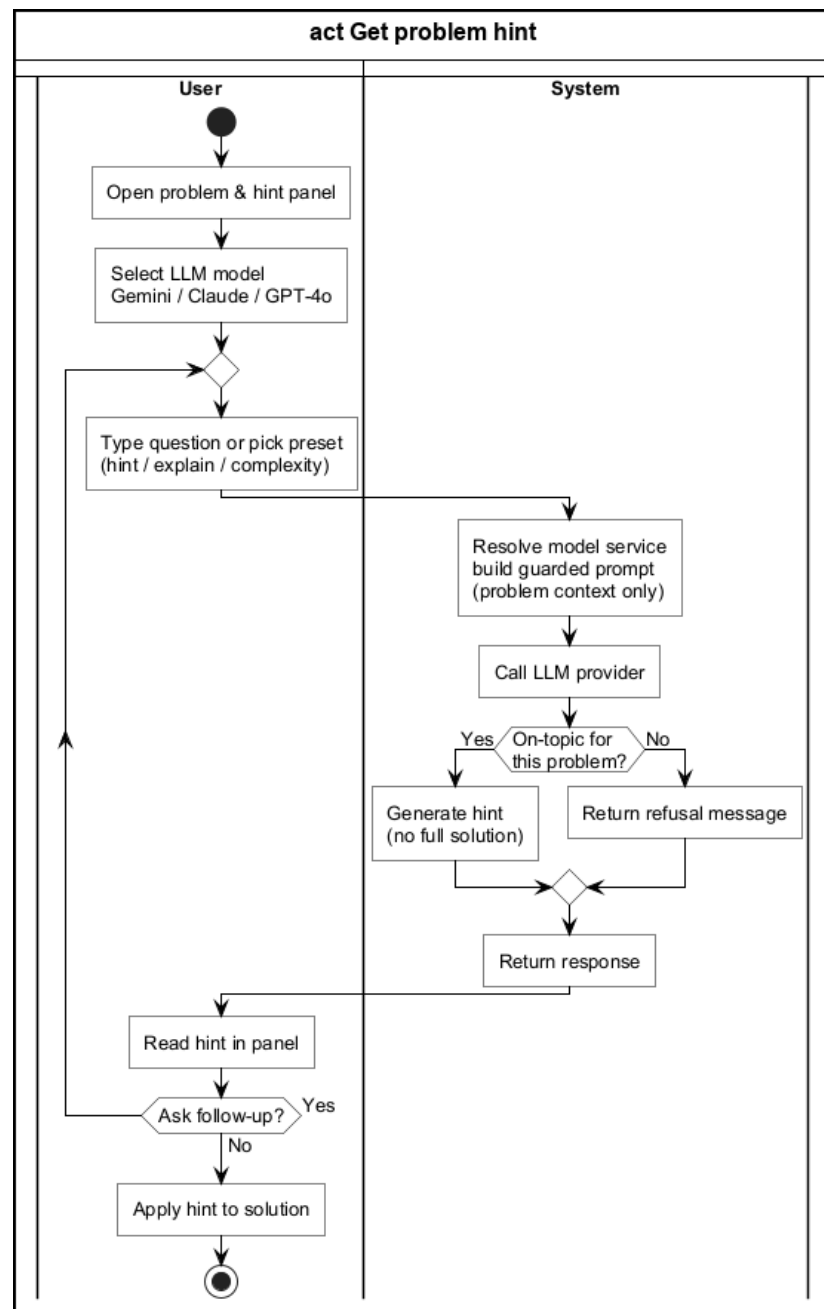
Hệ thống xác định dịch vụ mô hình tương ứng, dựng câu nhắc (prompt) được kiểm soát chỉ chứa ngữ cảnh của bài toán hiện tại, rồi gọi nhà cung cấp LLM. Nếu câu hỏi đúng phạm vi bài toán, hệ thống trả về gợi ý không kèm lời giải đầy đủ; ngược lại, hệ thống trả về thông báo từ chối nhằm ngăn lạm dụng. Bảng 3.7 trình bày đặc tả chi tiết của Use Case này.

Bảng 3.7: Bảng đặc tả Use case Gọi ý học tập bằng LLM

Use Case	Gợi ý học tập bằng LLM	
Bối cảnh	Người dùng gặp khó khăn với một bài toán và muốn nhận gợi ý định hướng thay vì lời giải hoàn chỉnh.	
Mô tả	Người dùng đặt câu hỏi cho mô hình ngôn ngữ lớn về bài toán; hệ thống dựng câu nhắc có kiểm soát và trả về gợi ý đúng phạm vi, không tiết lộ lời giải đầy đủ.	
Tác nhân	Người dùng (sinh viên); hệ thống LLM bên ngoài.	
Sự kiện kích hoạt	Người dùng gửi câu hỏi trong khung gợi ý của trang bài toán.	
Điều kiện tiên quyết	+ Người dùng đã đăng nhập và đang xem một bài toán cụ thể.	
Kết quả	Người dùng nhận gợi ý phù hợp với bài toán; các câu hỏi ngoài phạm vi bị từ chối.	
Luồng sự kiện	Tác nhân	Hệ thống
	1. Người dùng mở khung gợi ý và chọn mô hình ngôn ngữ lớn.	

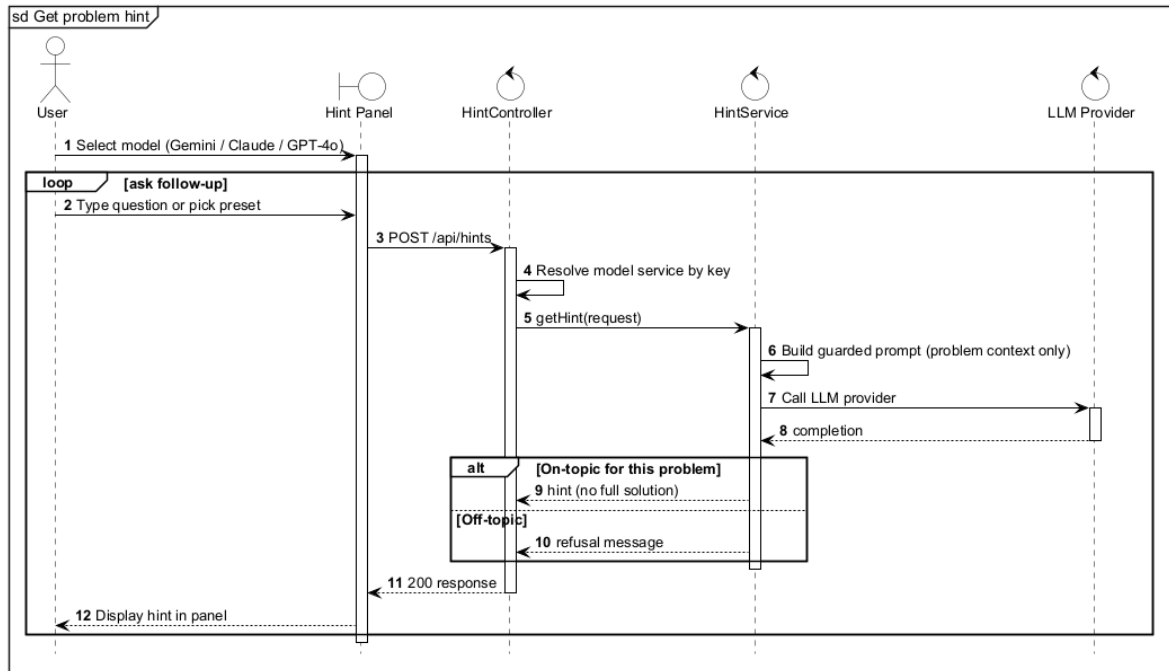
	2. Người dùng nhập câu hỏi hoặc chọn một mẫu yêu cầu sẵn có. 3. Người dùng đọc gợi ý và có thể đặt câu hỏi tiếp theo.	2.1 Hệ thống xác định dịch vụ mô hình tương ứng và dựng câu nhắc có kiểm soát, chỉ chứa ngữ cảnh bài toán. 2.2 Hệ thống gọi nhà cung cấp LLM và nhận phản hồi. 2.3 Nếu câu hỏi đúng phạm vi, hệ thống trả về gợi ý không kèm lời giải đầy đủ; nếu không, hệ thống trả về thông báo từ chối. 3.1 Hệ thống hiển thị phản hồi trong khung gợi ý.
Ngoại lệ	Câu hỏi nằm ngoài phạm vi bài toán $\Rightarrow$ hệ thống trả về thông báo từ chối và không sinh gợi ý.	

Luồng xử lý của Use Case được mô tả trong sơ đồ hoạt động ở Hình 3.10.



Hình 3.10: Sơ đồ hoạt động cho Use Case Gợi ý học tập bằng LLM

Sự tương tác giữa người dùng và các thành phần của hệ thống được thể hiện qua sơ đồ tuần tự trong Hình 3.11.



Hình 3.11: Sơ đồ tuần tự cho Use Case Gọi ý học tập bằng LLM

3.3.2.6 Use case Quản lý bài thực hành

Quản lý bài thực hành phục vụ hoạt động giảng dạy trong phạm vi một tổ chức. Giảng viên tạo một bài thực hành (lab), chọn các bài toán riêng tư thuộc sở hữu của mình cùng thời hạn nộp và giao cho các thành viên của tổ chức; hệ thống từ chối các bài toán công khai hoặc không thuộc quyền sở hữu nhằm bảo đảm đề bài chưa bị lộ. Khác với cuộc thi, bài toán trong bài thực hành được giữ nguyên trạng thái riêng tư sau khi hết thời hạn và có thể được dùng lại cho các bài thực hành khác. Trong quá trình học, sinh viên nộp lời giải cho các bài toán trong bài thực hành và hệ thống chấm bài, theo dõi số bài đã giải của từng sinh viên.

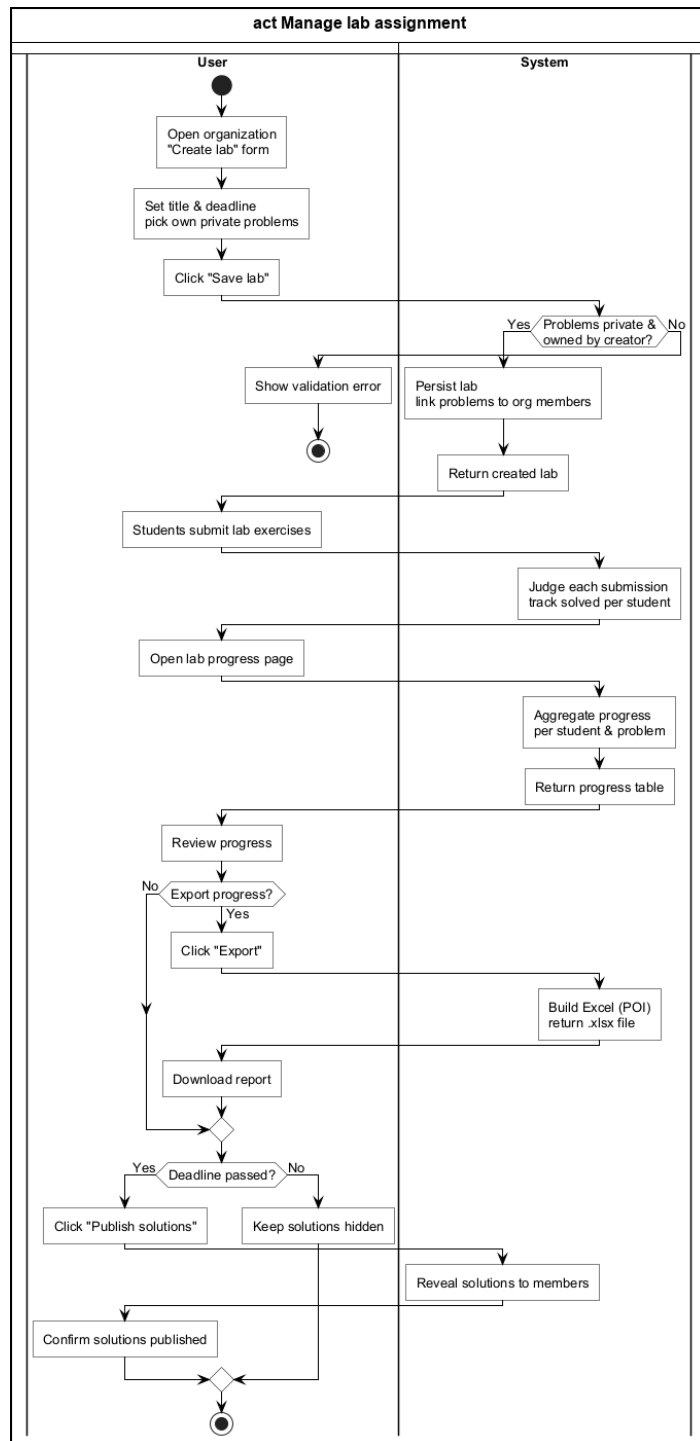
Giảng viên mở trang tiến độ để xem bảng tổng hợp kết quả theo từng sinh viên và từng bài toán, có thể kết xuất báo cáo ra tập tin Excel bằng thư viện Apache POI. Khi hết thời hạn, giảng viên có thể công bố lời giải để các thành viên tham khảo. Bảng 3.8 trình bày đặc tả chi tiết của Use Case này.

Bảng 3.8: Bảng đặc tả Use case Quản lý bài thực hành

Use Case	Quản lý bài thực hành	
Bối cảnh	Giảng viên muốn giao bài thực hành cho thành viên tổ chức và theo dõi tiến độ làm bài của họ.	
Mô tả	Giảng viên tạo bài thực hành gồm danh sách bài toán và thời hạn; hệ thống chấm bài nộp, tổng hợp tiến độ, hỗ trợ kết xuất báo cáo và công bố lời giải.	
Tác nhân	Contest Creator (giảng viên); Người dùng (sinh viên nộp bài).	
Sự kiện kích hoạt	Giảng viên mở biểu mẫu “Tạo bài thực hành”.	
Điều kiện tiên quyết	+ Người dùng đã đăng nhập với vai trò Contest Creator. + Tổ chức đã tồn tại và các bài toán cần dùng là bài riêng tư thuộc sở hữu của giảng viên.	
Kết quả	Bài thực hành được tạo và giao cho thành viên; tiến độ được tổng hợp, có thể kết xuất báo cáo và công bố lời giải sau thời hạn.	
Luồng sự kiện	Tác nhân	Hệ thống
	1. Giảng viên nhập tiêu đề, chọn các bài toán riêng tư của mình và thời hạn rồi nhấn “Lưu”. 2. Sinh viên nộp lời giải cho các bài toán trong bài thực hành.	1.1 Hệ thống kiểm tra các bài toán phải ở trạng thái riêng tư và thuộc sở hữu của giảng viên, sau đó lưu bài thực hành, liên kết các bài toán với thành viên tổ chức và trả về bài thực hành đã tạo. 2.1 Hệ thống chấm từng bài nộp và cập nhật số bài đã giải của mỗi sinh viên.

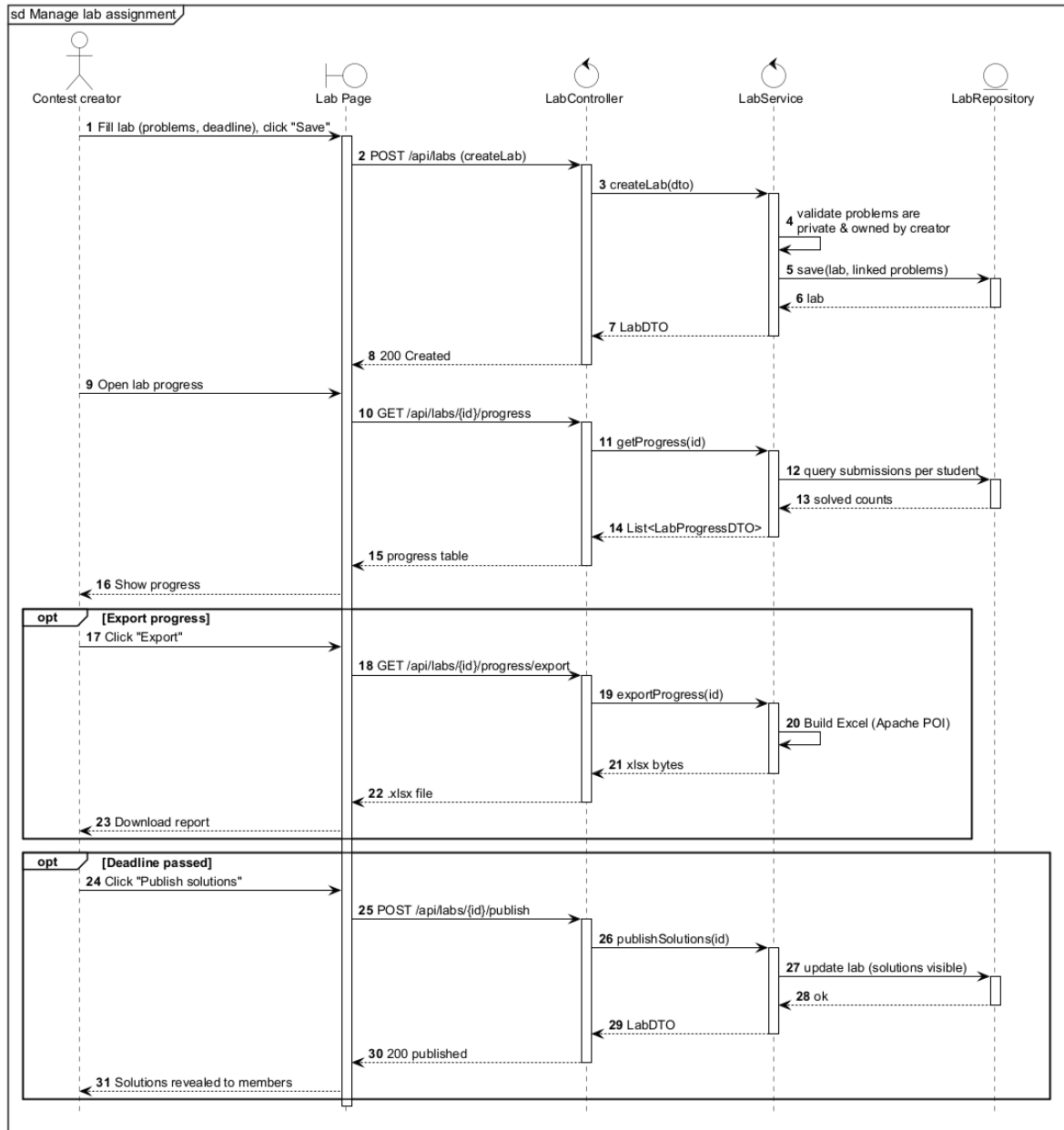
	3. Giảng viên mở trang tiến độ. 4. (Tùy chọn) Giảng viên nhấn nút “Kết xuất”. 5. (Tùy chọn) Sau thời hạn, giảng viên nhấn “Công bố lời giải”.	3.1 Hệ thống tổng hợp tiến độ theo từng sinh viên và từng bài toán, trả về bảng tiến độ. 4.1 Hệ thống dựng tập tin Excel bằng Apache POI và trả về để giảng viên tải xuống. 5.1 Hệ thống mở quyền xem lời giải cho các thành viên của tổ chức.
Ngoại lệ	Thiếu thông tin bắt buộc, hoặc danh sách chứa bài toán công khai / không thuộc sở hữu của giảng viên $\Rightarrow$ hệ thống hiển thị thông báo lỗi và không lưu.	

Luồng xử lý của Use Case được mô tả trong sơ đồ hoạt động ở Hình 3.12.



Hình 3.12: Sơ đồ hoạt động cho Use Case Quản lý bài thực hành

Sự tương tác giữa giảng viên và các thành phần của hệ thống được thể hiện qua sơ đồ tuần tự trong Hình 3.13.



Hình 3.13: Sơ đồ tuần tự cho Use Case Quản lý bài thực hành

3.4 Yêu cầu phi chức năng (Non-functional Requirements)

Bên cạnh các chức năng nghiệp vụ, hệ thống chấm bài trực tuyến còn phải đáp ứng một số yêu cầu phi chức năng đặc thù. Do phải tiếp nhận và thực thi mã nguồn không tin cậy từ nhiều người dùng cùng lúc, đặc biệt trong các kỳ thi, các yêu cầu về hiệu năng, khả năng mở rộng và bảo mật được đặt lên hàng đầu. Phần này trình bày những yêu cầu phi chức năng chính mà TDTUOJ hướng tới.

3.4.1 Hiệu năng và thời gian phản hồi

Thao tác nộp bài không được làm nghẽn giao diện người dùng. Vì vậy việc chấm bài được xử lý bất đồng bộ: bài nộp được ghi nhận ngay với trạng thái chờ xử lý (PENDING), đưa vào hàng đợi Redis và do các tiến trình xử lý (worker) lấy ra gửi sang Judge0, nhờ đó người dùng nhận phản hồi tức thì mà không phải chờ kết thúc quá trình chấm. Các thao tác đọc thường gặp như xem danh sách bài toán hay bảng xếp hạng được tối ưu bằng bộ nhớ đệm Redis để giảm tải cho cơ sở dữ liệu. Mỗi bài nộp đều bị ràng buộc giới hạn thời gian và bộ nhớ do Judge0 áp đặt, bảo đảm một lời giải sai hoặc kém hiệu quả không chiếm dụng tài nguyên vô hạn.

3.4.2 Khả năng mở rộng

Hệ thống được thiết kế để chịu được lượng truy cập và bài nộp tăng đột biến trong thời gian diễn ra kỳ thi. Phía máy chủ áp dụng cơ chế xác thực phi trạng thái bằng JWT nên có thể nhân bản thành nhiều thực thể và cân bằng tải theo chiều ngang mà không cần chia sẻ phiên làm việc. Khâu chấm bài được tách rời qua hàng đợi, cho phép tăng số lượng worker và số máy chủ Judge0 một cách độc lập khi tải tăng. Toàn bộ thành phần được đóng gói bằng Docker, thuận tiện cho việc triển khai và nhân rộng.

3.4.3 Bảo mật

Bảo mật là yêu cầu trọng yếu do hệ thống thực thi mã nguồn tùy ý từ người dùng. Mã nguồn nộp lên luôn được chạy trong môi trường cô lập bằng container của Judge0, tách biệt khỏi máy chủ ứng dụng nhằm ngăn mã độc truy cập hệ thống hoặc dữ liệu. Việc truy cập được kiểm soát bằng JWT kết hợp phân quyền theo vai trò (User, Contest Creator, Admin) thông qua các chú thích phân quyền ở mức phương thức; mật khẩu được băm bằng thuật toán BCrypt trước khi lưu. Chính sách Cross-Origin Resource Sharing (CORS) giới hạn nguồn gọi API, còn tính năng gợi ý bằng LLM dùng câu nhắc được kiểm soát chặt để tránh tiết lộ lời giải hoặc bị lạm dụng ngoài phạm vi học tập.

3.4.4 Độ tin cậy và tính sẵn sàng

Kết quả chấm bài cùng dữ liệu xếp hạng phải chính xác và không được mất mát. Trạng thái của mỗi bài nộp được lưu bền vững qua các giai đoạn chờ xử lý, đang chấm và hoàn tất, nên một sự cố tạm thời của worker hay của Judge0 không làm thất lạc bài nộp đã tiếp nhận. Cơ chế chấm bất đồng bộ qua hàng đợi cũng giúp hệ thống tiếp tục nhận bài ngay cả khi khâu chấm tạm thời quá tải, các bài nộp dồn lại sẽ được xử lý dần khi tài nguyên được giải phóng.

3.4.5 Khả năng bảo trì và khả năng sử dụng

Mã nguồn phía máy chủ được tổ chức theo kiến trúc phân tầng Controller – Service – Repository và chia thành các module theo nghiệp vụ, giúp dễ đọc, dễ kiểm thử và dễ mở rộng tính năng mới. Toàn bộ API được mô tả tự động bằng OpenAPI để thuận tiện cho việc tích hợp và bảo trì. Về phía người dùng, giao diện hỗ trợ trình soạn thảo mã nguồn Monaco cùng trình minh họa cấu trúc dữ liệu, mang lại trải nghiệm gần với môi trường lập trình thực tế và hỗ trợ tốt cho mục tiêu học tập.

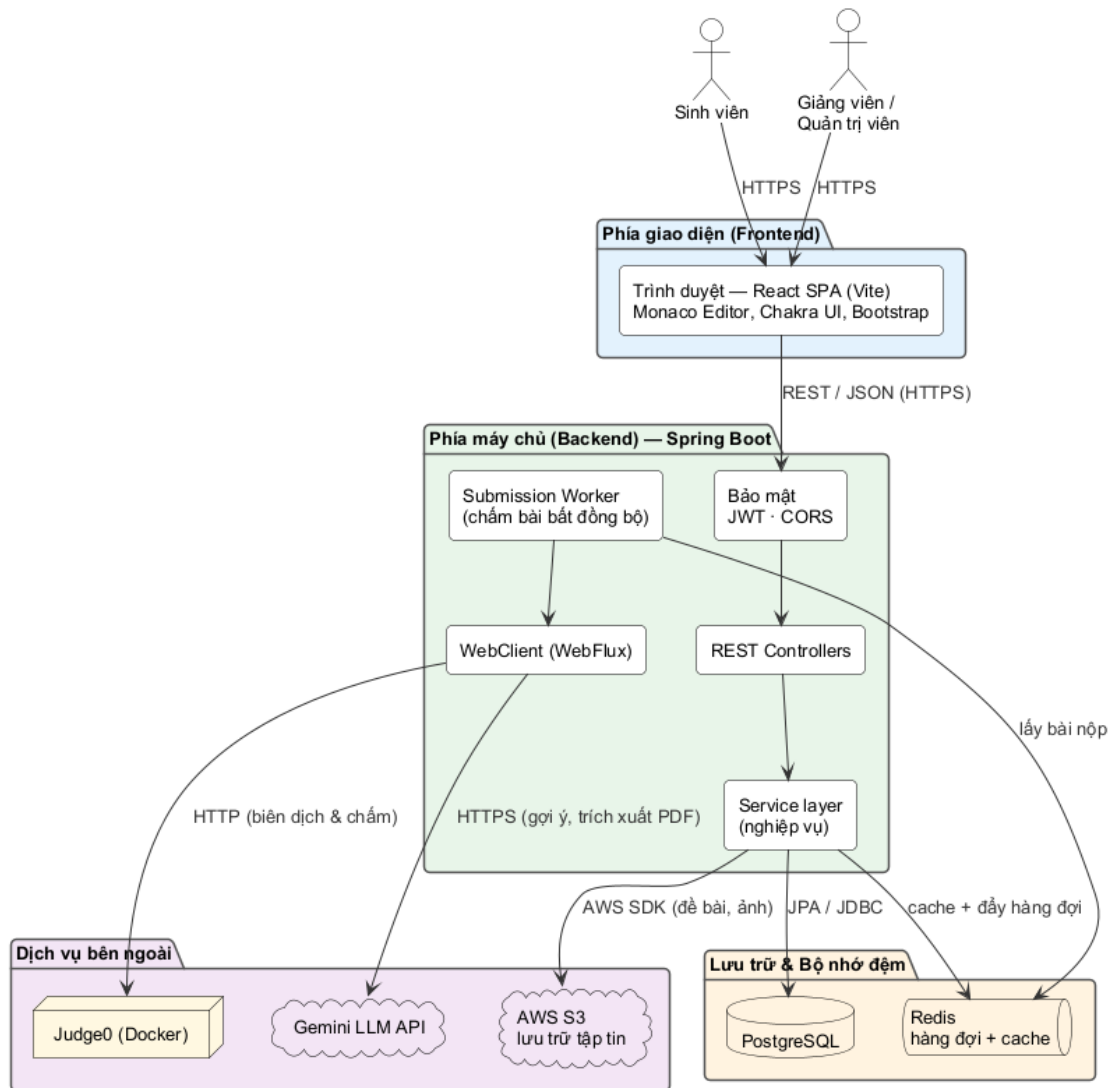
CHƯƠNG 4. THIẾT KẾ HỆ THỐNG

4.1 Thiết kế kiến trúc

Hệ thống TDTUOJ được xây dựng theo kiến trúc nhiều tầng với sự tách biệt rõ ràng giữa phía giao diện người dùng (frontend) và phía máy chủ (backend), giao tiếp với nhau qua các giao diện lập trình ứng dụng (API) dạng REST. Cách tổ chức này giúp hai phía phát triển và triển khai độc lập, đồng thời thuận tiện cho việc mở rộng theo chiều ngang. Kiến trúc tổng quan của hệ thống được trình bày trong Hình 4.1.

Phía giao diện là một ứng dụng trang đơn (Single Page Application) viết bằng React, tích hợp trình soạn thảo mã nguồn Monaco; toàn bộ tương tác với máy chủ được thực hiện qua REST trên giao thức HTTP. Phía máy chủ là một ứng dụng Spring Boot, gồm tầng bảo mật (xác thực JWT và kiểm soát CORS), tầng REST Controller tiếp nhận yêu cầu, tầng Service xử lý nghiệp vụ, một tiến trình xử lý bài nộp (worker) chấm bài bất đồng bộ và một WebClient dựa trên WebFlux để gọi các dịch vụ bên ngoài.

Dữ liệu nghiệp vụ được lưu trong cơ sở dữ liệu quan hệ PostgreSQL, trong khi Redis vừa đóng vai trò bộ nhớ đệm vừa làm hàng đợi cho khâu chấm bài. Khi tiếp nhận một bài nộp, tầng Service ghi bản ghi vào PostgreSQL và đẩy công việc vào hàng đợi Redis; tiến trình worker lấy bài nộp ra khỏi hàng đợi và gửi đến Judge0 (chạy trong Docker) để biên dịch và thực thi. Ngoài ra, hệ thống còn kết nối tới hai dịch vụ bên ngoài khác là AWS S3 để lưu trữ tập tin đề bài cùng ảnh đại diện, và Gemini LLM để sinh gợi ý và trích xuất đề bài từ tập tin PDF.



Hình 4.1: Kiến trúc tổng quan của hệ thống TDTUOJ

4.2 Thiết kế cơ sở dữ liệu / ERD

4.2.1 Tổng quan

Cơ sở dữ liệu của hệ thống TDTUOJ được xây dựng theo mô hình quan hệ trên hệ quản trị PostgreSQL, trong đó mỗi thực thể nghiệp vụ được ánh xạ thành một bảng và các mối liên kết được thể hiện thông qua khóa ngoại. Thiết kế hướng đến bảo đảm tính toàn vẹn dữ liệu trong các tình huống nhiều bài nộp xử lý đồng thời, đồng thời giữ cấu trúc đủ đơn giản để truy vấn hiệu quả và mở rộng về sau.

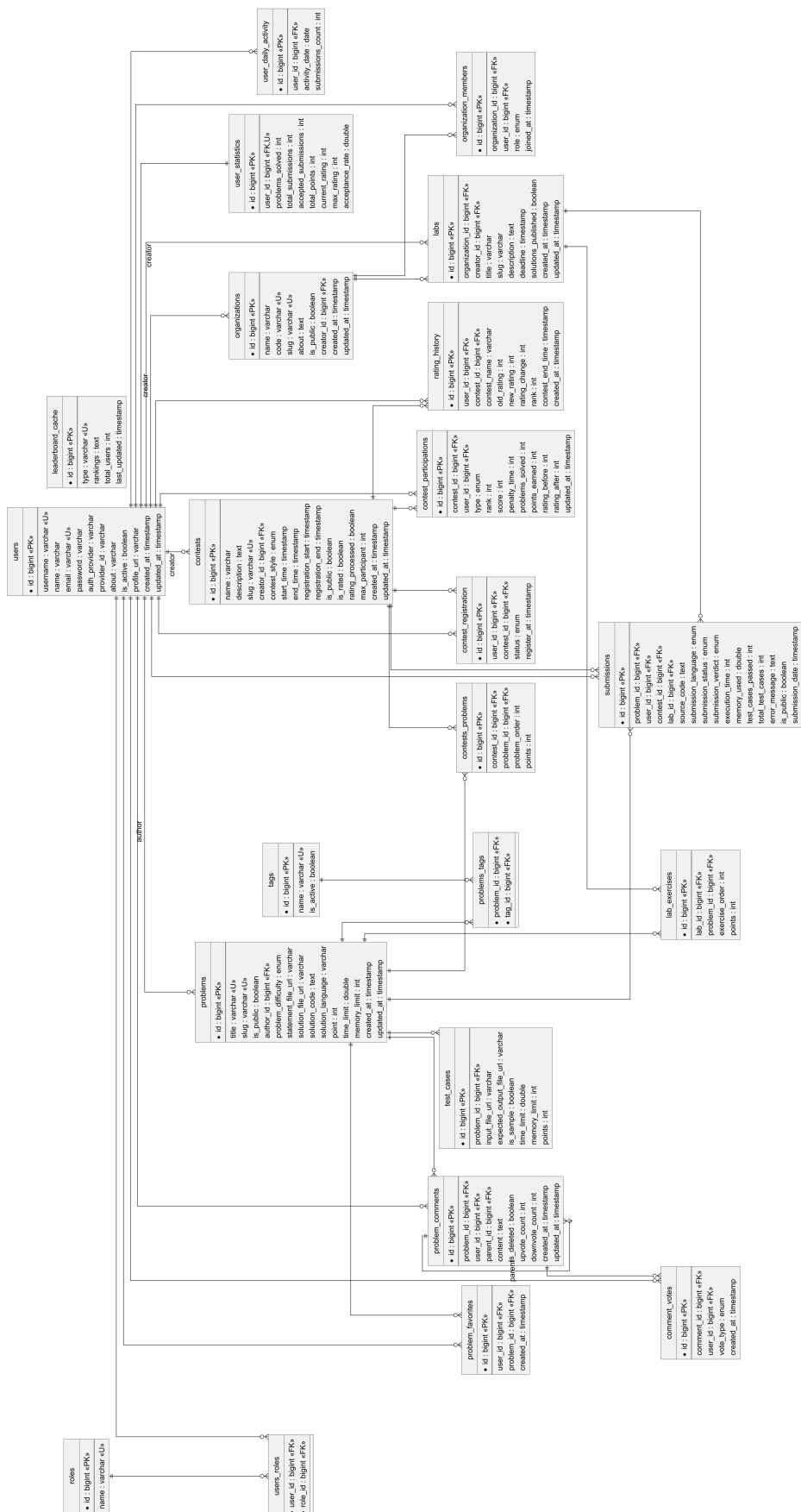
4.2.2 Sơ đồ thực thể – quan hệ (ERD)

Sơ đồ thực thể – quan hệ (Entity-Relationship Diagram – Sơ đồ thực thể quan hệ (ERD)) là công cụ trực quan mô tả cấu trúc của cơ sở dữ liệu, gồm ba thành phần chính. Thực thể (entity) tương ứng với một bảng dữ liệu, ví dụ người dùng hay bài toán. Thuộc tính (attribute) là các cột dữ liệu mô tả một thực thể, chẳng hạn tên đăng nhập hay địa chỉ thư điện tử. Quan hệ (relationship) thể hiện liên kết giữa các thực thể, cho biết chúng phối hợp với nhau như thế nào trong nghiệp vụ.

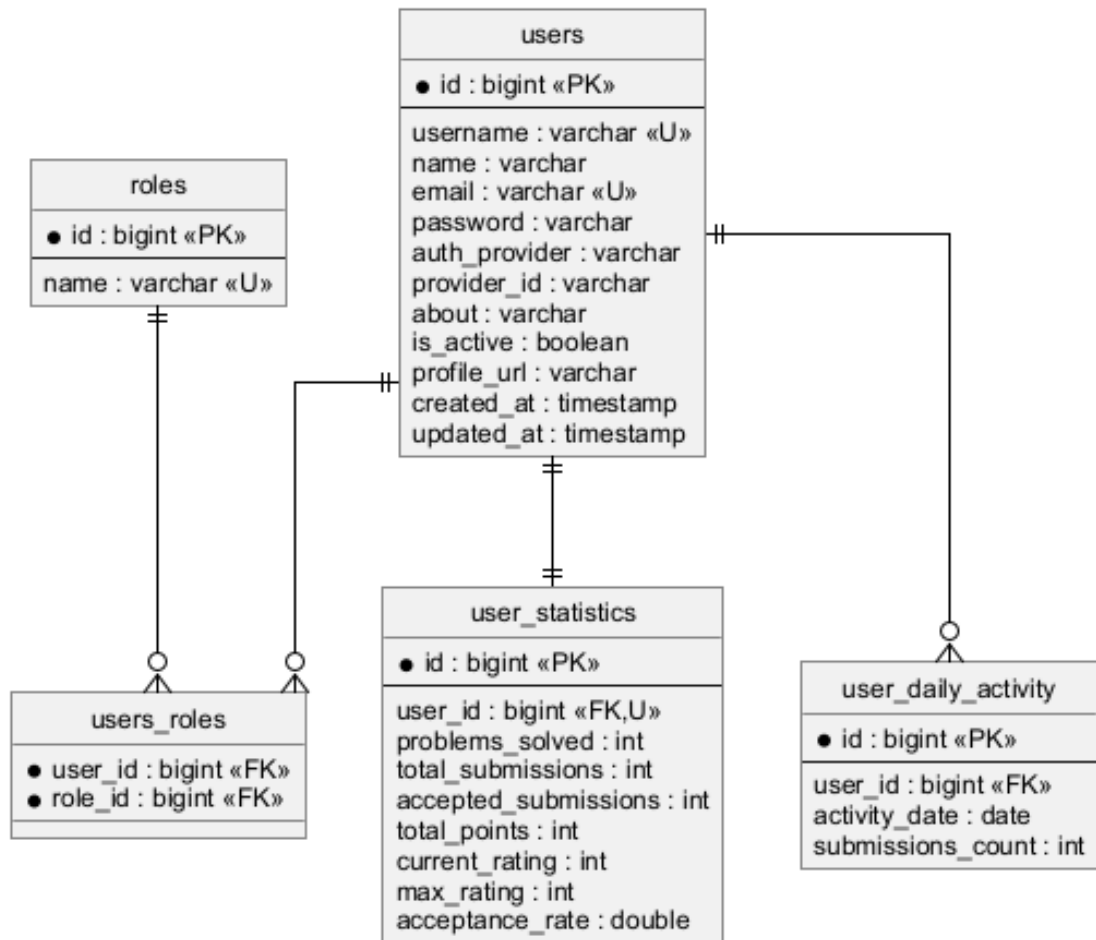
Sơ đồ ERD tổng quan của hệ thống được biểu diễn theo ký pháp chân chim (Crow's foot) trong Hình 4.2. Trong sơ đồ, mỗi thực thể tương ứng với một bảng, khóa chính được đánh dấu Primary Key – Khóa chính (PK), khóa ngoại đánh dấu Foreign Key – Khóa ngoại (FK) và các ràng buộc duy nhất đánh dấu U; ký pháp chân chim ở hai đầu đường nối thể hiện lượng số (cardinality) của quan hệ. Toàn bộ lược đồ gồm hơn hai mươi bảng, được tổ chức xoay quanh một số thực thể trung tâm. Để thuận tiện trình bày, các mục dưới đây tách sơ đồ thành từng phân mảnh theo thực thể trung tâm và mô tả lần lượt; mỗi mục liệt kê các thuộc tính chính cùng các quan hệ của thực thể tương ứng, trong đó những bảng liên kết và bảng phụ trợ được trình bày kèm trong phần quan hệ của bảng chủ.

4.2.2.1 Thực thể Người dùng (User)

Bảng users là thực thể trung tâm của hệ thống, lưu trữ thông tin tài khoản của mọi người dùng. Hầu hết hoạt động nghiệp vụ như nộp bài, bình luận, tham gia cuộc thi hay tạo nội dung đều gắn với một người dùng. Bảng hỗ trợ cả đăng nhập nội bộ và đăng nhập qua nhà cung cấp bên ngoài thông qua trường auth_provider. Bảng 4.1 mô tả chi tiết thực thể này. Phân mảnh sơ đồ tương ứng được minh họa trong Hình 4.3.



Hình 4.2: Sơ đồ thực thể – quan hệ của hệ thống TDTUOJ



Hình 4.3: Phân mảnh ERD – Người dùng

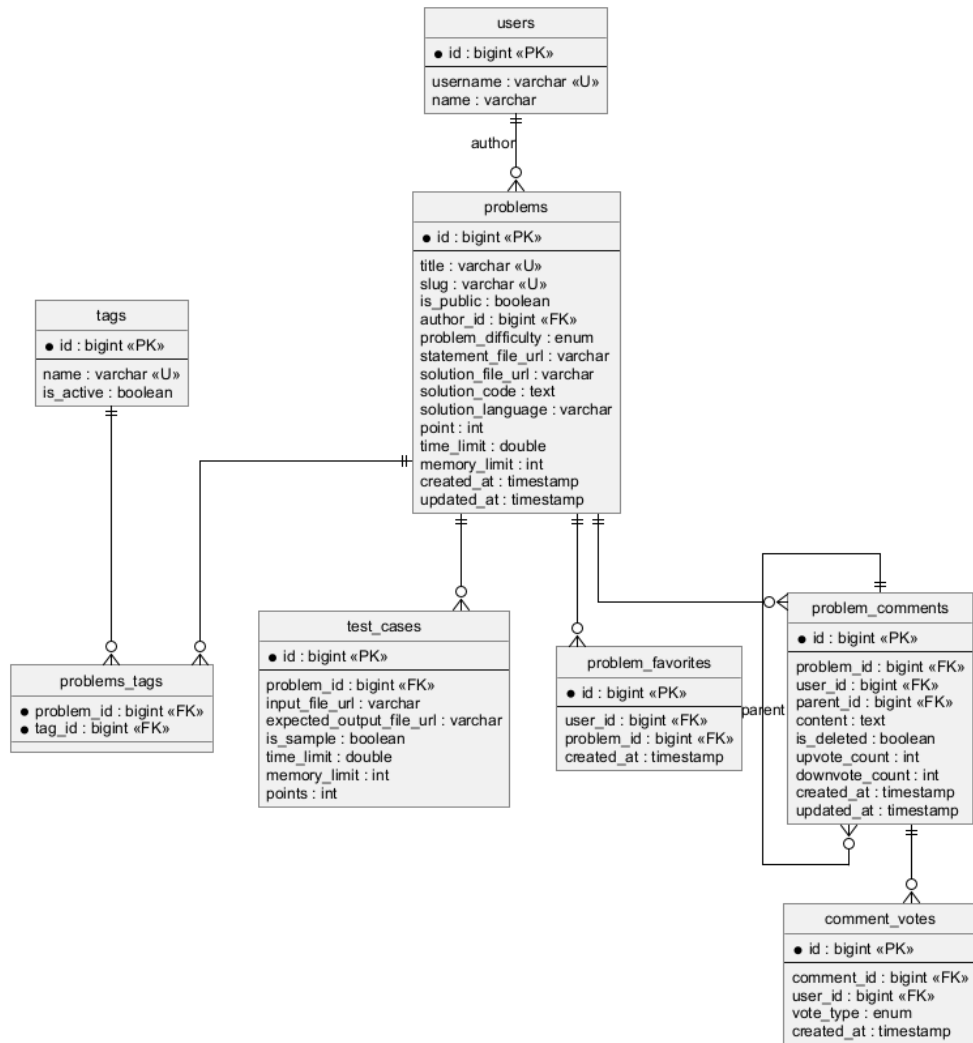
Bảng 4.1: Mô tả chi tiết thực thể users

Thuộc tính	
id	Khóa chính, định danh người dùng.
username	Tên đăng nhập dạng slug, ràng buộc duy nhất.
name	Tên hiển thị của người dùng.
email	Địa chỉ thư điện tử, ràng buộc duy nhất.
password	Mật khẩu đã được băm bằng BCrypt.
auth_provider, provider_id	Nhà cung cấp đăng nhập (nội bộ hoặc bên ngoài) và định danh tương ứng.
about, profile_url	Giới thiệu ngắn và đường dẫn ảnh đại diện.

is_active	Trạng thái kích hoạt của tài khoản.
created_at, up-dated_at	Thời điểm tạo và cập nhật gần nhất.
Quan hệ	
roles	Nhiều – nhiều với bảng roles thông qua bảng liên kết users_roles; một người dùng có thể mang nhiều vai trò (User, Contest Creator, Admin).
user_statistics	Một – một với bảng user_statistics lưu thống kê tổng hợp (số bài đã giải, điểm xếp hạng...).
user_daily_activity	Một – nhiều; ghi nhận số bài nộp theo từng ngày phục vụ biểu đồ hoạt động.
Nội dung do người dùng tạo	Một – nhiều tới các bảng problems (tác giả), submissions, problem_comments, comment_votes, problem_favorites và rating_history.

4.2.2.2 Thực thể Bài toán (Problem)

Bảng problems lưu trữ đề bài cùng các cấu hình chấm. Mỗi bài toán thuộc về một tác giả, có thể được gán nhiều thể phân loại, chứa nhiều trường hợp kiểm thử và được sử dụng lại trong cuộc thi hoặc bài thực hành. Bảng 4.2 mô tả chi tiết thực thể này. Phân mảnh sơ đồ tương ứng được minh họa trong Hình 4.4.



Hình 4.4: Phân mảnh ERD – Bài toán

Bảng 4.2: Mô tả chi tiết thực thể problems

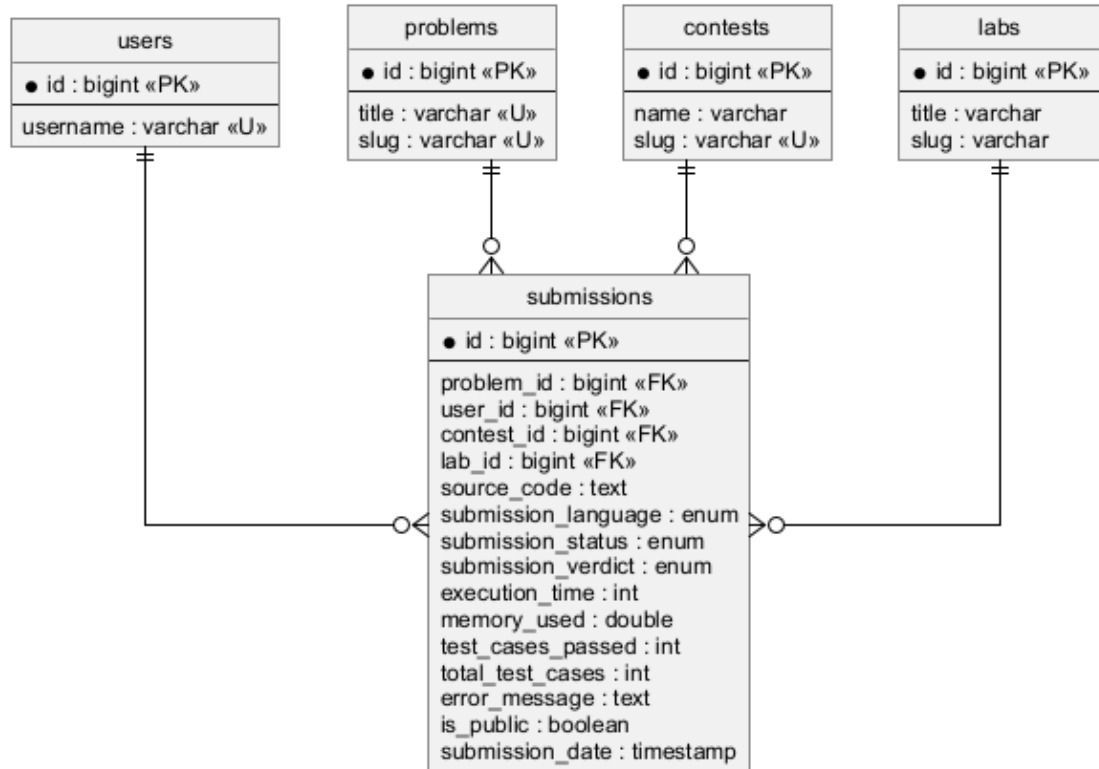
Thuộc tính	
id	Khóa chính, định danh bài toán.
title, slug	Tiêu đề và đường dẫn rút gọn, điều ràng buộc duy nhất.
is_public	Phạm vi hiển thị; bài toán riêng tư chỉ dùng trong cuộc thi hoặc bài thực hành.
author_id	Khóa ngoại tới users, là người tạo bài toán.

problem_difficulty	Mức độ khó (EASY, MEDIUM, HARD).
statement_file_url, solution_file_url	Đường dẫn trên S3 tới tập tin đề bài và lời giải.
solution_code, solution_language	Mã nguồn lời giải mẫu và ngôn ngữ tương ứng.
point	Điểm của bài toán.
time_limit, memory_limit	Giới hạn thời gian (giây) và bộ nhớ (KB).
created_at, updated_at	Thời điểm tạo và cập nhật gần nhất.
Quan hệ	
author	Nhiều – một tới users; mỗi bài toán có một tác giả.
test_cases	Một – nhiều; mỗi bài toán có nhiều trường hợp kiểm thử (đầu vào, đầu ra mong đợi, giới hạn, đánh dấu là ví dụ mẫu).
tags	Nhiều – nhiều với bảng tags qua bảng liên kết problems_tags.
submissions	Một – nhiều; mỗi bài toán nhận nhiều bài nộp.
Sử dụng lại	Một – nhiều tới contests_problems và lab_exercises khi bài toán được đưa vào cuộc thi hoặc bài thực hành.
Tương tác người dùng	Một – nhiều tới problem_comments và problem_favorites.

4.2.2.3 Thực thể Bài nộp (Submission)

Bảng submissions lưu mỗi lần người dùng nộp lời giải cùng kết quả chấm tương ứng. Mỗi bài nộp luôn gắn với một bài toán và một người dùng; ngoài ra có thể

gắn với một cuộc thi hoặc một bài thực hành tùy theo bối cảnh nộp. Bảng 4.3 mô tả chi tiết thực thể này. Phân mảnh sơ đồ tương ứng được minh họa trong Hình 4.5.



Hình 4.5: Phân mảnh ERD – Bài nộp

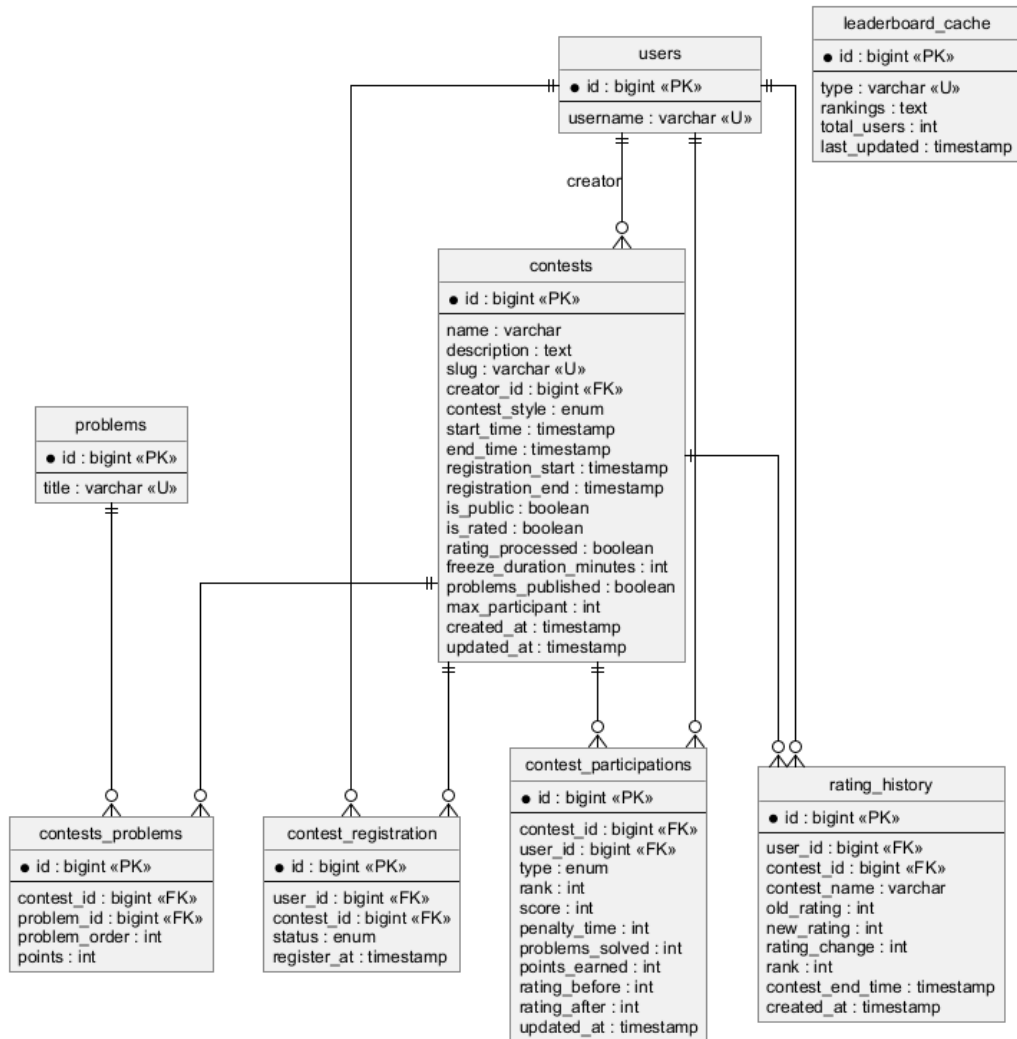
Bảng 4.3: Mô tả chi tiết thực thể submissions

Thuộc tính	
id	Khóa chính, định danh bài nộp.
source_code	Mã nguồn lời giải do người dùng nộp.
submission_language	Ngôn ngữ lập trình (PYTHON, JAVA, C, CPP).
submission_status	Trạng thái xử lý (PENDING, JUDGING, COMPLETED).
submission_verdict	Kết quả chấm (AC, WA, TLE, MLE, RE, CE).

execution_time, memory_used	Thời gian thực thi và bộ nhớ sử dụng.
test_cases_passed, total_test_cases	Số trường hợp kiểm thử đã vượt qua trên tổng số.
error_message	Thông báo lỗi khi biên dịch hoặc khi chạy (nếu có).
is_public	Cho biết bài nộp có được hiển thị công khai hay không.
submission_date	Thời điểm nộp bài.
Quan hệ	
problem_id	Nhiều – một tới problems; bài toán được nộp.
user_id	Nhiều – một tới users; người nộp bài.
contest_id	Nhiều – một tới contests (tùy chọn) khi bài nộp thuộc một cuộc thi.
lab_id	Nhiều – một tới labs (tùy chọn) khi bài nộp thuộc một bài thực hành.

4.2.2.4 Thực thể Cuộc thi (Contest)

Bảng contests lưu thông tin các cuộc thi tổ chức trên hệ thống, theo thể thức ICPC. Một cuộc thi do một người tạo lập, chứa nhiều bài toán, ghi nhận đăng ký và quá trình tham gia của người dùng. Bảng 4.4 mô tả chi tiết thực thể này. Phân mảnh sơ đồ tương ứng được minh họa trong Hình 4.6.



Hình 4.6: Phân mảnh ERD – Cuộc thi

Bảng 4.4: Mô tả chi tiết thực thể contests

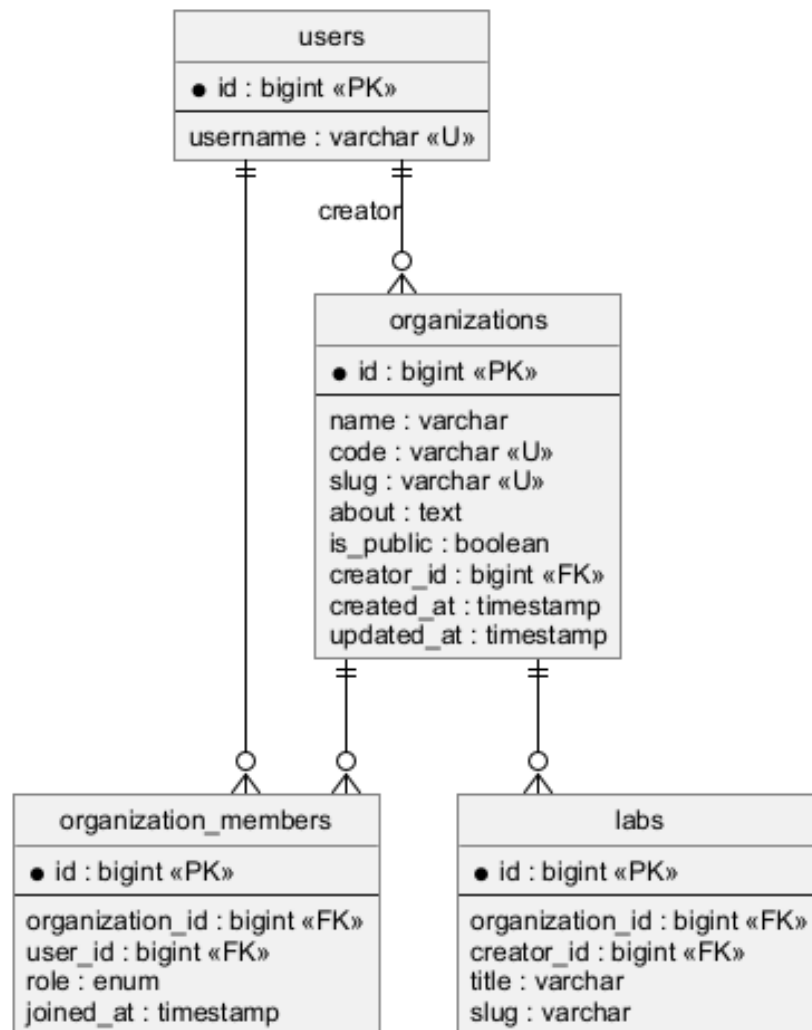
Thuộc tính	
id	Khóa chính, định danh cuộc thi.
name, slug	Tên cuộc thi và đường dẫn rút gọn (duy nhất).
description	Mô tả chi tiết cuộc thi.
creator_id	Khóa ngoại tới users, là người tạo cuộc thi.
contest_style	Thể thức tính điểm (ICPC).

start_time, end_time	Thời gian bắt đầu và kết thúc.
registration_start, registration_end	Khoảng thời gian mở đăng ký.
is_public, is_rated	Cuộc thi công khai hay không, có tính điểm xếp hạng hay không.
rating_processed	Đánh dấu đã cập nhật điểm xếp hạng sau cuộc thi.
freeze_duration_minutes	Số phút đóng băng bảng xếp hạng trước khi kết thúc (rỗng hoặc 0 nghĩa là không đóng băng).
problems_published	Đánh dấu các bài toán của cuộc thi đã được tự động công khai sau khi kết thúc.
max_participant	Số lượng người tham gia tối đa.
created_at, updated_at	Thời điểm tạo và cập nhật gần nhất.
Quan hệ	
creator	Nhiều – một tới users; mỗi cuộc thi có một người tạo.
contests_problems	Một – nhiều; liên kết nhiều – nhiều với problems, kèm thứ tự và điểm riêng trong cuộc thi.
contest_registration	Một – nhiều; lưu đăng ký tham gia kèm trạng thái phê duyệt.
contest_participations	Một – nhiều; lưu điểm, thứ hạng và thời gian phạt của từng người tham gia.

rating_history	Một – nhiều; lịch sử thay đổi điểm xếp hạng sau cuộc thi. Bảng leaderboard_cache lưu ảnh chụp bảng xếp hạng dưới dạng JSON, không ràng buộc khóa ngoại cứng.
----------------	--

4.2.2.5 Thực thể Tổ chức (Organization)

Bảng organizations đại diện cho một đơn vị như lớp học hoặc câu lạc bộ, là nơi giảng viên quản lý thành viên và giao bài thực hành. Bảng 4.5 mô tả chi tiết thực thể này. Phân mảnh sơ đồ tương ứng được minh họa trong Hình 4.7.



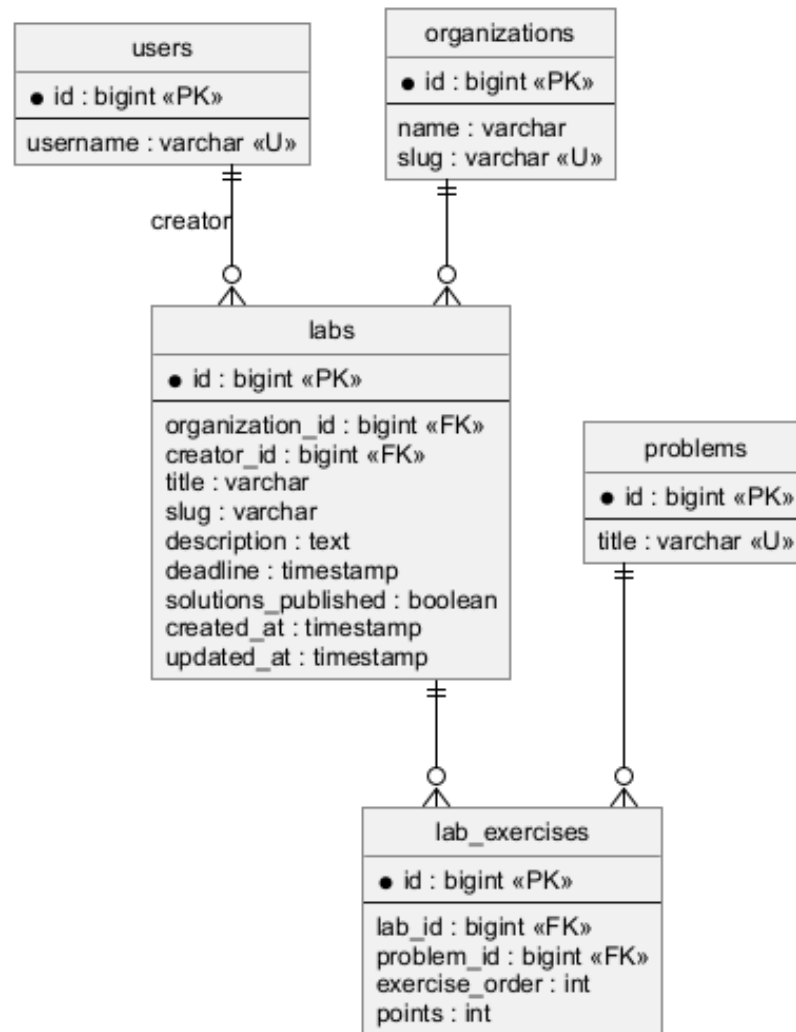
Hình 4.7: Phân mảnh ERD – Tổ chức

Bảng 4.5: Mô tả chi tiết thực thể organizations

Thuộc tính	
id	Khóa chính, định danh tổ chức.
name	Tên tổ chức.
code, slug	Mã tham gia và đường dẫn rút gọn, đều ràng buộc duy nhất.
about	Giới thiệu về tổ chức.
is_public	Phạm vi hiển thị của tổ chức.
creator_id	Khóa ngoại tới users, là người tạo tổ chức.
created_at, up-dated_at	Thời điểm tạo và cập nhật gần nhất.
Quan hệ	
creator	Nhiều – một tới users; mỗi tổ chức có một người tạo.
organization_members	Một – nhiều; liên kết nhiều – nhiều với users, kèm vai trò thành viên (OWNER, ADMIN, MEMBER).
labs	Một – nhiều; mỗi tổ chức chứa nhiều bài thực hành.

4.2.2.6 Thực thể Bài thực hành (Lab)

Bảng labs lưu các bài thực hành do giảng viên giao trong phạm vi một tổ chức, gồm danh sách bài toán và thời hạn nộp. Bảng 4.6 mô tả chi tiết thực thể này. Phân mảnh sơ đồ tương ứng được minh họa trong Hình 4.8.



Hình 4.8: Phân mảnh ERD – Bài thực hành

Bảng 4.6: Mô tả chi tiết thực thể labs

Thuộc tính	
id	Khóa chính, định danh bài thực hành.
organization_id	Khóa ngoại tới organizations chứa bài thực hành.
creator_id	Khóa ngoại tới users, là giảng viên tạo bài thực hành.
title, slug	Tiêu đề và đường dẫn rút gọn.

description	Mô tả nội dung bài thực hành.
deadline	Thời hạn nộp bài.
solutions_published	Đánh dấu lời giải đã được công bố cho thành viên hay chưa.
created_at, up-dated_at	Thời điểm tạo và cập nhật gần nhất.
Quan hệ	
organization	Nhiều – một tới organizations.
creator	Nhiều – một tới users.
lab_exercises	Một – nhiều; liên kết nhiều – nhiều với problems, kèm thứ tự và điểm của từng bài trong bài thực hành.

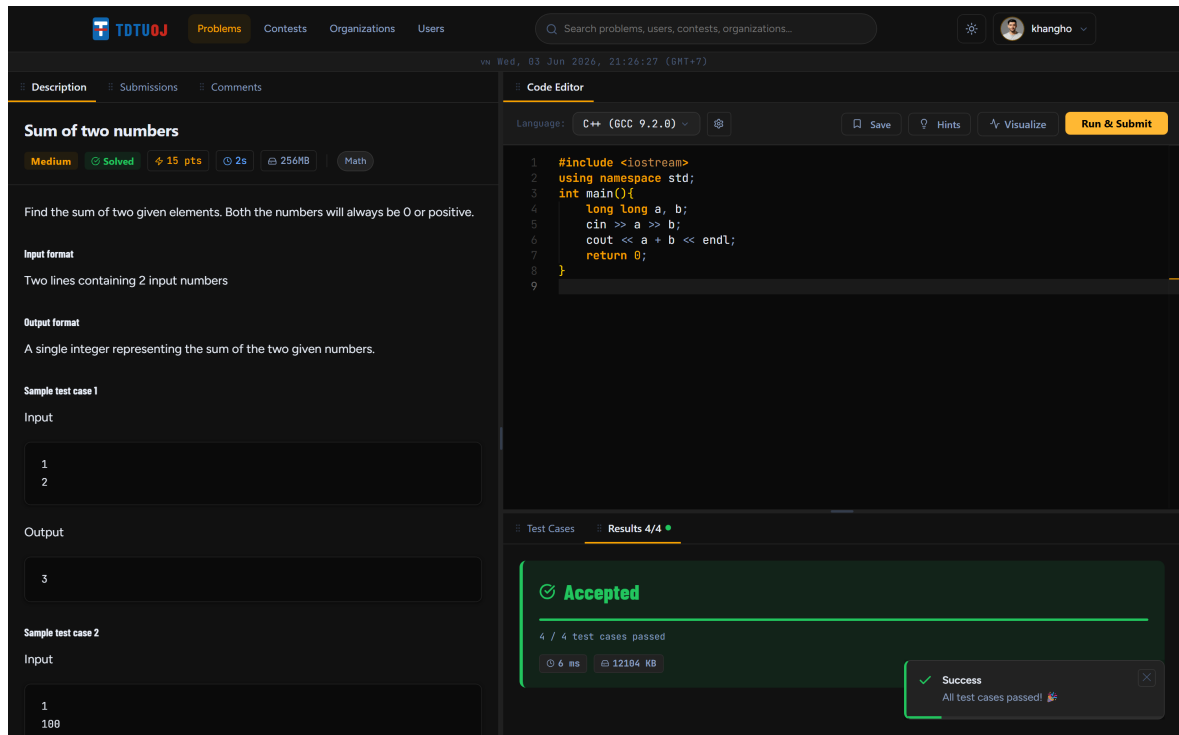
4.3 Thiết kế giao diện

Giao diện người dùng được xây dựng bằng React theo mô hình thành phần (component), kết hợp hai thư viện trình bày là Chakra UI và Bootstrap. Toàn bộ ứng dụng tuân theo một ngôn ngữ thị giác thống nhất với nền tối, màu nhấn vàng cam và thanh điều hướng cố định ở đầu trang, nhờ đó người dùng dễ dàng di chuyển giữa các phân hệ bài toán, cuộc thi, tổ chức và người dùng. Bố cục được thiết kế theo hướng đáp ứng (responsive), tự điều chỉnh theo kích thước màn hình, đồng thời ưu tiên tính rõ ràng của thông tin để phục vụ thao tác luyện tập và thi đấu. Các tiểu mục dưới đây trình bày những màn hình tiêu biểu của hệ thống.

4.3.1 Trang chi tiết bài toán và nộp bài

Trang chi tiết bài toán là màn hình làm việc trọng tâm của người dùng, được chia thành hai khu vực như Hình 4.9. Khu vực bên trái trình bày đề bài gồm phát biểu, định dạng dữ liệu vào – ra và các bộ dữ liệu mẫu. Khu vực bên phải là trình soạn thảo mã nguồn Monaco, cho phép chọn ngôn ngữ lập trình và viết lời giải trực

tiếp trên trình duyệt. Khi người dùng nộp bài, mã nguồn được gửi tới hệ thống chấm và kết quả được hiển thị ở bảng phía dưới: trong hình, lời giải được chấm đạt toàn bộ bốn trên bốn trường hợp kiểm thử kèm thông tin thời gian và bộ nhớ tiêu thụ.



Hình 4.9: Giao diện trang chi tiết bài toán và kết quả nộp bài

4.3.2 Trang cuộc thi và bảng xếp hạng

Trang cuộc thi cung cấp thông tin chi tiết của một cuộc thi cùng bảng xếp hạng theo thời gian thực. Như Hình 4.10 thể hiện, đầu trang hiển thị tên, thể thức, thời gian diễn ra và trạng thái của cuộc thi, kèm bảng thông tin tóm tắt ở cạnh phải. Phần thân trang gồm hai thẻ là danh sách bài toán và bảng xếp hạng; bảng xếp hạng liệt kê thứ hạng, tên thí sinh, số bài giải được và điểm phạt, được cập nhật tự động trong suốt quá trình thi đấu.

The screenshot displays the TDTU Game contest page. At the top, there's a navigation bar with 'Problems', 'Contests', 'Organizations', and 'Users'. A search bar is also present. The main header shows 'TDTU Game' with a 'test' subtitle and a trophy icon. Below this, a 'Leaderboard' tab is active, showing a table of participants. To the right, a 'CONTEST INFO' panel provides details about the contest.

RANK	PARTICIPANT	SOLVED	PENALTY	A	B
1	Loi123	2	35m	+1.7m	8m
2	loi1	2	37m	15m	5m
3	SuperDog123	1	6m	6m	---
4	mongo	1	1h 37m	+1.5h 17m	---

CONTEST INFO	
Style	ICPC
Duration	1h 34m
Problems	2
Max size	5
Visibility	Public
Rated	Yes
Host	khangho

Hình 4.10: Giao diện trang cuộc thi và bảng xếp hạng

4.3.3 Trang quản trị

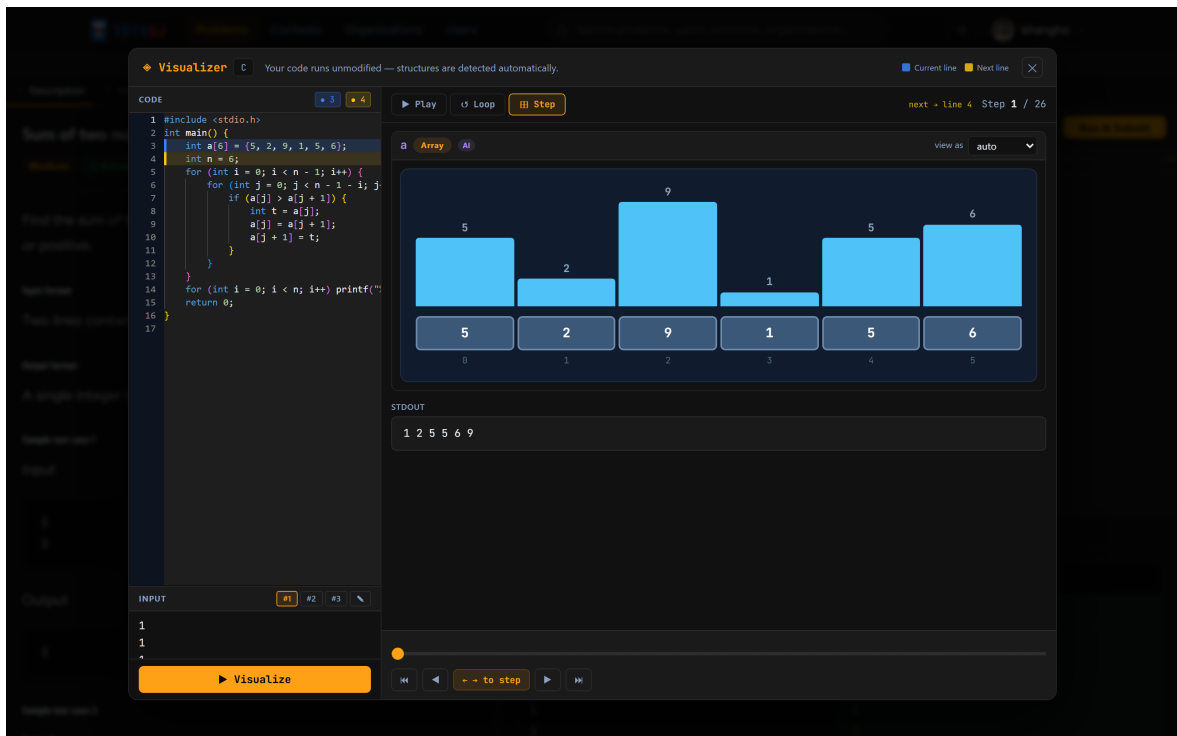
Trang quản trị dành cho quản trị viên và giảng viên, tập hợp các chức năng quản lý nội dung của hệ thống. Như Hình 4.11 minh họa, giao diện gồm thanh điều hướng bên trái liệt kê các phân hệ quản lý như bài toán, nhân bài toán, cuộc thi, tổ chức và người dùng. Khu vực nội dung bên phải hiển thị danh sách bài toán dưới dạng bảng kèm các thao tác chỉnh sửa, xóa và thêm mới, giúp người quản trị duy trì và cập nhật kho bài toán một cách thuận tiện.

#	PROBLEM	DIFFICULTY	TAGS	POINTS	TIME LIMIT	ACTIONS
3	Sum of two numbers	Medium	Math	15	2ms	[Edit] [Delete]
4	Combination Sum	Medium	Backtracking	20	2ms	[Edit] [Delete]
9	Sum of array	Easy	Array	5	2ms	[Edit] [Delete]
10	Graph Cycle	Hard	Graph	25	2ms	[Edit] [Delete]
11	Matrix	Medium	Matrix	15	2ms	[Edit] [Delete]
12	Tree problem	Hard	Binary Tree	21	2ms	[Edit] [Delete]
13	Valid Parentheses	Medium	Stack	17	2ms	[Edit] [Delete]
14	Moving Average of a Sliding Window	Medium	Queue	18	2ms	[Edit] [Delete]
15	Unique Paths in a Grid with Obstacles	Hard	Matrix, Backtracking	30	2ms	[Edit] [Delete]
16	BUYING	Easy	Math	5	2ms	[Edit] [Delete]

Hình 4.11: Giao diện trang quản trị bài toán

4.3.4 Trang trực quan hóa cấu trúc dữ liệu

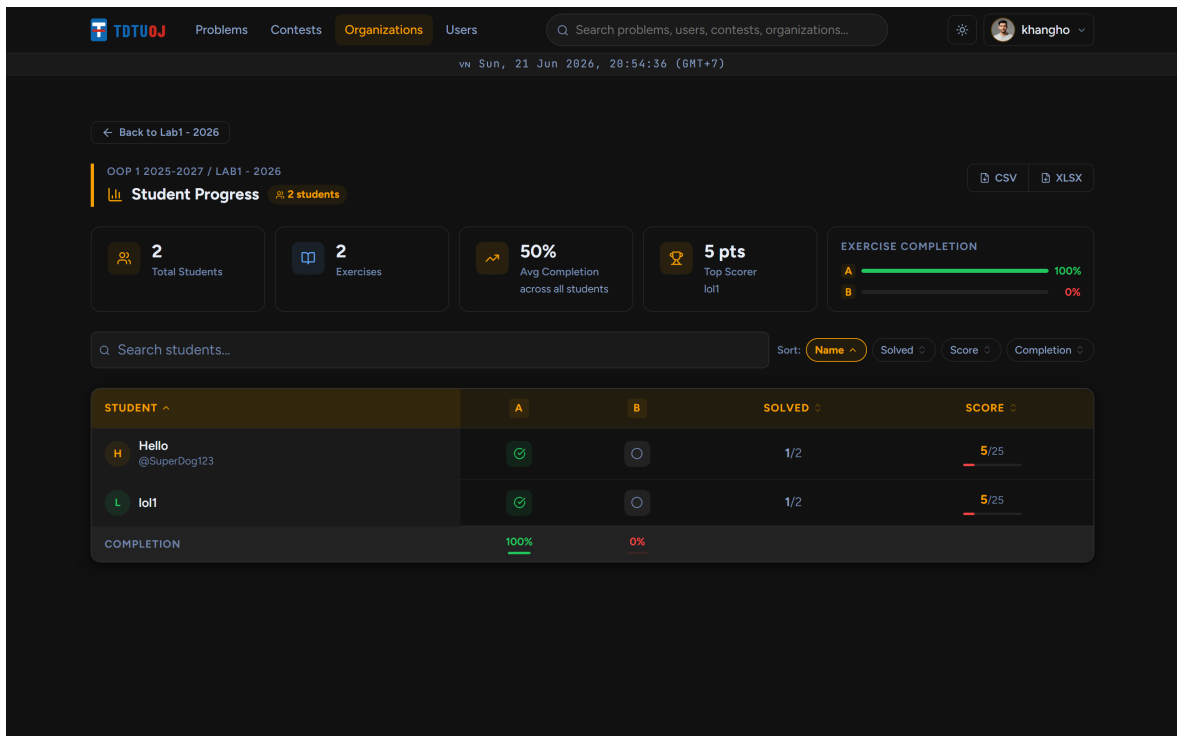
Trình minh họa cấu trúc dữ liệu là một cửa sổ phụ mở ra ngay trên màn hình làm bài. Như Hình 4.12 mô tả, cửa sổ chia thành hai phần: bên trái là mã nguồn của người dùng với dòng đang thực thi được tô sáng, bên phải là khung minh họa cấu trúc dữ liệu được phát hiện tự động từ dữ liệu trong quá trình chạy. Trong hình, một mảng đang được sắp xếp được biểu diễn dưới dạng biểu đồ cột kèm giá trị từng phần tử. Thanh điều khiển phía dưới cho phép phát lại từng bước thực thi như một đoạn hoạt ảnh, đồng thời khu vực kết quả hiển thị dữ liệu xuất ra của chương trình, hỗ trợ người học quan sát trực quan diễn biến của thuật toán.



Hình 4.12: Giao diện trình minh họa cấu trúc dữ liệu

4.3.5 Trang theo dõi tiến độ bài thực hành

Trang theo dõi tiến độ hỗ trợ giảng viên giám sát kết quả làm bài của sinh viên trong một bài thực hành. Như Hình 4.13 thể hiện, đầu trang tổng hợp các chỉ số chung như số sinh viên, số bài toán, tỷ lệ hoàn thành trung bình và sinh viên đạt điểm cao nhất. Bảng bên dưới hiển thị tiến độ chi tiết của từng sinh viên theo từng bài toán, đánh dấu trạng thái đã giải kèm điểm số đạt được. Giao diện còn cho phép xuất dữ liệu tiến độ ra tệp định dạng Comma-Separated Values (CSV) hoặc Excel để phục vụ việc đánh giá và lưu trữ.



Hình 4.13: Giao diện trang theo dõi tiến độ bài thực hành

4.4 Thiết kế hiệu năng và bảo mật

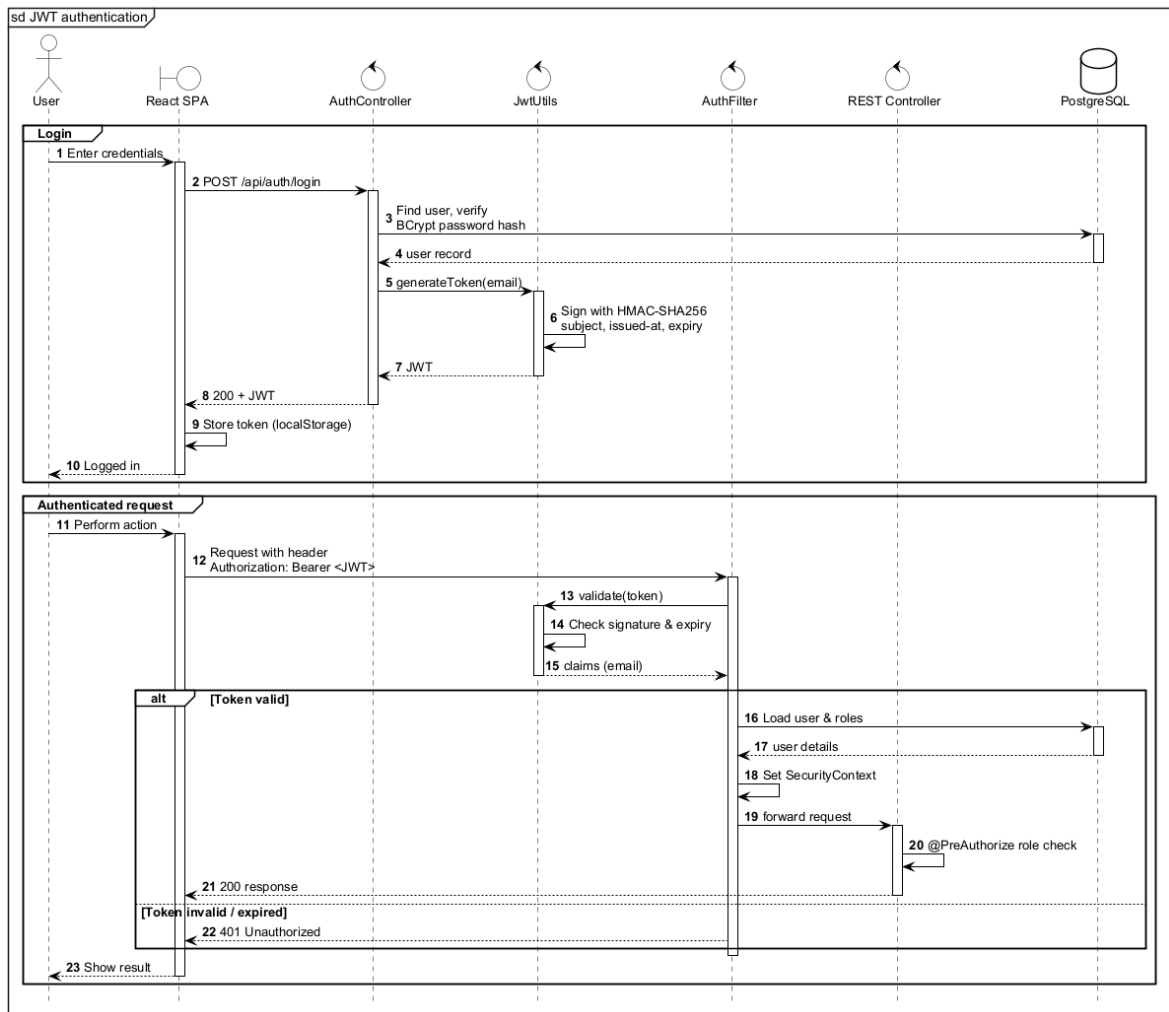
Bên cạnh các thiết kế về kiến trúc, dữ liệu và giao diện, hệ thống TDTUOJ còn được thiết kế nhằm đáp ứng hai yêu cầu phi chức năng trọng yếu đã nêu ở Chương 3 là bảo mật và hiệu năng khi phục vụ nhiều người dùng đồng thời. Mục này trình bày hai cơ chế cốt lõi tương ứng: xác thực phi trạng thái bằng JWT và giới hạn tần suất truy cập (rate limiting) dựa trên Redis. Cả hai cơ chế được hiện thực dưới dạng bộ lọc (filter) đặt trước tầng điều khiển, nhờ đó áp dụng nhất quán cho toàn bộ các yêu cầu REST mà không xen lẫn vào mã nghiệp vụ.

4.4.1 Xác thực và phân quyền bằng JWT

Hệ thống áp dụng mô hình xác thực phi trạng thái (stateless): máy chủ không lưu phiên đăng nhập mà thay vào đó cấp cho người dùng một mã thông báo JWT sau khi đăng nhập thành công. Mã thông báo được sinh bởi lớp *JwtUtils* với thuật toán ký HMAC-SHA256, mang định danh người dùng (địa chỉ thư điện tử) ở trường *subject*, thời điểm phát hành và thời hạn hiệu lực. Sau khi nhận được, phía giao diện lưu mã

thông báo và đính kèm vào tiêu đề *Authorization* dưới dạng lược đồ *Bearer* trong mọi yêu cầu kế tiếp.

Toàn bộ luồng xác thực, từ lúc đăng nhập đến khi một yêu cầu mang mã thông báo được kiểm tra và chuyển tiếp, được mô tả trong sơ đồ tuần tự ở Hình 4.14.



Hình 4.14: Sơ đồ tuần tự luồng xác thực và phân quyền bằng JWT

Với mỗi yêu cầu đến, bộ lọc *AuthFilter* (kế thừa *OncePerRequestFilter*) trích mã thông báo từ tiêu đề, kiểm tra chữ ký và thời hạn, nạp thông tin người dùng từ cơ sở dữ liệu rồi thiết lập ngữ cảnh bảo mật của Spring Security. Nếu mã thông báo không hợp lệ hoặc đã hết hạn, yêu cầu bị từ chối với mã trạng thái HTTP 401. Việc phân quyền theo vai trò (ADMIN, CREATOR, PARTICIPANT) được kiểm soát ở mức phương thức thông qua chú thích *@PreAuthorize*. Nhờ không phụ thuộc phiên

phía máy chủ, thiết kế này thuận lợi cho việc mở rộng theo chiều ngang khi triển khai nhiều thực thể máy chủ.

4.4.2 Giới hạn tần suất truy cập với Redis

Nhằm chống lạm dụng và bảo vệ các tài nguyên tốn kém như chấm bài hay gọi LLM, hệ thống áp dụng cơ chế giới hạn tần suất truy cập theo thuật toán thùng token (token bucket) của thư viện Bucket4j. Trạng thái của mỗi thùng được lưu tập trung trên Redis (qua *LettuceBasedProxyManager*), nhờ đó hạn mức được áp dụng nhất quán ngay cả khi hệ thống chạy trên nhiều thực thể máy chủ.

Cơ chế được hiện thực bởi bộ lọc *RateLimitFilter*, đặt ngay sau *AuthFilter* để biết được danh tính người gọi. Khóa định danh do *RateLimitKeyResolver* sinh ra: với người dùng đã xác thực dùng khóa theo mã định danh tài khoản (tiền tố *rl:user:*), còn với người dùng ẩn danh dùng khóa theo địa chỉ IP (tiền tố *rl:ip:*). Mỗi yêu cầu được phân vào một bậc giới hạn (tier) bởi *RateLimitTierResolver* theo thứ tự ưu tiên: chú thích *@RateLimit* trên phương thức điều khiển, kế đến là quy tắc khớp đường dẫn và phương thức HTTP, cuối cùng là bậc mặc định. Mỗi bậc quy định hạn mức riêng cho người dùng ẩn danh và người dùng đã xác thực trong một cửa sổ thời gian; cấu hình các bậc được trình bày trong Bảng 4.7.

Bảng 4.7: Cấu hình các bậc giới hạn tần suất truy cập

Bậc	Phạm vi áp dụng	Ấn danh	Đã xác thực	Cửa sổ (giây)
default	Các yêu cầu còn lại	60	120	60
read-heavy	GET danh sách bài toán, cuộc thi, người dùng, tổ chức	120	240	60
auth	Đăng nhập, đăng ký, đăng nhập Google	5	10	60
submission	Nộp bài (<i>POST</i> <i>/api/submissions/**</i>)	3	10	60
visualizer	Trực quan hóa (<i>POST</i> <i>/api/visualize</i>)	5	20	60
ai	Gợi ý và trích xuất bằng LLM	0	10	60
admin	Các yêu cầu quản trị (<i>/api/admin/**</i>)	0	300	60

Khi một yêu cầu được chấp nhận, máy chủ gắn thêm các tiêu đề thông tin như *X-RateLimit-Limit* và *X-RateLimit-Remaining* để phía giao diện theo dõi hạn mức còn lại. Khi vượt hạn mức, máy chủ trả về mã trạng thái HTTP 429 kèm tiêu đề *Retry-After* cho biết thời gian cần chờ. Giá trị hạn mức bằng 0 ở bậc *ai* và *admin* có nghĩa là chặn hoàn toàn người dùng ấn danh, chỉ cho phép tài khoản đã xác thực truy cập. Để ưu tiên tính sẵn sàng, bộ lọc được thiết kế theo nguyên tắc *fail-open*: nếu Redis gặp sự cố, yêu cầu vẫn được cho đi qua thay vì bị chặn. Ngoài ra, các chỉ số về số yêu cầu được chấp nhận, bị từ chối và lỗi Redis được thu thập qua *RateLimitMetrics* để phục vụ giám sát hệ thống.

CHƯƠNG 5. CÀI ĐẶT VÀ KIỂM THỬ

Chương này trình bày quá trình cài đặt hệ thống TDTUOJ và công tác kiểm thử nhằm bảo đảm chất lượng. Phần đầu mô tả môi trường, công cụ và cách tổ chức mã nguồn theo tầng, tập trung vào luồng nghiệp vụ tiêu biểu là nộp bài và chấm bài. Phần sau trình bày hai mức kiểm thử: kiểm thử đơn vị cho tầng nghiệp vụ và kiểm thử tải kết hợp giám sát hệ thống.

5.1 Môi trường và công cụ cài đặt

Hệ thống được phát triển và vận hành thử nghiệm trên một máy tính cá nhân, với phía máy chủ (backend) và phía giao diện (frontend) chạy độc lập và giao tiếp qua các giao diện lập trình ứng dụng (API) dạng REST. Phía máy chủ là một ứng dụng Spring Boot viết bằng Java 21, biên dịch và chạy bằng công cụ Maven thông qua tập tin bao bọc mvnw. Phía giao diện là một ứng dụng React, được dựng và chạy bằng Vite qua trình quản lý gói npm. Các dịch vụ phụ trợ được đóng gói và chạy trong Docker để thuận tiện cài đặt và tái lập. Bảng 5.1 liệt kê các thành phần chính cùng công cụ và cổng dịch vụ tương ứng.

Thành phần	Công nghệ	Cổng
Phía máy chủ	Spring Boot (Java 21), Maven	8090
Phía giao diện	React, Vite	5173
Cơ sở dữ liệu	PostgreSQL	5431
Bộ nhớ đệm, hàng đợi	Redis	6379
Dịch vụ chấm bài	Judge0 (Docker)	2358

Bảng 5.1: Các thành phần trong môi trường cài đặt

5.2 Cài đặt hệ thống

5.2.1 Tổ chức mã nguồn theo tầng

Mã nguồn phía máy chủ được tổ chức theo mô hình phân tầng quen thuộc của Spring Boot, mỗi phân hệ nghiệp vụ là một gói (package) riêng chứa đầy đủ các tầng. Tầng điều khiển (Controller) tiếp nhận yêu cầu REST và trả về phản hồi; tầng dịch vụ (Service) chứa logic nghiệp vụ, được tách thành giao diện và lớp cài đặt *Impl; tầng kho dữ liệu (Repository) truy xuất cơ sở dữ liệu thông qua Spring Data JPA. Dữ liệu được biểu diễn bằng hai loại đối tượng: thực thể (Entity) ánh xạ trực tiếp với bảng trong cơ sở dữ liệu, và đối tượng truyền dữ liệu (Data Transfer Object – DTO) dùng để trao đổi với phía giao diện; việc chuyển đổi giữa hai loại được thực hiện bằng thư viện ModelMapper. Cách phân chia này tương ứng với khuôn mẫu kiến trúc MVC và giúp tách bạch rõ trách nhiệm giữa các tầng. Bảng 5.2 tóm tắt vai trò của từng tầng kèm lớp tiêu biểu.

Tầng	Trách nhiệm	Lớp ví dụ
Controller	Tiếp nhận yêu cầu REST, kiểm tra đầu vào, trả phản hồi	SubmissionController
Service	Xử lý logic nghiệp vụ (giao diện và lớp cài đặt)	SubmissionServiceImpl
Repository	Truy xuất cơ sở dữ liệu qua Spring Data JPA	SubmissionRepository
Entity	Ánh xạ đối tượng với bảng dữ liệu	Submission
DTO	Truyền dữ liệu giữa các tầng và với phía giao diện	SubmissionDTO

Bảng 5.2: Các tầng trong mã nguồn phía máy chủ

Để bảo đảm tính nhất quán, mọi phản hồi từ phía máy chủ đều được bọc trong một lớp vỏ chung `Response<T>` gồm mã trạng thái, thông điệp và dữ liệu; các lỗi phát sinh được xử lý tập trung tại lớp `GlobalExceptionHandler`. Việc xác thực và phân

quyền được thực hiện bởi bộ lọc AuthFilter kết hợp JWT, kiểm tra mã thông báo trước khi yêu cầu đến tầng điều khiển. Các phân hệ quản lý dữ liệu khác như bài toán, cuộc thi, tổ chức và bài thực hành đều tuân theo cùng khuôn mẫu nói trên nên không trình bày lặp lại.

5.2.2 Luồng nộp bài và chấm bài bất đồng bộ

Nộp bài và chấm bài là luồng nghiệp vụ quan trọng nhất, đồng thời minh họa rõ cách các tầng phối hợp. Khi người dùng nộp một lời giải, SubmissionController chuyển yêu cầu cho SubmissionServiceImpl; lớp này kiểm tra điều kiện (khoảng chờ giữa hai lần nộp, quyền tham gia cuộc thi), lưu bản ghi bài nộp ở trạng thái chờ rồi đẩy một công việc vào hàng đợi trên Redis thông qua SubmissionQueueService. Việc chấm bài không diễn ra trực tiếp trong luồng xử lý yêu cầu mà được thực hiện bất đồng bộ: một tiến trình xử lý (worker) là SubmissionWorker định kỳ lấy công việc khỏi hàng đợi và gọi SubmissionJudgeService, lớp này gửi mã nguồn đến Judge0 để biên dịch và chạy thử trên từng trường hợp kiểm thử, sau đó lưu kết quả chấm. Toàn bộ trình tự tương tác này đã được trình bày ở Hình 3.3.

Thiết kế tách khâu chấm bài ra khỏi luồng yêu cầu giúp phía giao diện nhận phản hồi tức thì và không bị nghẽn khi nhiều bài nộp đến cùng lúc, đáp ứng yêu cầu phi chức năng về hiệu năng đã nêu ở Chương 3. Các tính năng nổi bật khác như giới hạn tần suất truy cập, trình minh họa cấu trúc dữ liệu và gợi ý học tập bằng mô hình ngôn ngữ lớn (LLM) cũng được cài đặt theo cùng nguyên tắc phân tầng và đã được mô tả ở các chương trước.

5.2.3 Tính điểm đánh giá năng lực sau cuộc thi

Đối với các cuộc thi có tính điểm (rated), hệ thống tự động cập nhật điểm đánh giá năng lực (rating) của thí sinh sau khi quá trình chấm bài kết thúc. Công việc này do lớp ContestRatingService đảm nhận, được kích hoạt bất đồng bộ bởi bộ lập lịch ContestRatingScheduler khi mọi bài nộp của cuộc thi đã được chấm xong. Mỗi thí sinh mới tham gia được gán một điểm khởi tạo mặc định là 1500.

Trước hết, các thí sinh được xếp hạng theo thể thức ICPC: ưu tiên số bài giải được nhiều hơn, nếu bằng nhau thì xét tổng thời gian phạt nhỏ hơn; thứ hạng thu được ký hiệu là r_i (đánh số từ 1). Gọi N là số thí sinh và R_i là điểm đánh giá hiện thời của thí sinh thứ i trước cuộc thi. Hệ thống áp dụng một công thức dựa trên ý tưởng của hệ thống Elo, trong đó mỗi thí sinh có một thứ hạng kỳ vọng (seed) s_i xác định bởi số thí sinh có điểm cao hơn mình, như trong Công thức (5.1).

$$s_i = 1 + |\{j : 1 \leq j \leq N, j \neq i, R_j > R_i\}| \quad (5.1)$$

Mức thay đổi điểm Δ_i của thí sinh được tính từ chênh lệch giữa thứ hạng kỳ vọng s_i và thứ hạng thực tế r_i theo Công thức (5.2), với $K = 100$ là hệ số điều chỉnh. Nếu cuộc thi chỉ có một thí sinh ($N \leq 1$) thì quy ước $\Delta_i = 0$.

$$\Delta_i = \text{round}\left(K \cdot \frac{s_i - r_i}{N}\right) \quad (5.2)$$

Để tránh biến động quá lớn sau một cuộc thi, mức thay đổi được giới hạn trong khoảng $[-\Delta_{\max}, \Delta_{\max}]$ với $\Delta_{\max} = 150$, sau đó điểm mới R'_i được cập nhật như Công thức (5.3); điểm đánh giá không bao giờ nhỏ hơn 1.

$$R'_i = \max(1, R_i + \text{clip}(\Delta_i, -\Delta_{\max}, \Delta_{\max})), \quad \text{clip}(x, a, b) = \max(a, \min(b, x)) \quad (5.3)$$

Kết quả được lưu vào bảng lịch sử điểm đánh giá (RatingHistory) gồm điểm trước và sau cuộc thi, mức thay đổi và thứ hạng; đồng thời điểm hiện thời và điểm cao nhất của thí sinh trong UserStatistics cũng được cập nhật. Do điểm trước một cuộc thi chính là điểm sau cuộc thi liền trước đó của thí sinh, khi một cuộc thi cũ được tính lại, hệ thống sẽ tính lại toàn bộ chuỗi lịch sử phía sau nhằm bảo đảm tính nhất quán.

5.3 Thiết kế API

Hệ thống cung cấp chức năng cho phía giao diện thông qua tập hợp giao diện lập trình ứng dụng (API) dạng REST, trao đổi dữ liệu bằng định dạng JSON trên giao thức HTTP. Mọi phản hồi đều tuân theo lớp vỏ chung `Response<T>` gồm mã trạng thái, thông điệp và dữ liệu; các yêu cầu cần định danh người dùng được bảo vệ bằng JWT và được phân quyền theo vai trò. Các endpoint được tổ chức theo từng phân hệ nghiệp vụ, mỗi phân hệ tương ứng với một nhóm API có đường dẫn gốc riêng.

5.3.1 Tổng quan các nhóm API

Bảng 5.3 liệt kê toàn bộ các nhóm API của hệ thống cùng đường dẫn gốc và chức năng chính. Mỗi nhóm gồm nhiều endpoint tương ứng với các thao tác thêm, sửa, xóa và truy vấn trên một loại tài nguyên.

Bảng 5.3: Tổng quan các nhóm API của hệ thống TDTUOJ

Nhóm API	Đường dẫn gốc	Chức năng chính
Xác thực	/api/auth	Đăng ký, đăng nhập, đăng nhập bằng Google
Người dùng	/api/users	Hồ sơ, hoạt động, thống kê, lịch sử nộp bài và lịch sử xếp hạng
Quản trị người dùng	/api/admin/users	Xem và cập nhật tài khoản (dành cho quản trị viên)
Vai trò	/api/roles	Quản lý vai trò người dùng
Bài toán	/api/problems	Quản lý bài toán, gán nhãn và lọc bài đủ điều kiện đưa vào cuộc thi
Nhãn bài toán	/api/problem-tags	Quản lý nhãn phân loại bài toán
Bình luận	/api/problems/{id}/comments	Bình luận theo bài toán và bình chọn

(tiếp theo Bảng 5.3)

Nhóm API	Đường dẫn gốc	Chức năng chính
Yêu thích	/api/favorites	Đánh dấu và bỏ đánh dấu bài toán yêu thích
Trường hợp kiểm thử	/api/testcases	Quản lý bộ dữ liệu kiểm thử của bài toán
Bài nộp	/api/submissions	Nộp bài, truy vấn trạng thái và lịch sử chấm bài
Cuộc thi	/api/contests	Quản lý cuộc thi, đăng ký, bảng xếp hạng và giám sát
Tổ chức	/api/organizations	Quản lý tổ chức và thành viên
Bài thực hành	/api/organizations/{orgId}/labs	Quản lý bài thực hành, theo dõi tiến độ và xuất kết quả
Trực quan hóa	/api/visualize	Trực quan hóa quá trình thực thi mã nguồn
Gợi ý học tập	/api/hints	Sinh gợi ý cho bài toán bằng LLM
Hỗ trợ soạn đề	/api/problem-ai	Trích xuất đề và sinh trường hợp kiểm thử bằng LLM
Bảng điều khiển	/api/admin/dashboard	Thống kê tổng quan cho quản trị viên
Tập tin	/api/files	Truy xuất tập tin lưu trên AWS S3

5.3.2 Công cụ tài liệu hóa và kiểm thử API

Trong quá trình phát triển, các endpoint API được tài liệu hóa và kiểm thử bằng hai công cụ. Thứ nhất là Swagger UI được sinh tự động bởi thư viện SpringDoc OpenAPI: giao diện này liệt kê toàn bộ endpoint kèm mô tả tham số, lược đồ yêu cầu và phản hồi, đồng thời cho phép gửi thử yêu cầu trực tiếp từ trình duyệt; đặc tả OpenAPI tương ứng được cung cấp tại đường dẫn /v3/api-docs. Thứ hai là Postman,

dùng để kiểm thử thủ công và lưu lại các bộ yêu cầu (collection) phục vụ kiểm thử hồi quy, đặc biệt thuận tiện cho việc đính kèm mã thông báo JWT và kiểm tra các luồng yêu cầu cần xác thực.

5.4 Mô tả các chức năng tiêu biểu

Bên cạnh các nhóm API quản lý dữ liệu thông thường, hệ thống có ba chức năng nổi bật mang lại giá trị riêng cho một hệ thống chấm bài trực tuyến phục vụ giảng dạy. Mục này mô tả chi tiết ba chức năng đó: nộp bài, trực quan hóa cấu trúc dữ liệu và hỗ trợ bằng mô hình ngôn ngữ lớn.

5.4.1 Chức năng nộp bài

Nhóm API nộp bài do lớp SubmissionController đảm nhận, cho phép người dùng gửi lời giải và theo dõi kết quả chấm. Do quá trình chấm bài diễn ra bất đồng bộ như đã trình bày ở mục trước, thiết kế được tách thành hai bước: yêu cầu tạo bài nộp trả về ngay một bản ghi ở trạng thái chờ kèm mã định danh, sau đó phía giao diện truy vấn định kỳ endpoint trạng thái để lấy kết quả khi việc chấm hoàn tất. Bảng 5.4 liệt kê các endpoint của nhóm chức năng này.

Bảng 5.4: Các endpoint API của chức năng nộp bài

Phương thức	Đường dẫn	Mô tả
POST	/api/submissions	Tạo bài nộp mới từ mã nguồn, ngôn ngữ và bài toán; trả bản ghi ở trạng thái chờ
GET	/api/submissions/{id}/status	Truy vấn trạng thái và kết quả chấm của một bài nộp
GET	/api/submissions/me	Lấy lịch sử bài nộp của người dùng hiện tại (phân trang, lọc theo bài toán)
GET	/api/submissions/count	Đếm tổng số bài nộp trong hệ thống

Để chống lạm dụng, nhóm endpoint nộp bài chịu ràng buộc về khoảng chờ tối

thiếu giữa hai lần nộp và bị giới hạn tần suất ở bậc submission như đã trình bày ở Chương 4.

5.4.2 Chức năng trực quan hóa cấu trúc dữ liệu

Chức năng trực quan hóa cho phép người dùng quan sát trạng thái các biến và cấu trúc dữ liệu của chương trình qua từng bước thực thi, phục vụ việc học thuật toán và gỡ lỗi. Toàn bộ chức năng được cung cấp qua một endpoint duy nhất POST /api/visualize, nhận mã nguồn, ngôn ngữ lập trình và dữ liệu đầu vào (nếu có), trả về danh sách các khung trạng thái (frame) để phía giao diện phát lại như một đoạn hoạt ảnh. Hệ thống hỗ trợ sáu ngôn ngữ: Python, JavaScript, Java, C#, C và C++.

Hạt nhân của chức năng là các bộ ghi vết (tracer), mỗi ngôn ngữ một bộ, do lớp VisualizerServiceImpl điều phối. Bộ ghi vết biến đổi mã nguồn của người dùng sao cho khi chạy, chương trình tự phát ra chuỗi khung trạng thái trên luồng lỗi chuẩn (stderr), được bao giữa hai dấu hiệu nhận biết để tách khỏi thông báo lỗi thật của chương trình; luồng ra chuẩn (stdout) được giữ nguyên cho kết quả của người dùng. Với Python, bộ ghi vết tận dụng cơ chế theo dõi thực thi có sẵn của trình thông dịch nên mã nguồn gốc không bị biến đổi; với các ngôn ngữ biên dịch, mã nguồn được viết lại để chèn lời gọi ghi vết tại từng dòng lệnh. Mọi bộ ghi vết đều phát ra cùng một lược đồ khung thống nhất: mỗi khung ghi số thứ tự bước, dòng lệnh tương ứng trong mã nguồn gốc, ngăn xếp lời gọi hàm kèm các biến cục bộ, và một vùng heap mô tả các giá trị phức hợp (danh sách, đối tượng, từ điển...) với định danh ổn định qua các khung – nhờ đó phía giao diện có thể tạo hoạt ảnh một nút dữ liệu di chuyển thay vì vẽ lại từ đầu. Để bảo đảm an toàn, vết thực thi bị giới hạn ở 5000 bước và kích thước từng vùng dữ liệu đều có ngưỡng cắt; mã đã biến đổi được thực thi trong môi trường cô lập của Judge0 với giới hạn thời gian và bộ nhớ riêng.

Song song với việc thực thi, hệ thống gửi mã nguồn gốc đến mô hình ngôn ngữ lớn Gemini để phân loại vai trò ngữ nghĩa của từng biến: một mảng hai chiều được duyệt theo đỉnh – đỉnh là ma trận kề của đồ thị, trong khi một mảng được điền theo quan hệ truy hồi là bảng quy hoạch động. Kết quả phân loại giúp trình minh

họa phía giao diện chọn cách hiển thị phù hợp (đồ thị, cây, ngăn xếp, hàng đợi...) thay vì chỉ dựa vào hình dạng dữ liệu. Bước phân loại này nằm ngoài đường găng của luồng xử lý: nó chạy đồng thời với việc chấm trên Judge0, bị giới hạn ngân sách thời gian ba giây và nếu thất bại thì trả về kết quả rỗng để phép suy luận theo kinh nghiệm (heuristic) ở phía giao diện tự đảm nhận. Kết quả phân loại được lưu đệm trên Redis theo mã băm của mã nguồn trong 24 giờ, nhờ đó các lần chạy lại khi gõ lỗi không tốn thêm chi phí gọi mô hình.

5.4.3 Chức năng hỗ trợ bằng mô hình ngôn ngữ lớn

Hệ thống tích hợp mô hình ngôn ngữ lớn (LLM) Gemini của Google nhằm cung cấp hai chức năng: gợi ý học tập cho người dùng và hỗ trợ soạn đề cho quản trị viên cùng giảng viên. Việc gọi mô hình được thực hiện ở phía máy chủ thông qua WebClient của Spring WebFlux, gửi yêu cầu HTTP đến dịch vụ Generative Language của Google (mô hình gemini-2.5-flash) và nhận phản hồi dạng JSON; cơ chế gọi lại (retry) được áp dụng khi dịch vụ quá tải. Bảng 5.5 liệt kê các endpoint của nhóm chức năng này.

Bảng 5.5: Các endpoint API của chức năng hỗ trợ bằng LLM

Phương thức	Đường dẫn	Mô tả	Quyền
POST	/api/hints	Sinh gợi ý cho bài toán hiện tại; chỉ dẫn dắt, không tiết lộ lời giải	Đã đăng nhập
POST	/api/problem-ai/extract-pdf	Trích xuất đề bài từ tập tin PDF tải lên	ADMIN, CREATOR
POST	/api/problem-ai/generate-test-cases	Sinh trường hợp kiểm thử từ nội dung đề bài	ADMIN, CREATOR

Đối với gợi ý học tập, endpoint /api/hints nhận đề bài, mã nguồn hiện tại, thông báo lỗi và lịch sử hội thoại của người dùng. Một chỉ thị hệ thống (system instruction) ràng buộc mô hình chỉ đưa ra các gợi ý tăng dần, từ chối tiết lộ lời giải hoàn chỉnh và

bỏ qua các câu hỏi nằm ngoài phạm vi bài toán. Thiết kế cho phép sử dụng nhiều nhà cung cấp mô hình thông qua việc tra cứu theo tên (Gemini là mặc định, ngoài ra còn hỗ trợ Claude và OpenAI), giúp dễ dàng thay thế hoặc bổ sung mô hình về sau.

Đối với hỗ trợ soạn đề, hai endpoint được giới hạn riêng cho quản trị viên và giảng viên bằng chú thích @PreAuthorize. Endpoint extract-pdf dùng thư viện Apache PDFBox để rút văn bản từ tập tin PDF, rồi nhờ Gemini trích xuất các trường có cấu trúc của bài toán gồm tiêu đề, đề bài, độ khó, giới hạn tài nguyên, trường hợp kiểm thử và nhãn gợi ý; nếu tập tin không chứa trường hợp kiểm thử, hệ thống tự động sinh năm trường hợp. Endpoint generate-test-cases sinh từ 1 đến 50 trường hợp kiểm thử từ nội dung đề bài. Cả hai chức năng đều bị giới hạn tần suất ở bậc ai, chặn hoàn toàn người dùng ẩn danh.

5.5 Kiểm thử đơn vị

Kiểm thử đơn vị (unit testing) được áp dụng cho tầng dịch vụ – nơi tập trung phần lớn logic nghiệp vụ. Bộ kiểm thử sử dụng JUnit 5 kết hợp Mockito để giả lập (mock) các phụ thuộc như kho dữ liệu và dịch vụ bên ngoài, nhờ đó mỗi trường hợp kiểm thử chỉ tập trung kiểm chứng một đơn vị logic một cách độc lập và tất định. Tổng cộng đề tài xây dựng 13 lớp kiểm thử với 101 ca kiểm thử, bao phủ các phân hệ nghiệp vụ trọng yếu.

Độ phủ mã nguồn được đo bằng công cụ JaCoCo. Do phạm vi kiểm thử đơn vị ưu tiên các phân hệ nghiệp vụ cốt lõi (mức ưu tiên P0 và P1), trong khi các thành phần ngoại vi như tầng điều khiển, tiện ích, trình minh họa và bình luận chưa được kiểm thử ở mức đơn vị, nên độ phủ trên toàn dự án ở mức khiêm tốn. Bảng 5.6 trình bày độ phủ dòng lệnh của các phân hệ dịch vụ đã được kiểm thử.

Phân hệ dịch vụ	Độ phủ dòng
Nộp bài (submission)	64,5%
Cuộc thi (contest)	53,7%
Bài toán (problem)	43,0%
Người dùng (user)	40,3%
Tổ chức (organization)	38,8%

Bảng 5.6: Độ phủ dòng lệnh của các phân hệ dịch vụ cốt lõi

Mỗi trường hợp kiểm thử kiểm chứng một tình huống nghiệp vụ cụ thể, bao gồm cả luồng thành công và các luồng ngoại lệ. Bảng 5.7 liệt kê một số trường hợp kiểm thử tiêu biểu cùng kết quả mong đợi.

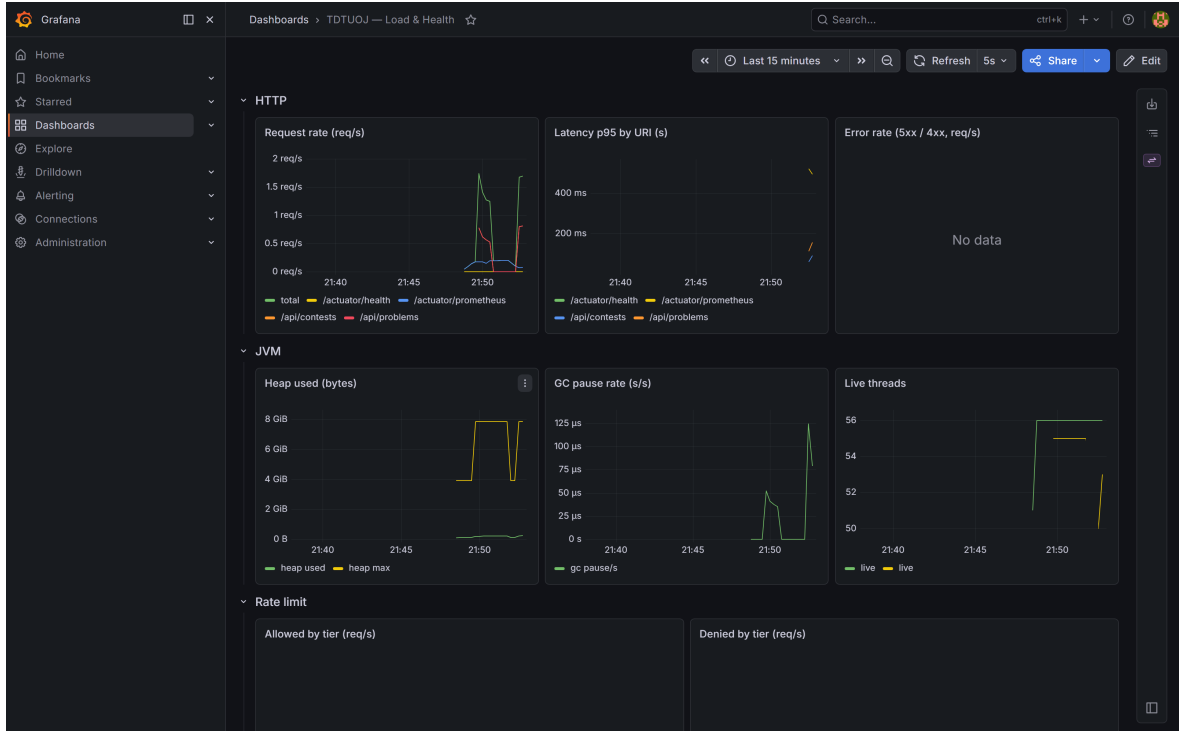
Ca kiểm thử	Tình huống	Kết quả mong đợi
login_WrongPassword	Đăng nhập sai mật khẩu	Báo lỗi 401
register_DuplicateEmail	Đăng ký trùng thư điện tử	Báo lỗi 400
createSubmission_OnCooldown	Nộp bài khi còn trong thời gian chờ	Báo lỗi 429
createSubmission_ContestWithoutRegistration	Nộp bài cuộc thi khi chưa đăng ký	Báo lỗi 403
getSubmissionStatus_NotFound	Tra cứu bài nộp không tồn tại	Báo lỗi không tìm thấy

Bảng 5.7: Một số trường hợp kiểm thử đơn vị tiêu biểu

5.6 Kiểm thử tải và giám sát

Bên cạnh tính đúng đắn, đề tài còn quan tâm đến khả năng chịu tải của hệ thống chấm bài, vốn phải phục vụ nhiều người dùng đồng thời trong các kỳ thi. Để đo lường, hệ thống được trang bị một hạ tầng giám sát và một bộ công cụ sinh tải. Phía máy chủ phơi bày các chỉ số (metrics) thông qua Spring Boot Actuator và thư viện Micrometer; Prometheus thu thập định kỳ các chỉ số này; Grafana trực quan hóa

thành các bảng điều khiển; còn công cụ k6 sinh tải theo kịch bản xác định. Bảng điều khiển tổng quan của hệ thống được trình bày trong Hình 5.1.



Hình 5.1: Bảng điều khiển giám sát tổng quan trên Grafana

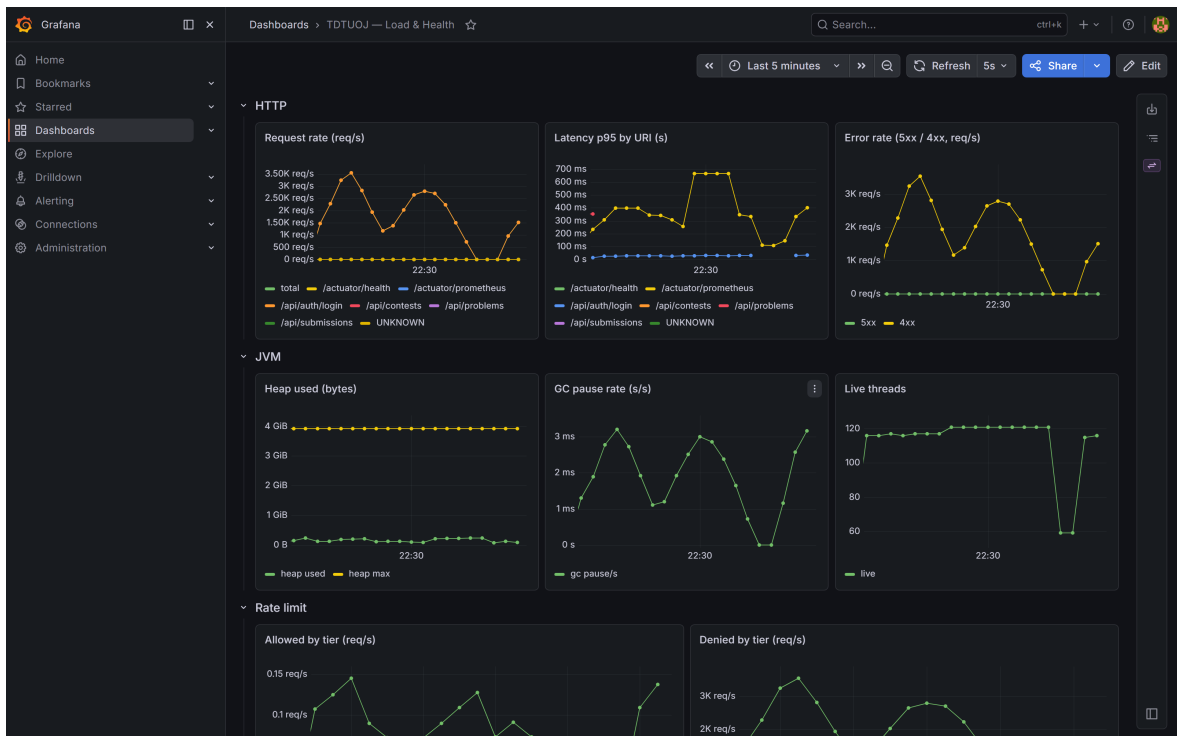
Các chỉ số được theo dõi thuộc ba lớp: lớp giao thức HTTP, lớp máy ảo Java (Java Virtual Machine – Máy ảo Java (JVM)) và lớp nghiệp vụ. Bảng 5.8 mô tả ý nghĩa của các chỉ số chính.

Chỉ số	Ý nghĩa	Lớp
Số yêu cầu mỗi giây	Thông lượng phía máy chủ phục vụ được	HTTP
Độ trễ phân vị 95	Thời gian phản hồi của 95% số yêu cầu	HTTP
Bộ nhớ heap, thời gian dọn rác	Áp lực tài nguyên bộ nhớ	JVM
Độ sâu hàng đợi chấm bài	Số bài nộp đang chờ chấm	Nghiệp vụ
Thông lượng chấm bài theo kết quả	Số bài chấm xong mỗi giây theo kết quả	Nghiệp vụ
Số lượt chấp nhận, từ chối theo tầng	Hiệu quả của cơ chế giới hạn tần suất	Nghiệp vụ

Bảng 5.8: Các chỉ số giám sát chính

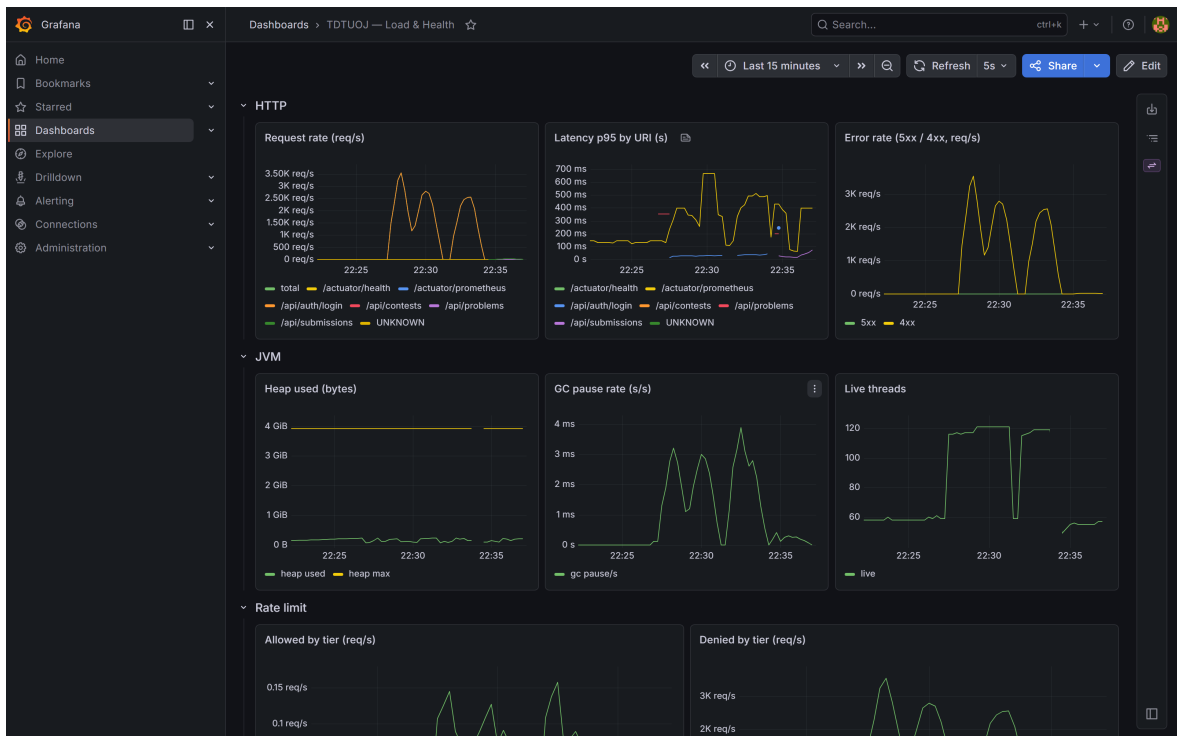
Quá trình kiểm thử tải sử dụng ba kịch bản. Kịch bản thứ nhất mô phỏng việc duyệt bài với số người dùng ảo (virtual user) tăng dần truy cập các endpoint công khai. Kịch bản thứ hai mô phỏng nộp bài liên tục để quan sát hoạt động của hàng đợi và tiến trình chấm. Kịch bản thứ ba dồn dập gửi yêu cầu vượt ngưỡng cho phép nhằm kiểm chứng cơ chế giới hạn tần suất.

Trong kịch bản giới hạn tần suất, kết quả đo cho thấy số lượt yêu cầu được chấp nhận duy trì ở mức trần đã cấu hình, trong khi số lượt bị từ chối tăng vọt khi lưu lượng vượt ngưỡng, như minh họa ở Hình 5.2. Điều này xác nhận cơ chế bảo vệ hoạt động đúng khi hệ thống bị truy cập dồn dập.



Hình 5.2: Số lượt chấp nhận và từ chối theo tầng khi bị dồn tần suất

Trong kịch bản nộp bài, do mỗi người dùng bị ràng buộc bởi khoảng chờ tối thiểu giữa hai lần nộp nhằm chống gửi trùng lặp, tiến trình chấm luôn xử lý kịp nên độ sâu hàng đợi được giữ ở mức thấp; các biểu đồ thông lượng chấm bài và độ trễ chấm phản ánh hoạt động chấm thực tế, đồng thời lưu lượng HTTP và mức tiêu thụ tài nguyên JVM đều ổn định, như trình bày ở Hình 5.3.



Hình 5.3: Giám sát hệ thống trong kịch bản nộp bài

Nhìn chung, kết quả kiểm thử tải cho thấy hệ thống đáp ứng ổn định dưới lưu lượng cao và cơ chế giới hạn tần suất bảo vệ hiệu quả. Cần lưu ý rằng các phép đo được thực hiện trên một máy tính cá nhân, nơi máy chủ, công cụ sinh tải và Judge0 cùng chia sẻ tài nguyên, do đó các con số mang tính so sánh tương đối thay vì phản ánh năng lực tuyệt đối trong môi trường triển khai thực tế.

CHƯƠNG 6. TRIỂN KHAI HỆ THỐNG

Sau khi hoàn thiện và kiểm thử, hệ thống TDTUOJ được triển khai lên môi trường trực tuyến nhằm phục vụ việc trình diễn và sử dụng thực tế. Chương này trình bày hạ tầng và công cụ được sử dụng để triển khai, quy trình đưa hệ thống lên môi trường vận hành kèm cơ chế tích hợp và triển khai liên tục (CI/CD), cùng kết quả đạt được và chi phí vận hành hằng tháng.

6.1 Hạ tầng và công cụ triển khai

Toàn bộ phần phía máy chủ được triển khai theo mô hình đóng gói bằng vùng chứa (container). Mã nguồn phía máy chủ được biên dịch thành một ảnh Docker, sau đó toàn bộ các thành phần phụ thuộc được khai báo và khởi chạy đồng thời trên cùng một máy ảo (Virtual Machine – Máy ảo (VM)) thông qua công cụ Docker Compose. Cách làm này bảo đảm môi trường vận hành nhất quán với môi trường phát triển và có thể tái dựng lại toàn bộ hệ thống chỉ bằng một câu lệnh.

Máy ảo được thuê từ dịch vụ điện toán đám mây Microsoft Azure, gói dành cho sinh viên (Azure for Students), với cấu hình hai nhân xử lý và tám gigabyte bộ nhớ, hệ điều hành Ubuntu. Trên máy ảo này, Docker Compose điều phối tám vùng chứa hoạt động đồng thời: phía máy chủ Spring Boot, cơ sở dữ liệu PostgreSQL và bộ nhớ đệm Redis của ứng dụng, cụm dịch vụ chấm bài Judge0 cùng cơ sở dữ liệu và bộ nhớ đệm riêng của nó, và một máy chủ ủy nhiệm ngược (reverse proxy) Caddy. Caddy đóng vai trò cổng vào duy nhất, tự động xin và gia hạn chứng chỉ số để cung cấp kết nối mã hóa HTTPS thông qua giao thức Transport Layer Security (TLS), đồng thời chuyển tiếp yêu cầu đến phía máy chủ; mọi thành phần còn lại chỉ giao tiếp nội bộ trong mạng riêng của Docker Compose, không phơi bày trực tiếp ra Internet.

Phía giao diện người dùng được triển khai tách biệt trên nền tảng Vercel, vốn chuyên dụng cho các ứng dụng web tĩnh và tự động biên dịch, phân phối mã nguồn React qua mạng phân phối nội dung toàn cầu. Hệ thống sử dụng tên miền riêng

tdtuoj.me, trong đó tên miền gốc trở tới giao diện người dùng trên Vercel còn tên miền phụ api.tdtuoj.me trở tới phía máy chủ trên máy ảo, việc phân giải được cấu hình thông qua bản ghi Domain Name System – Hệ thống phân giải tên miền (DNS). Cả hai đầu đều được bảo vệ bằng HTTPS, qua đó cho phép cơ chế đăng nhập bằng tài khoản Google hoạt động đúng yêu cầu bảo mật. Bảng 6.1 tổng hợp các thành phần hạ tầng chính.

Thành phần	Công cụ / dịch vụ	Vai trò
Phía máy chủ + chấm bài	Máy ảo Azure (Docker Compose)	Chạy backend, Judge0, cơ sở dữ liệu, bộ nhớ đệm
Cổng vào & HTTPS	Caddy	Ủy nhiệm ngược, tự cấp chứng chỉ TLS
Phía giao diện	Vercel	Phân phối ứng dụng React
Tên miền & phân giải	Namecheap (DNS)	Tên miền tdtuoj.me
Tích hợp liên tục	GitHub Actions	Tự động biên dịch và triển khai

Bảng 6.1: Các thành phần hạ tầng triển khai chính

Trong quá trình triển khai, một trở ngại kỹ thuật đáng chú ý phát sinh ở dịch vụ chấm bài Judge0. Thành phần này dựa trên cơ chế cô lập tiến trình của nhân Linux thông qua nhóm điều khiển tài nguyên (control group) phiên bản một, trong khi nhân Linux mới trên máy ảo đã loại bỏ thành phần quản lý bộ nhớ của phiên bản này. Hệ quả là mọi bài nộp đều báo lỗi nội bộ. Vấn đề được khắc phục bằng cách cài đặt và khởi động một phiên bản nhân Linux cũ hơn còn hỗ trợ đầy đủ cơ chế nói trên, nhờ đó dịch vụ chấm bài hoạt động ổn định.

6.2 Quy trình triển khai và tích hợp liên tục

Quy trình đưa hệ thống lên môi trường vận hành gồm các bước chính: khởi tạo máy ảo và mở các cổng cần thiết, cài đặt Docker, sao chép mã nguồn về máy ảo, khai báo các thông tin bí mật (mật khẩu cơ sở dữ liệu, khóa bí mật, khóa truy cập dịch vụ

ngoài) trong tập tin cấu hình không được đưa vào kho mã nguồn, rồi khởi chạy toàn bộ hệ thống bằng Docker Compose. Việc cấu hình bản ghi DNS và để Caddy tự cấp chứng chỉ giúp hoàn tất kết nối HTTPS.

Để giảm thao tác thủ công mỗi khi cập nhật mã nguồn, đề tài thiết lập một quy trình CI/CD bằng GitHub Actions. Mỗi khi mã nguồn được đẩy lên nhánh chính, hệ thống tự động thực hiện hai giai đoạn: giai đoạn kiểm tra biên dịch và chạy các trường hợp kiểm thử đơn vị nhằm bảo đảm mã nguồn hợp lệ; sau đó, nếu thành công, giai đoạn triển khai sẽ kết nối tới máy ảo qua giao thức SSH, cập nhật mã nguồn mới và dựng lại các vùng chứa. Phía giao diện trên Vercel được tự động biên dịch và phân phối lại ngay khi có thay đổi mà không cần cấu hình thêm. Nhờ vậy, một thay đổi trong mã nguồn có thể được đưa lên môi trường vận hành một cách tự động và nhất quán.

6.3 Kết quả triển khai và chi phí vận hành

Hệ thống đã được triển khai thành công và truy cập được công khai tại địa chỉ <https://tdtuoj.me>, với phía máy chủ phục vụ tại <https://api.tdtuoj.me>. Các chức năng chính đều hoạt động đúng trong môi trường vận hành thực tế: người dùng đăng ký và đăng nhập (kể cả đăng nhập bằng tài khoản Google), duyệt và nộp bài, dịch vụ Judge0 chấm bài và trả kết quả chính xác, kết nối được mã hóa HTTPS đầu cuối. Quy trình CI/CD cũng vận hành đúng, tự động cập nhật hệ thống sau mỗi lần thay đổi mã nguồn.

Về chi phí, hầu hết các dịch vụ đều thuộc các gói miễn phí dành cho sinh viên hoặc nhà phát triển. Khoản chi phí đáng kể duy nhất là máy ảo trên Azure. Với cấu hình đã chọn, đơn giá xấp xỉ 62 đô-la Mỹ mỗi tháng nếu để chạy liên tục cả ngày lẫn đêm. Tuy nhiên, Azure chỉ tính phí phần xử lý theo số giờ máy ảo thực sự hoạt động; khi không cần trình diễn, máy ảo được giải phóng tài nguyên (deallocate) nên phần phí xử lý trở về không, chỉ còn phí lưu trữ đĩa và địa chỉ mạng tĩnh ở mức thấp, khoảng 5–9 đô-la Mỹ mỗi tháng.

Thành phần	Chi phí hằng tháng
Máy ảo Azure (đang chạy)	Khoảng 62 đô-la Mỹ nếu chạy liên tục 24/7
Máy ảo Azure (đã giải phóng)	Khoảng 5–9 đô-la Mỹ (chỉ phí đĩa và địa chỉ mạng tĩnh)
Phía giao diện (Vercel)	Miễn phí (gói Hobby)
Tên miền tdtuoj.me	Miễn phí năm đầu, khoảng 20 đô-la Mỹ mỗi năm sau đó
Tích hợp liên tục (GitHub Actions)	Miễn phí

Bảng 6.2: Chi phí vận hành các thành phần triển khai

Nhìn chung, việc triển khai cho thấy hệ thống TDTUOJ không chỉ hoạt động được trong môi trường phát triển mà còn vận hành ổn định trên môi trường trực tuyến thực tế với chi phí hợp lý, đồng thời có thể tái dựng nhanh chóng nhờ mô hình đóng gói bằng vùng chứa.

CHƯƠNG 7. KẾT LUẬN

7.1 Kết quả đạt được

Đề tài đã xây dựng hoàn chỉnh hệ thống chấm bài trực tuyến TDTUOJ phục vụ việc giảng dạy và rèn luyện lập trình tại Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng. Hệ thống bao phủ trọn vẹn luồng nghiệp vụ của một nền tảng chấm bài: quản lý kho bài toán với phân quyền công khai – riêng tư, nộp bài và chấm tự động bất đồng bộ qua Judge0 với bốn ngôn ngữ lập trình, tổ chức cuộc thi theo thể thức ICPC kèm bảng xếp hạng thời gian thực, cơ chế đóng băng bảng xếp hạng và tính điểm đánh giá năng lực sau cuộc thi, cùng mô hình tổ chức – bài thực hành phục vụ quản lý lớp học của giảng viên.

Bên cạnh các chức năng cốt lõi, đề tài đóng góp hai nhóm tính năng hỗ trợ học tập có tính đặc thù. Thứ nhất là trình trực quan hóa cấu trúc dữ liệu: mã nguồn của người dùng được biến đổi để tự ghi vết khi thực thi, kết hợp mô hình ngôn ngữ lớn phân loại vai trò ngữ nghĩa của biến, cho phép minh họa từng bước chạy của thuật toán trên sáu ngôn ngữ lập trình. Thứ hai là lớp hỗ trợ học tập bằng LLM: gợi ý hướng giải có kiểm soát không tiết lộ lời giải, trích xuất đề bài tự động từ tập tin PDF và sinh trường hợp kiểm thử, giúp giảm đáng kể công sức soạn đề của giảng viên.

Về mặt kỹ thuật, hệ thống được thiết kế theo kiến trúc phân tầng với xác thực phi trạng thái bằng JWT, hàng đợi chấm bài và bộ nhớ đệm trên Redis, giới hạn tần suất truy cập theo thuật toán thùng token, cùng hạ tầng giám sát bằng Prometheus và Grafana. Chất lượng được kiểm chứng qua 101 trường hợp kiểm thử đơn vị cho tầng nghiệp vụ và các kịch bản kiểm thử tải xác nhận hệ thống vận hành ổn định dưới lưu lượng cao.

Ngoài ra, hệ thống đã được triển khai thành công lên môi trường trực tuyến thực tế và truy cập được công khai qua tên miền riêng với kết nối mã hóa HTTPS. Toàn bộ phần phía máy chủ được đóng gói bằng vùng chứa và điều phối bằng Docker Compose trên một máy ảo đám mây, phía giao diện được phân phối trên Vercel, kèm

quy trình tích hợp và triển khai liên tục (CI/CD) tự động cập nhật hệ thống sau mỗi thay đổi mã nguồn. Việc triển khai chứng tỏ hệ thống có thể vận hành thực tế với chi phí hợp lý nhờ tận dụng các gói dịch vụ miễn phí dành cho sinh viên.

7.2 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống còn một số hạn chế. Thứ nhất, các phép đo hiệu năng được thực hiện trên một máy tính cá nhân nơi mọi thành phần chia sẻ tài nguyên, nên chưa phản ánh năng lực thực tế trong môi trường triển khai phân tán. Thứ hai, độ phủ kiểm thử đơn vị mới tập trung ở các phân hệ nghiệp vụ cốt lõi, các thành phần ngoại vi như tầng điều khiển và trình minh họa chưa được kiểm thử ở mức đơn vị. Thứ ba, kết quả chấm bài được phía giao diện truy vấn định kỳ (polling) thay vì đẩy chủ động, gây độ trễ nhỏ và lưu lượng dư thừa khi nhiều người cùng chờ kết quả. Cuối cùng, các tính năng phụ thuộc LLM chịu ràng buộc về chi phí và hạn mức gọi dịch vụ bên ngoài, đồng thời chất lượng phản hồi chưa thể bảo đảm tuyệt đối trong mọi tình huống.

7.3 Hướng phát triển

Trong thời gian tới, hệ thống có thể được phát triển theo một số hướng. Trước hết, từ nền tảng đã được triển khai trực tuyến trên một máy ảo đám mây, có thể tách các thành phần ra nhiều máy chủ riêng biệt, đặc biệt là cụm dịch vụ chấm bài, nhằm nâng cao khả năng chịu tải và đánh giá lại hiệu năng trong điều kiện phân tán thực tế. Về chức năng, các hướng mở rộng gồm: thay cơ chế truy vấn định kỳ bằng kết nối WebSocket để đẩy kết quả chấm theo thời gian thực; bổ sung ngôn ngữ lập trình mới vốn đã được Judge0 hỗ trợ sẵn; xây dựng cơ chế phát hiện sao chép mã nguồn giữa các bài nộp trong cuộc thi; và tích hợp với hệ thống quản lý học tập của nhà trường để đồng bộ danh sách lớp và điểm số. Về chất lượng, việc mở rộng độ phủ kiểm thử sang tầng điều khiển và bổ sung kiểm thử tích hợp tự động sẽ giúp hệ thống sẵn sàng hơn cho vận hành lâu dài.

TÀI LIỆU THAM KHẢO

- Docker Inc. (2024). Docker documentation [Truy cập ngày 01/06/2026]. <https://docs.docker.com>
- Došilović, H. Z. (2024). Judge0: Open-source online judge system [Truy cập ngày 01/06/2026]. <https://judge0.com>
- Grafana Labs. (2024a). Grafana documentation [Truy cập ngày 01/06/2026]. <https://grafana.com/docs>
- Grafana Labs. (2024b). K6 – load testing for engineering teams [Truy cập ngày 01/06/2026]. <https://grafana.com/docs/k6>
- Meta Platforms, Inc. (2024). React – a javascript library for building user interfaces [Truy cập ngày 01/06/2026]. <https://react.dev>
- Redis Ltd. (2024). Redis documentation [Truy cập ngày 01/06/2026]. <https://redis.io/docs>
- The PostgreSQL Global Development Group. (2024). Postgresql 16 documentation [Truy cập ngày 01/06/2026]. <https://www.postgresql.org/docs/16/>
- Walls, C. (2016). *Spring boot in action*. Manning Publications.
- Wasik, S., Antczak, M., Badura, J., Laskowski, A., & Sternal, T. (2018). A survey on online judge systems and their applications. *ACM Computing Surveys*, 51(1), 1–34.